XenFlo study guide

How the Knowledge Builder works, from a pasted URL to a saved knowledge base · 12 chapters, 89 pages

Contents

00 · The map: XenFlo in one page 2
What XenFlo does, from pasting a URL to saving a knowledge base.

01 · The tech stack and the concepts underneath it 9
What each tool in XenFlo is, why it was picked, and the 1–2 files where you'll see it most.

02 · The data model: one object, two schemas, five tables 17
The `KnowledgeBase` object: every section, and the `Field` wrapper that makes every value traceable.

03 · The scrape flow: from a pasted URL to a knowledge base 27
Every step between pressing **Scrape** and seeing tabs, in the order the code runs them.

04 · The extractors: how facts are pulled out of HTML 37
The order every page goes through, and why structured data runs first.

05 · AI enrichment: how "Enrich with AI" stays honest 46
The full path of one Enrich with AI run, from the button to accepted values in the knowledge base.

06 · Menus: PDFs, pictures, prices and brands 54
How a menu travels from a link on a page to priced offerings under the right restaurant.

07 · The UI: components, context and one click traced end to end 62
The component tree of both pages, and where the knowledge base lives while you edit.

08 · Saving, versions, re-scrape and the saved list 69
Exactly what happens when you press Save: the request, the database function, the new version, and the 409.

09 · Testing and deploying 75
How the tests are organized, how to run them, and three real tests explained line by line.

10 · Glossary 81
Every technical term used in this guide, in plain words.

11 · Simplify later: every "could be simpler" note, ranked 87
Every complexity, duplication, dead code and real bug the guide flagged, in one list.

Suggested sessions: 1) 00–02 · 2) 03–04 · 3) 05–06 · 4) 07–09, with 10 and 11 as reference.

00 · The map: XenFlo in one page

What you'll learn

1. What XenFlo does, from pasting a URL to saving a knowledge base.
2. The three tiers (browser, server, database) and how they talk to each other.
3. Where every folder lives, and where to look when something goes wrong.

Before this: nothing. This is the starting point.

---

1. The app in five sentences

1. The owner pastes their website address into the **/knowledge** page.
2. A server-side **scraper** (a program that downloads web pages and pulls facts out of them) reads up to 15 pages of the site, politely and within the site's rules.
3. Everything it finds lands in one big object called the **knowledge base**: company facts, contact details, brand colors, people, offerings, testimonials and more. Every single value remembers *where it came from* and *how sure we are*.
4. The owner reviews and edits it in tabs. Optionally, they press **Enrich with AI** or **Read menus with AI** to fill gaps, but AI only *suggests*, and the owner accepts or rejects.
5. **Save** writes it to the database as a new **version**, and **/knowledge/view** lists everything saved.

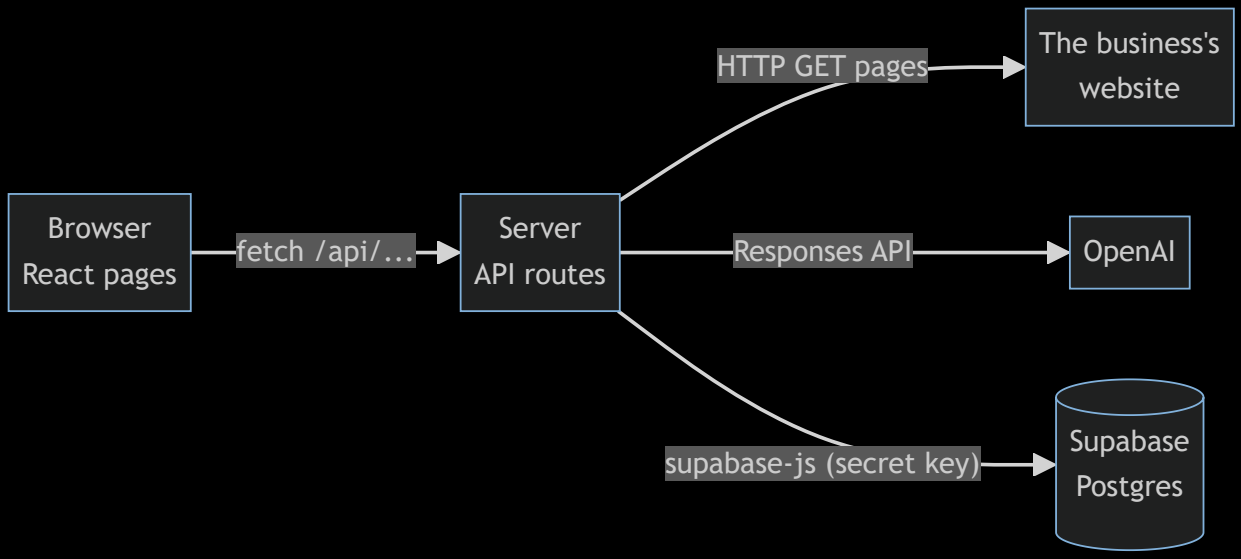
That knowledge base is what MoFlo's apps (MoSocial, MoMail, MoBlogs, MoReviews) would read so that "Flo", the AI writer, sounds like the business. The footer says it in one line (`src/components/knowledge/FooterNote.tsx`): "*Everything here powers Flo.*"

---

2. The three tiers

A **tier** here just means "a place where code runs". XenFlo has three:

Tier	Where it runs	What lives there	Folder
Browser (client)	The owner's laptop or phone	Pages, tabs, buttons, editing, the in-memory knowledge base	`src/app/knowledge/*`, `src/components/*`, `src/context/*`
Server (API routes)	Vercel's servers (or `npm run dev` on your PC)	Scraping, AI calls, database writes: anything that needs secrets or is slow	`src/app/api/*`, `src/lib/scraper`, `src/lib/ai`, `src/lib/db`
Database	Supabase (hosted Postgres)	Saved knowledge bases, version history, AI cache, daily AI counter	`supabase/migrations/*`



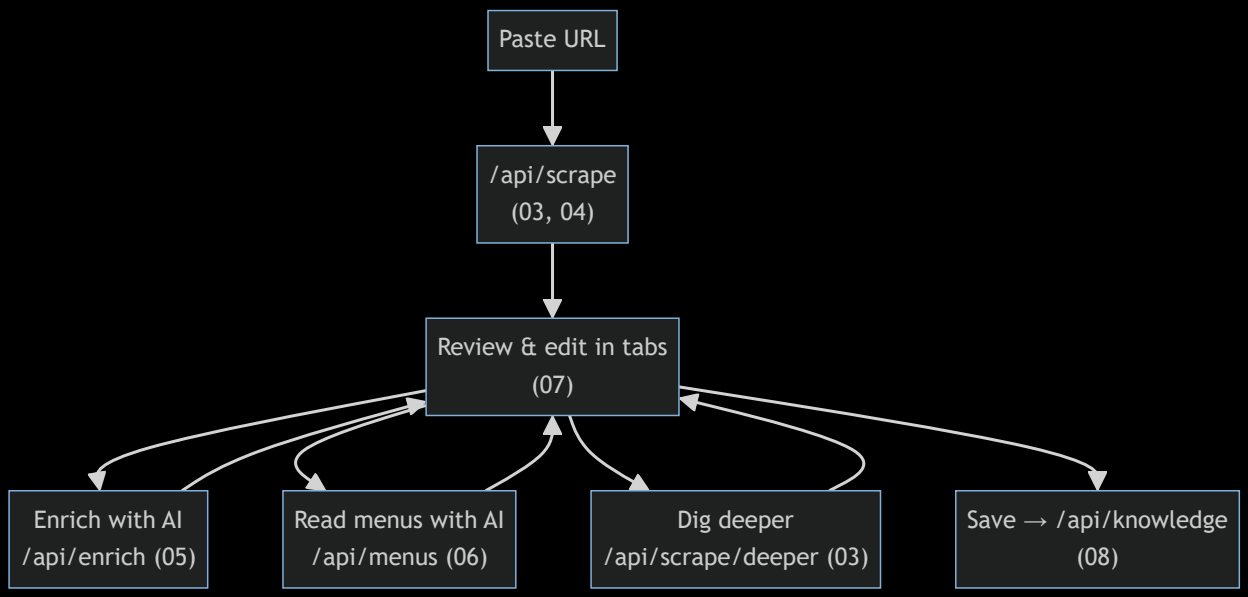
> **Why it's built this way:** the browser never talks to Supabase or OpenAI directly. Both need secret keys, and anything shipped to the browser can be read by anyone (open DevTools and it's there). So the browser only calls *our own* API routes, and those routes hold the keys. You'll see this rule in `src/lib/db/client.ts:1-4`: "SERVER ONLY: it uses the secret (service role) key... never import this from a client component."

The browser side of every call lives in one file, `src/lib/api/client.ts`. Its header says every call "resolves to either `{ data }` or `{ error: { code, message } }` and never throws", so a component never has to wrap a call in try/catch.

---

3. The main journeys

There are four journeys. Each later chapter zooms into one of them.



---

4. Folder tree with one line per job

This is the real tree, grouped. Test files (`*.test.ts`) sit next to the file they test and are covered in chapter 09.

`src/app`: pages and API routes (Next.js **App Router**: the folder path *is* the URL)

```
src/app/
├─ layout.tsx            Root HTML: loads the Poppins font, wraps everything in <Providers>
├─ providers.tsx         Client-side wrapper: Framer Motion "reduce motion" setting
├─ globals.css           Tailwind v4 + design tokens (colors, radius, shadows)
├─ page.tsx              "/" just redirects to /knowledge
├─ knowledge/
│  ├─ layout.tsx         Puts the sidebar + top bar (AppShell) around both pages
│  ├─ page.tsx           /knowledge: the builder (scrape, review, edit, save)
│  └─ view/page.tsx      /knowledge/view: the saved list (cards, table, detailed)
└─ api/                  Server code, one folder per URL
   ├─ scrape/route.ts            POST: crawl a site → knowledge base (not saved)
   ├─ scrape/deeper/route.ts     POST: Dig deeper, 15 more pages
   ├─ enrich/route.ts            POST: Enrich with AI (text + vision)
   ├─ menus/route.ts             POST: Read menus with AI
   ├─ extract/route.ts           POST: pasted text / HTML through the same extractors
   ├─ uploads/route.ts           POST: screenshot upload to private Storage
   └─ knowledge/…                Save, list, load, delete, versions, restore
```

`src/lib`: the brains (plain TypeScript, no UI)

```
src/lib/
├─ scraper/              Everything about reading a website (chapters 03, 04, 06)
│  ├─ index.ts           scrapeSite() and digDeeper(): the crawl recipe
│  ├─ crawl.ts           Worker pool, adaptive "crawl for missing fields", legal page
│  ├─ discover.ts        Find links (nav + sitemap), categorize and rank them
│  ├─ focus.ts           After 30 pages, only high-value links
│  ├─ fetch.ts           Download a page or file with timeouts, size caps, SSRF checks
│  ├─ safety.ts          SSRF protection: refuse private/internal addresses
│  ├─ robots.ts          Read robots.txt, decide what we may fetch
│  ├─ url.ts             Clean/normalize URLs, "same site?" checks, pageKey()
│  ├─ merge.ts           setField()/addItem(): which value wins
│  ├─ score.ts           Knowledge Health score (weights add to 100)
│  ├─ styles.ts colors.ts  Download CSS, parse colors (hex, rgb, oklch)
│  ├─ address.ts         US street address finder (text or maps link)
│  ├─ entity-suffixes.ts LLC / Inc / Corp → entity type, platform names
│  ├─ content.ts text-structure.ts  Pasted text/HTML → same extractors (upload fallback)
│  ├─ errors.ts          Typed scraper errors (INVALID_URL, TIMEOUT, BLOCKED_ROBOTS…)
│  ├─ extract/           One file per kind of fact (chapter 04)
│  └─ menus/             PDF menus: download, read text, parse prices, group by brand (06)
├─ ai/                   Everything about OpenAI (chapters 05, 06)
│  ├─ enrich.ts          THE single entry point for Enrich with AI
│  ├─ openai.ts          One HTTP call to the Responses API
│  ├─ prompts.ts         Loads prompts/*.md into system prompts
│  ├─ schemas.ts         Zod shapes the model must answer in
│  ├─ evidence.ts input.ts   What the model reads (page evidence, images)
│  ├─ confidence.ts      "High confidence + 2 evidence items" rule in code
│  ├─ field-tiers.ts     Tier 1/2/3 per field + filterByTier()
│  ├─ apply.ts           Accept suggestions / "Wrong? Remove" (pure functions)
│  ├─ mock.ts mockPreview.ts heuristics.ts  Preview mode without AI
│  ├─ menus.ts read-menus.ts  Read menus with AI (rules + network side)
│  └─ config.ts          Env vars, token budgets, daily limit
├─ db/                   The ONLY place that knows we use Supabase (chapter 02, 08)
│  ├─ client.ts          Creates the server Supabase client (secret key)
│  ├─ knowledge.ts       create / get / list / update / delete / versions
│  ├─ ai.ts              AI cache + daily quota (with in-memory fallback)
│  ├─ storage.ts         Screenshot files in a private bucket
│  └─ errors.ts types.ts Error codes (NOT_FOUND, CONFLICT…) and row shapes
├─ api/                  Browser-side: client.ts (fetch wrappers), messages.ts (friendly errors)
├─ utils/                Small shared helpers: field(), path get/set, overlay (re-scrape),
│                        quick-picks, offerings, logo choice, time ago, cn() for classes
└─ view/filters.ts       /knowledge/view keeps its filters in the URL
```

`src/components`, `src/context`, `src/types`

```
src/components/
├─ shell/        Sidebar, TopBar, AppShell (the frame around every page)
├─ knowledge/    The builder: workspace, scrape bar/progress, results, tabs/,
│                ai/ (EnrichModal), offerings/ (menus), fallback/ (blocked/upload)
├─ view/         The saved list: cards, table, detailed view, bulk bar, versions drawer
├─ ui/           Reusable pieces: Button, Card, Badge, EditableField, Modal, Menu, Gauge…
└─ tour/         First-visit spotlight tour
src/context/
├─ KnowledgeContext.tsx  The knowledge base in memory + every edit action (chapter 07)
└─ NavGuardContext.tsx   "You have unsaved changes" guard when leaving
src/types/
├─ knowledge.ts          THE data model (chapter 02)
├─ knowledge.schema.ts   The same model as a Zod schema (runtime check)
└─ enrichment.ts         Shapes for AI suggestions and menu results
```

Outside `src/`

```
prompts/              Versioned AI prompts (.md). In use: understand-business.v1,
                      logo-vision.v1, menu-reader.v2 (v1 kept for its cache).
                      company-pitch, writing-style, ideal-persona, content-kit = reference only
docs/                 data-quality.md, enrichment.md, schema.md, improvements.md
supabase/migrations/  SQL that builds the database, in date order (5 files)
data/examples/        Real scrape outputs: apex-hosting, dragon-factory, goettl-las-vegas
scratch/              Your notes and test runs (gitignored, never deployed)
```

> **Could be simpler:** four of the eight prompt files (`company-pitch`, `writing-style`, `ideal-persona`, `content-kit`) are never loaded by code. `prompts/README.md` says they "stay here as reference". That's fine for the challenge deliverable ("at least 3 enrichment prompts"), but a reader can easily think they're live. Moving them into `prompts/reference/` would make that obvious.

---

5. "Where do I look when…"

Symptom	Start here	Then
A field is wrong right after scraping	`src/lib/scraper/extract/<kind>.ts` (see chapter 04)	`src/lib/scraper/merge.ts` (which value won)
The wrong pages were crawled, or too few	`src/lib/scraper/discover.ts` (categories, scores)	`crawl.ts`, `focus.ts`, `score.ts` `FIELD_HINTS`
"Blocked by robots.txt" seems wrong	`src/lib/scraper/robots.ts`	`src/components/knowledge/fallback/BlockedPanel.tsx`
A scrape fails with a private-address error	`src/lib/scraper/safety.ts`	`fetch.ts` `openChecked`
The phone/address belongs to another branch	`extract/contact.ts` (`locationCity`, `cityMatch`), `extract/address.ts`	`src/lib/scraper/address.ts`

Symptom	Start here	Then
Brand colors look like WordPress defaults	`extract/branding.ts`	`colors.ts`, `styles.ts`
AI suggested something weird	the prompt in `prompts/`, `src/lib/ai/schemas.ts`	`confidence.ts`, `field-tiers.ts` `filterByTier`
AI said "preview" instead of live	`src/lib/ai/config.ts` (`liveAvailable`)	`src/lib/db/ai.ts` (daily quota)
Menu prices wrong or missing	`src/lib/scraper/menus/parse.ts`	`src/lib/ai/menus.ts`, `prompts/menu-reader.v2.md`
Menu items under the wrong restaurant	`src/lib/scraper/menus/organize.ts`	`menus/brands.ts`, `menus/group.ts`
Health score seems off	`src/lib/scraper/score.ts`	`kb.notApplicable` (counts as done)
Save fails with "Someone saved a newer version"	`src/lib/db/knowledge.ts` `updateKnowledgeBase`	migration `20261009130000_version_conflict_code.sql`
A saved record won't open ("unexpected format")	`src/types/knowledge.schema.ts`	`src/lib/utils/http.ts` (ZodError → 500)
A button does nothing / edits don't stick	`src/context/KnowledgeContext.tsx`	the component calling it
Edits lost after re-scrape	`src/lib/utils/overlay.ts` `keepUserEdits`	chapter 08 table
An error message is confusing	`src/lib/api/messages.ts`	`src/lib/utils/http.ts` (status codes)

Check yourself

1. Name the three tiers and say which one is allowed to hold the OpenAI key.
2. Why does the browser call `/api/knowledge` instead of talking to Supabase directly?
3. In the App Router, what URL does `src/app/api/scrape/deeper/route.ts` answer?
4. Which single folder would you rewrite to swap Supabase for another database?
5. A customer says "the address shown is our other location". Which two files do you open first?
6. Which prompt files are actually loaded by code today?
7. What does `src/lib/api/client.ts` return instead of throwing an error?

Answers

1. Browser, server (API routes), database. Only the server holds the OpenAI key (`src/lib/ai/config.ts` reads it from `process.env`).
2. Supabase writes need the secret key, and anything in the browser is public. So only server routes use `src/lib/db/client.ts`.
3. `POST /api/scrape/deeper`. The folder path becomes the URL.
4. `src/lib/db/` (its header: "swapping databases means rewriting this folder and nothing else").
5. `src/lib/scraper/extract/contact.ts` (branch phone logic) and `extract/address.ts` (main vs other locations).

6. `understand-business.v1`, `logo-vision.v1`, `menu-reader.v2`, and `menu-reader.v1` (only for its cached answers). See `src/lib/ai/prompts.ts:11-16`.
7. An object that's either `{ data }` or `{ error: { code, message } }`.

01 · The tech stack and the concepts underneath it

> **What you'll learn**
>
> 1. What each tool in XenFlo is, why it was picked, and the 1–2 files where you'll see it most.
> 2. The core ideas the code leans on: server vs client components, API routes, env vars, JSON, schemas, async/await.
> 3. How to read a Next.js route file and a client component without getting lost.
>
> **Before this:** 00-map.md

Part A: the tools

Next.js 15 (App Router)

Next.js is a framework on top of React. React draws the UI; Next.js adds the parts a real website needs: URLs (routing), a server, building for production, and deploying to Vercel. The **App Router** is Next's folder-based routing: a folder under `src/app` *is* a URL path.

- `page.tsx` = the page shown at that path.
- `layout.tsx` = a frame around every page below it.
- `route.ts` = an **API route** (server code that answers HTTP requests, no UI).

Why here: the challenge requires it, and it gives you the browser and the server in one project. The scraper has to run on a server (browsers can't read other sites because of CORS, explained in the glossary), so having API routes next to the pages is ideal.

See it in: `src/app/knowledge/page.tsx` (a page) and `src/app/api/scrape/route.ts` (an API route).

React 19

React builds the UI out of **components**: functions that return markup (JSX) and re-run when their **state** changes. **Hooks** (`useState`, `useEffect`, `useContext` …) are how a component keeps state and reacts to changes. The project rule is "state: React hooks and context only", so there's no Redux or Zustand. The shared state lives in one **Context** (chapter 07).

See it in: `src/components/knowledge/KnowledgeWorkspace.tsx` (state and effects), `src/context/KnowledgeContext.tsx` (context).

TypeScript (strict)

TypeScript is JavaScript plus **types**, labels that say what shape a value has. The editor and `npx tsc --noEmit` check them before the code runs. **Strict mode** (`tsconfig.json:7`, `"strict": true`) turns on every check, most importantly "this might be `null`, handle it".

Why here: the knowledge base has around 60 fields, each wrapped in metadata. Without types, a typo like `kb.company.yearFouned` would only show up as a blank screen at runtime. `tsconfig.json:21-23` also sets up the `@/` shortcut: `@/lib/scraper` means `src/lib/scraper`.

See it in: `src/types/knowledge.ts`, which is the whole data model as types.

Tailwind CSS v4

Tailwind styles elements with small class names right in the markup (`rounded-2xl p-6 text-sm`) instead of separate CSS files. Version 4 reads **design tokens** from a `@theme` block in CSS. `src/app/globals.css:8-40` defines `--color-primary: #2563eb`, and from that Tailwind creates `bg-primary`, `text-primary` and `border-primary`.

Why here: fast to match MoFlo's look (light gray page, white rounded cards, blue #2563EB) and keep it consistent, because every color comes from one token list.

See it in: `src/app/globals.css`, `src/components/ui/Button.tsx`.

Framer Motion

Framer Motion is an animation library for React: `<motion.div initial={…} animate={…}>` animates between two states. `<AnimatePresence>` lets things animate *out* before they're removed.

Why here: the brief asks for staggered dropdowns, modals that fade and float in, and a spotlight tour. `src/app/providers.tsx:5-7` wraps the app in `MotionConfig reducedMotion="user"`, so people who asked their OS for less motion get none. That's an accessibility default.

See it in: `src/components/ui/Menu.tsx:118-151` (stagger), `src/components/tour/Tour.tsx`.

Cheerio

Cheerio loads an HTML string and lets you query it with jQuery-style selectors on the server: `$("footer").text()`, `$('a[href^="tel:"]')`. It does **not** run JavaScript or render anything, so it's fast and safe.

Why here: the scraper needs to read HTML on the server. A real headless browser (a hidden Chrome that runs each page's JavaScript) is far heavier, slower to start, and hard to fit inside a 60-second serverless function.

- **The tradeoff:** sites that build their content with JavaScript (some Wix pages) can look empty to Cheerio.
- **How that's handled:** `src/lib/scraper/index.ts` has `MIN_WORDS = 30` to detect a "JS-only shell" homepage.

See it in: `src/lib/scraper/extract/index.ts` (`cheerio.load(html)`), any extractor.

Zod

Zod describes the shape of data at **runtime** and checks real data against it. TypeScript types disappear when the code runs; Zod schemas don't. `schema.safeParse(x)` returns either `{ success: true, data }` or `{ success: false, error }`.

Why here: anything coming from *outside* can't be trusted just because TypeScript says so. "Outside" means a browser request, a database row or an OpenAI reply. Zod is the checkpoint at each of these doors (more in chapter 02).

See it in: `src/types/knowledge.schema.ts`, `src/lib/ai/schemas.ts`.

Supabase (Postgres)

Supabase is hosted **Postgres** (a SQL database) plus extras: an auto-generated HTTP API (**PostgREST**), file **Storage**, auth, and a dashboard. The app talks to it through the `@supabase/supabase-js` library.

Why here: the challenge requires it, and Postgres's **JSONB** column type can store the whole knowledge base as one JSON document while still allowing normal columns for search and filters.

See it in: `src/lib/db/client.ts`, `src/lib/db/knowledge.ts`, `supabase/migrations/`.

OpenAI Responses API

The **Responses API** is OpenAI's current HTTP endpoint for calling models (`POST https://api.openai.com/v1/responses`). XenFlo calls it with plain `fetch` (no SDK) in exactly one place:

```
// src/lib/ai/openai.ts:36-52
const res = await fetch("https://api.openai.com/v1/responses", {
  method: "POST",
  signal: controller.signal,
  headers: { authorization: `Bearer ${key}`, "content-type": "application/json" },
  body: JSON.stringify({
    model: opts.model,
    // No hidden reasoning: reasoning tokens count toward max_output_tokens, and these are
    // summarize-and-format tasks where the whole budget should go to the answer.
    reasoning: { effort: "none" },
    max_output_tokens: opts.maxOutputTokens ?? LIMITS.outputTokens,
    store: false,
    input: [
      { role: "system", content: opts.system },
      { role: "user", content: opts.content },
    ],
    text: { format: { type: "json_schema", name: opts.schemaName, strict: true, schema:
toOpenAiSchema(opts.schema) } },
  }),
});
```

Why here: AI is optional in this project; the app works fully in preview mode. When it is used, `json_schema` with `strict: true` (**structured outputs**) forces the model to answer in the exact shape the code expects. Chapter 05 goes deep.

unpdf

unpdf extracts text from PDFs. It's a serverless build of Mozilla's pdf.js: pure JavaScript, no native programs.

Why here: restaurant menus are often PDFs, and Vercel functions can't install system tools. `src/lib/scraper/menus/pdf.ts:1-5` says exactly this. Picture-only PDFs come back empty and go to the AI menu reader instead (chapter 06).

Vitest

Vitest is the test runner: `npm test` runs every `src/**/*.test.ts`. Its config, `vitest.config.mts`, copies the `@/` alias from `tsconfig.json` so tests can import like the app does. Chapter 09 walks through three tests.

Vercel

Vercel hosts the app. It builds it on every push to GitHub and runs each API route as a **serverless function**, a small program started on demand. On the Hobby plan, a function may run at most **60 seconds**, which is why you'll see `export const maxDuration = 60` in route files and "stop after 30/40 seconds" budgets throughout the code.

Part B: the concepts

1. Server components vs client components

In the App Router, components run on the **server** by default: they render to HTML and send no JavaScript for themselves. A file that starts with `"use client"` is a **client component**: its code ships to the browser so it can use state, effects and click handlers.

- `src/app/knowledge/page.tsx` has **no** `"use client"`. It's a server component that only arranges other components and sets the page title (`metadata`).
- `src/components/knowledge/KnowledgeWorkspace.tsx:1` starts with `"use client"` because it uses `useState`, `useEffect` and click handlers.

62 files in `src/` are client components. Rule of thumb: anything interactive is a client component, and anything that needs secrets must **never** be one.

> **Why it's built this way:** keeping secrets and heavy libraries (Cheerio, unpdf) out of client components means they never reach the browser. `src/lib/db/client.ts` is only imported by server code for this reason.

2. API routes

A `route.ts` file exports functions named after HTTP methods. Here's a whole one:

```
// src/app/api/scrape/route.ts:7-32
// Cheerio and long-running fetches need the Node runtime, not Edge.
export const runtime = "nodejs";
export const maxDuration = 60;

const bodySchema = z.object({
  url: z.string().min(1).max(2048),
  maxPages: z.number().int().min(1).max(30).optional(),
  ownerConsent: z.literal(true).optional(),
});

export async function POST(request: Request) {
  const parsed = bodySchema.safeParse(await request.json().catch(() => null));
  if (!parsed.success) {
    return badRequest("INVALID_URL", "Send JSON like { \"url\": \"example.com\" }.");
  }
  try {
    const kb = await scrapeSite(parsed.data.url, { maxPages: parsed.data.maxPages, ownerConsent:
parsed.data.ownerConsent });
    return NextResponse.json({ knowledgeBase: knowledgeBaseSchema.parse(kb) });
  } catch (err) {
    return errorResponse(err);
  }
}
```

Read it top to bottom:

1. `runtime = "nodejs"`: run on full Node.js, not the lighter "Edge" runtime, because Cheerio and long downloads need Node.
2. `maxDuration = 60`: Vercel may give this route up to 60 s.
3. **Validate the request body** with Zod. If it's bad, answer 400 right away.
4. **Do the work**, then **validate our own output** before sending it ("schema drift shows up immediately").
5. **Any thrown error** becomes a JSON error with the right HTTP status (`src/lib/utils/http.ts:41-54`).

Every route in `src/app/api` follows this same shape.

> **Could be simpler:** `maxPages` here is still capped at `max(30)` (line 13), a leftover from the old 30-page hard max. Nothing in the UI sends `maxPages`, so it's harmless, but it now disagrees with `MAX_CRAWL_PAGES`.

3. Environment variables and secrets

Environment variables ("env vars") are settings passed to a program from outside its code. Locally they live in `.env.local`, which is never committed; on Vercel you set them in the dashboard. Code reads them with `process.env.NAME`.

- Names starting with `NEXT_PUBLIC_` are copied into the browser bundle, so they're public by design. The Supabase URL is one.
- Everything else is **server-only**: `SUPABASE_SERVICE_ROLE_KEY`, `OPENAI_API_KEY`, `AI_PASSCODE`.

`.env.example` lists them all with blank values (chapter 09 has the full table). The code reads them in two places: `src/lib/ai/config.ts` and `src/lib/db/client.ts:13-14`.

> **Could be simpler:** `.env.example` lists `NEXT_PUBLIC_SUPABASE_ANON_KEY`, but no code reads it (search `src/` for `ANON_KEY`: zero hits). The server uses only the secret key. It's harmless, but it's a dead setting.

4. JSON

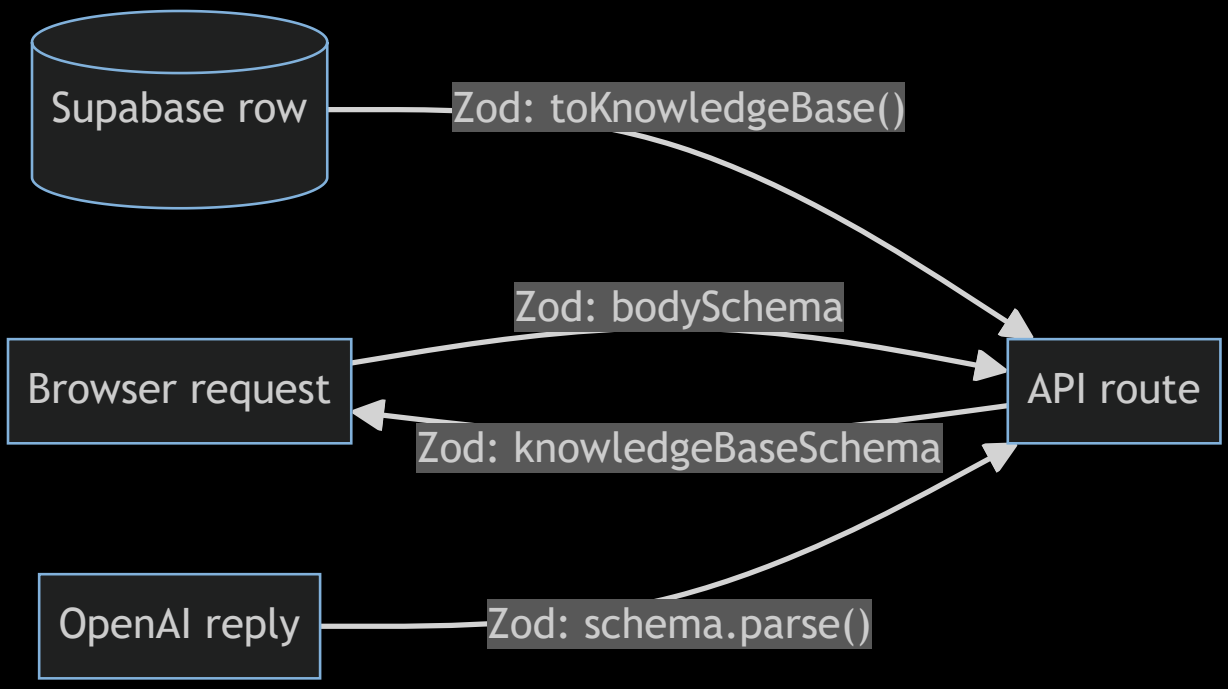
JSON (JavaScript Object Notation) is the text format for data on the wire: `{"name": "Apex", "pages": 15}`.

- `JSON.stringify(obj)` turns an object into text.
- `JSON.parse(text)` turns text back into an object.

Every API request and response here is JSON, the knowledge base is stored as JSON in Postgres (JSONB), and the AI answers in JSON.

5. Schemas and validation

A **schema** is a description of the allowed shape of data. **Validation** is checking real data against it. XenFlo uses one rule: *validate at every border*.



Each arrow is a real `parse` call:

- `src/app/api/scrape/route.ts:20` and `:28` (request and response)
- `src/lib/db/knowledge.ts:54-56` (database row)
- `src/lib/ai/openai.ts:64-65` (OpenAI reply)

6. async / await and promises

A **promise** is an object for "a value that will exist later", like a download that hasn't finished. `await` pauses *this function* until the promise settles, without freezing the whole program. An `async` function always returns a promise.

```
// from src/app/api/scrape/route.ts:26
const kb = await scrapeSite(parsed.data.url, { ... });
```

7. Promises in parallel

`await` one-by-one is slow when the tasks don't depend on each other. `Promise.all([...])` starts them all at once and waits for all of them; `Promise.allSettled` waits for all of them even if some fail. XenFlo uses this in several places:

Where	What runs at the same time
`src/lib/scraper/crawl.ts:91`	2 crawl "workers" pulling pages from a shared queue
`src/lib/ai/enrich.ts:98`	the text AI call and the vision AI call (`allSettled`: one can fail and the other still counts)
`src/lib/ai/read-menus.ts:62` and `:81`	all cache lookups, then all menu reads
`src/lib/scraper/styles.ts:31`	downloading up to 3 stylesheets
`src/components/view/useRecordActions.ts:79`	loading several saved records for bulk export

The crawl one is a classic **worker pool**:

```
// src/lib/scraper/crawl.ts:91
await Promise.all(Array.from({ length: Math.max(1, session.concurrency) }, worker));
```

`concurrency` is 2, so this makes two copies of the same `worker` function. Each one loops "take the next link from the queue, fetch it, extract it" until the queue is empty. There are only two at a time to stay polite to small business websites.

> **Why it's built this way:** parallel AI calls are why one Enrich with AI run finishes in about 25 s instead of about 50 s, and that matters with a 60 s Vercel ceiling.

Part C: the honest progress bar

One thing that surprises people: the scrape progress steps you see ("Checking robots.txt", "Fetching homepage"…) are **not live**. `/api/scrape` answers once, at the end. `src/components/knowledge/ScrapeProgress.tsx:3-6` says so: it walks through the scraper's real stages "on a typical timeline", and the actual log is shown afterwards (`CrawlSummary`).

> **Could be simpler / later:** the scraper already has an `onProgress` callback (`src/lib/scraper/index.ts`, `ScrapeOptions.onProgress`) that nothing uses yet. Making progress real would mean **streaming** the response (sending log lines as they happen). It's a nice upgrade, but not needed for the demo.

Check yourself

1. What makes a file a client component, and why can't `src/lib/db/client.ts` be imported by one?
2. In a `route.ts`, what do `runtime = "nodejs"` and `maxDuration = 60` each mean?
3. Why validate data with Zod when TypeScript already has types?
4. Which env vars are safe to expose to the browser, and how can you tell from the name?
5. What's the difference between `Promise.all` and `Promise.allSettled`, and why does `enrich.ts` use `allSettled`?
6. Why does the crawl use exactly 2 workers instead of 10?
7. Is the scrape progress list live? Where does the real log appear?

Answers

1. A first line of `"use client"`. `db/client.ts` uses the Supabase **secret** key, and anything in a client component ships to the browser, where anyone could read the key.
2. `runtime = "nodejs"`: run on full Node (Cheerio and long downloads need it), not Edge. `maxDuration = 60`: Vercel may let this function run up to 60 seconds.
3. Types vanish at runtime. Data from a request, a database row or OpenAI can be any shape, and Zod checks it when the code actually runs.
4. Only `NEXT_PUBLIC_*` ones; that prefix makes Next.js copy them into the browser bundle.
5. `all` fails as soon as one promise fails; `allSettled` waits for every promise and reports each result. With `allSettled`, the vision call can fail and the text suggestions are still used.

6. To be polite to small business servers (along with delays and robots.txt), and to stay inside the time budget without hammering anyone.
7. No. It's a timed walkthrough of the real stages (`ScrapeProgress.tsx:3-6`). The real log is shown afterwards in `CrawlSummary`, and in full in the Sources tab.

02 · The data model: one object, two schemas, five tables

What you'll learn

1. The `KnowledgeBase` object: every section, and the `Field<T>` wrapper that makes every value traceable.
2. Confidence levels and the three tiers (read / inferred / never guessed), and why both exist.
3. How the same shape lives as a TypeScript type, a Zod schema and Postgres tables, plus RLS, versions and the 409 conflict.

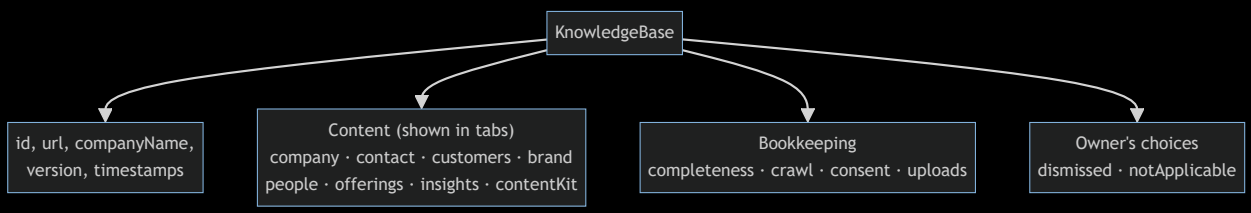
Before this: 00-map.md, 01-tech-stack.md (Zod, JSON, env vars)

---

1. One object to rule them all

Everything XenFlo knows about a business is **one JavaScript object** of type `KnowledgeBase`, defined in `src/types/knowledge.ts:352-375`. The file's header calls it "the single source of truth for the whole app". The scraper fills it, the tabs display it, AI adds to it, and the database stores it as one JSON document.

```
// src/types/knowledge.ts:352-375 (comments trimmed)
export interface KnowledgeBase {
  id: string;
  url: string;
  companyName: string;
  createdAt: string;
  updatedAt: string;
  version: number;
  company: CompanySection;
  contact: ContactSection;
  customers: CustomersSection;
  brand: BrandSection;
  people: FieldList<Person>;
  offerings: FieldList<Offering>;
  insights: InsightsSection;
  contentKit: ContentKitSection;
  completeness: Completeness;
  crawl: CrawlInfo;
  consent: Consent | null;
  uploads: UploadRecord[];
  dismissed: Dismissal[];
  notApplicable: string[];
}
```



2. `Field<T>`: every value carries its receipt

Here's the most important idea in the whole app. A value is never stored bare. It's wrapped:

```
// src/types/knowledge.ts:21-33
export interface Field<T> {
  value: T | null;
  /** URL (or "upload"/"user") the value came from. Null when missing. */
  source: string | null;
  confidence: Confidence;
  /** ISO timestamp */
  updatedAt: string;
  /** For AI and inferred values: the short quotes or page paths they're based on (shown in the badge
tooltip) */
  evidence?: string[];
  /** When the owner checked an AI value and kept it ("Mark as reviewed" on a menu). It is then
user_edited. */
  reviewedAt?: string;
}

export type FieldList<T> = Field<T>[];
```

- `<T>` is a **generic**, a type parameter. `Field<number>` is a field whose value is a number (like `yearFounded`), and `Field<Address>` holds an address object. One wrapper fits every kind of value.
- **`value: T | null`**: `null` means "we don't know". It's never `""` or `0` pretending to be data.
- **`source`**: the page URL it was read from, `"user"` when the owner typed it, or `"ai:<model>"` for AI.
- **`confidence`**: how it got here (next section).
- **`evidence`**: the quotes it's based on. For example, after Phase 11 the legal name `Apex Hosting LLC` carries the evidence `"© 2013-2026 Apex Hosting LLC"`.
- **`FieldList<T>`** is a list where *each item* has its own receipt. Two phone numbers can come from two different pages.

Two helpers create fields, and you'll see them everywhere:

```
// src/lib/utils/knowledge.ts:3-11
export function field<T>(value: T, source: string | null, confidence: Confidence): Field<T> {
  return { value, source, confidence, updatedAt: new Date().toISOString() };
}

export function missing<T>(): Field<T> {
  return { value: null, source: null, confidence: "missing", updatedAt: new Date().toISOString() };
}
```

`emptyKnowledgeBase(url)` (`src/lib/utils/knowledge.ts:14`) builds a blank one: every scalar is `missing()` and every list is `[]`. The scraper starts from that and fills it in.

> **Why it's built this way:** the brief says "Every extracted value records its source URL and confidence" and "Never invent facts. Unknown stays empty and marked Missing." Wrapping every value makes that impossible to forget. You *can't* store a value without saying where it came from, because the type won't allow it.

---

3. The sections, grouped

You don't need to memorize this. Treat it as a reference you can come back to. Types are in `src/types/knowledge.ts:153-223`; scalar = `Field<…>`, list = `FieldList<…>`.

Company (`CompanySection`, lines 153-171), the Company tab

Field	Kind	Note
name, overview, website, industry, businessModel, companyRole	scalar string	
yearFounded	scalar **number**	
legalEntityType, legalName, employeeCount, revenue	scalar string	`employeeCount` is a string because ranges ("11 to 50") and "168 technicians (Las Vegas)" aren't numbers
mainAddress	scalar `Address`	`{ street, city, region, postalCode, country, formatted }`
otherLocations	list `Address`	
serviceLocations, alternateNames	list string	
pitch, foundingStory	scalar string	

Contact (lines 173-177)

emails and phones (lists), and contactPageUrl (scalar).

Customers (lines 179-189)

targetBuyers, customerNeeds, industryGroupings, channels and funnels are string lists. idealPersona and industryOutlook are scalars. `ctas` is a list of `{ text, url }` and `suppliersPartners` is a list of `{ name, category }`.

Brand (lines 191-199)

writingStyle and artStyle (scalars); fonts and colors (string lists, colors as `"#rrggbb"`); logos (`{ url, kind, alt }`); socialLinks (`{ platform, url }`).

People and offerings (top-level lists)

- **`Person`** (lines 72-80): name, title, role, bio, imageUrl and **`type: "team" | "customer_partner"`**. That tag is how testimonial authors are kept off the team.
- **`Offering`** (lines 84-105): name, category, description, features, `pricingType` (`fixed`, `starting_at`, `range`, `subscription`, `quote`, `free`, `unknown`), priceText (the price exactly as shown), priceAmount and currency.

- The Phase 10 menu fields are `group` (brand), `location`, `sourceKind` and `foundOn`.

Insights (lines 201-214), an Advanced tab

testimonials, faqs, differentiators, trustSignals, contentThemes, promotions, pressMentions, communityValues, legalLinks, positioningSignals and seasonalMessaging. All of them are lists.

Content Kit (lines 216-223), an Advanced tab

contentPillars, socialHooks, hashtags, emailSubjects and blogIdeas are lists. voiceGuide is a scalar `{ wordsToUse, wordsToAvoid, dos, donts }`.

Bookkeeping

- **`completeness`** `{ score, missing[] }`: the Knowledge Health gauge (chapter 07).
- **`crawl`** (`CrawlInfo`, lines 303-316): pages crawled (each with title, status and trimmed evidence), the log, `pendingUrls` for Dig deeper, `menuSources` and `maxPages`.
- **`consent`**: `{ confirmed: true, timestamp, method }`, set when the owner ticks "I own this business..." (chapter 03).
- **`uploads`**: pasted text, HTML and screenshots.
- **`dismissed`**: values the owner removed with "Wrong? Remove", so AI won't suggest them again.
- **`notApplicable`**: field paths the owner marked N/A. They count as complete.

---

4. Confidence: how a value got here

```
// src/types/knowledge.ts:13-19
export type Confidence =
  | "scraped" // read directly from the website
  | "inferred" // derived by rules from scraped data (e.g. name from a logo file name)
  | "ai_mock" // placeholder AI output (no API key), labeled "AI preview"
  | "ai_live" // real AI output
  | "user_edited" // typed or changed by the user
  | "missing"; // we looked and found nothing
```

Each one becomes a colored badge (`src/components/ui/Badge.tsx:31-38`): Scraped is green, Inferred amber, AI preview and AI purple, User edited blue and Missing gray. Hovering a badge shows `Based on: • …evidence` or `Source: <url>` (line 47).

When two values compete for the same field, a **rank** decides:

```
// src/lib/scraper/merge.ts:10-17
const RANK: Record<Confidence, number> = {
  missing: 0,
  ai_mock: 1,
  inferred: 2,
  ai_live: 3,
  scraped: 4,
  user_edited: 5,
};
```

The owner always wins (5), then what the site literally says (4), then real AI (3), rules (2), placeholders (1), and nothing (0). Chapter 04 shows `setField` using this table.

---

5. The three tiers: what AI is *allowed* to fill

Confidence says how a value *got* here. The **tier** says who is *allowed* to fill it at all. This lives in `src/lib/ai/field-tiers.ts:1-14`:

Tier	Meaning	Examples	AI may…
1 Read	Stated facts the scraper extracts	name, address, phones, logos, prices, FAQs	never overwrite; fill an empty one only if the value appears **word for word** in the evidence
2 Inferred	Obvious to a human reader but not written as a label, plus writing grounded in it	industry, business model, channels, pitch, writing style, Content Kit	fill, with evidence, at high confidence only
3 Never guessed	People and private business facts	people, legal name, entity type, employee count, revenue, testimonial authors	**never**: any AI suggestion is dropped

The map is a plain object of dot paths:

```
// src/lib/ai/field-tiers.ts:21-33 (excerpt)
export const FIELD_TIERS: Record<string, Tier> = {
  "company.name": 1,
  "company.overview": 1,
  "company.website": 1,
  "company.industry": 2,
  "company.businessModel": 2,
  "company.companyRole": 2,
  "company.yearFounded": 1,
  "company.legalEntityType": 3,
  "company.legalName": 3,
  "company.employeeCount": 3,
  "company.revenue": 3,
  // …
```

And the guard that enforces it **in code**, not just in the prompt:

```
// src/lib/ai/field-tiers.ts:138-151
export function filterByTier(suggestions: Suggestion[], kb: KnowledgeBase): Suggestion[] {
  const dismissed = new Set((kb.dismissed ?? []).map((d) => `${d.path}|${d.key}`));
  const na = new Set(kb.notApplicable ?? []);
  return suggestions.flatMap((s) => {
    const tier = tierOf(s.path);
    if (tier === 3 || na.has(s.path)) return [];
    if (tier === 1) {
      const quoted = (v: unknown) => s.basedOn.some((e) => e.toLowerCase().includes(valueKey(v)));
      if (!isEmptyAt(kb, s.path) || !(s.list ? (s.value as unknown[]).every(quoted) : quoted(s.value)))
return [];
    }
    if (!s.list) return dismissed.has(`${s.path}|${valueKey(s.value)}`) ? [] : [s];
    const kept = (s.value as unknown[]).filter((v) => !dismissed.has(`${s.path}|${valueKey(v)}`));
    return kept.length ? [{ ...s, value: kept }] : [];
  });
}
```

In plain words: drop tier 3. Drop anything marked N/A. For tier 1, keep it only if the field is empty **and** the value is quoted in the evidence. Then drop anything the owner dismissed before.

`tierOf()` (lines 109-119) turns `"offerings.3.category"` into `"offerings.*.category"` before looking it up. **Unknown paths count as tier 3**, which is the safest default.

> **Why it's built this way:** a prompt is a *request*, not a guarantee. Models sometimes ignore instructions. The tier rule is written in code so that even if the model "helpfully" guesses an employee count, it never reaches the owner.

---

6. Two schemas for one shape: TypeScript and Zod

The same shape is written twice:

- `src/types/knowledge.ts`: **TypeScript types**, checked while you code and build, then erased.
- `src/types/knowledge.schema.ts`: **Zod schemas**, real objects that check data **at runtime**.

```
// src/types/knowledge.schema.ts:18-28
export function fieldSchema<T extends z.ZodType>(inner: T) {
  return z.object({
    value: inner.nullable(),
    source: z.string().nullable(),
    confidence: confidenceSchema,
    updatedAt: z.string(),
    evidence: z.array(z.string()).optional(),
    reviewedAt: z.string().optional(),
  });
}
```

Why both? TypeScript only checks *your* code. It can't check a JSON body a browser sends, a row from Postgres or a reply from OpenAI. Zod can. The schema also *repairs* old records: fields added later use `.default([])`, so a record saved before Phase 9 still loads:

```
// src/types/knowledge.schema.ts:297-299
  // Added in Phase 9
  dismissed: z.array(z.object({ path: str, key: str, at: str })).default([]),
  notApplicable: z.array(str).default([]),
```

How they're kept in sync: a compile-time trick at the bottom of the schema file fails the build if the two ever disagree.

```
// src/types/knowledge.schema.ts:302-305
type Same<A, B> = [A] extends [B] ? ([B] extends [A] ? true : false) : false;
const schemaMatchesTypes: Same<z.infer<typeof knowledgeBaseSchema>, KnowledgeBase> = true;
void schemaMatchesTypes;
```

`z.infer<…>` asks Zod "what TypeScript type does this schema describe?". `Same<A, B>` is `true` only if A fits B *and* B fits A. If you add a field to the type but forget the schema, the type becomes `false`, and `= true` is a type error.

> **Could be simpler:** you could write *only* the Zod schema and derive the type with `type KnowledgeBase = z.infer<typeof knowledgeBaseSchema>`. That's one source, not two. The project keeps hand-written interfaces because they read more clearly and carry doc comments, and the `Same` guard keeps them honest. It's a deliberate tradeoff, but it means every new field is typed twice.

---

7. The database: five tables and two helpers

All SQL is in `supabase/migrations/`, applied in filename (date) order.

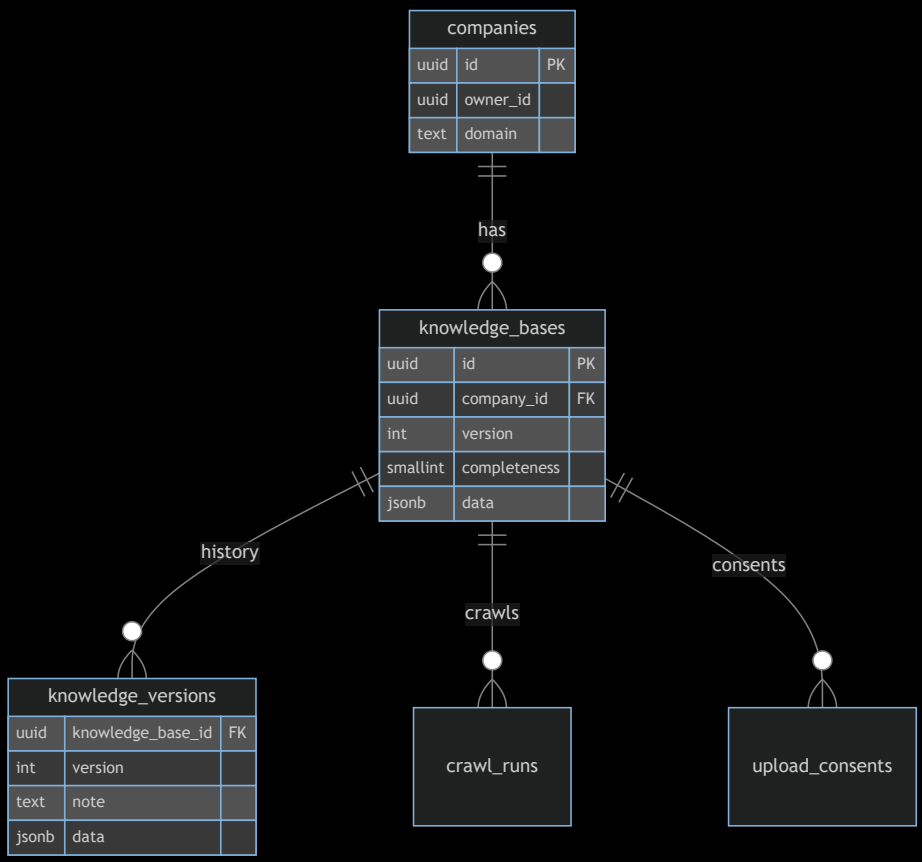


Table	One row per…	Key idea	Defined at
`companies`	business domain per owner	`unique nulls not distinct (owner_id, domain)`: one company per domain, even in the no-login demo where `owner_id` is null	`…120000_knowledge_schema.sql:23-34`
`knowledge_bases`	knowledge base (its **current** version)	the whole object in `data jsonb`, plus copied columns (`company_name`, `industry`, `completeness`) for fast search and filters	`:36-51`
`knowledge_versions`	saved version	a full snapshot of `data` per version, `unique (knowledge_base_id, version)`	`:53-63`
`crawl_runs`	crawl	pages and log, for history	`:65-78`
`upload_consents`	consent given	`check (confirmed)`, so a row can only exist if consent was true	`:80-89`

Two more tables come from the AI migration (`20261009165731_ai_enrichment.sql`):

- `ai_enrichments`: an AI answer cache keyed by a hash of prompt + model + input.
- `ai_usage`: one row per UTC day with a counter.

They have no relationships, and nobody but the server can touch them (RLS on, no policies).

> **Why it's built this way (JSONB + copied columns):** the knowledge base has around 60 nested fields that change shape between phases. One JSONB column means no database migration every time a field is added. But searching inside JSON is slow, so the few things the list page filters and sorts on are *copied out* into normal columns, with trigram indexes for `ILIKE '%term%'` search (`:93-96`).

8. RLS in plain words

Row Level Security (RLS) is a Postgres feature where the database itself checks, for every row, "is this user allowed to see or change *this* row?". Think of it as each row having a little lock, with the database holding the keys.

```
-- supabase/migrations/20261009120000_knowledge_schema.sql:130-135 (inside a loop over the 5 tables)
execute format('create policy %I on public.%I for select to authenticated using (owner_id = (select
auth.uid()))', t || '_select_own', t);
execute format('create policy %I on public.%I for insert to authenticated with check (owner_id = (select
auth.uid()))', t || '_insert_own', t);
-- …update and delete the same way
```

Each policy says: a signed-in (`authenticated`) user may only touch rows where `owner_id` is their own user id.

So how does the demo work with no login?

- There are **no policies for `anon`** (the public key), so a public client reads nothing.
- The server uses the **service role (secret) key**, which **bypasses RLS** entirely (comment at `:115-118`). That's why that key must never reach the browser.
- `owner_id` is nullable today. When real logins arrive, the policies are already in place.

`(select auth.uid())` instead of a bare `auth.uid()` is a performance trick (the comment at line 131 explains it): it's evaluated once per query, not once per row.

---

9. Saving through RPC functions, and the 409

`supabase-js` can't run a multi-step **transaction** (a group of writes that all succeed or all fail). So each save is **one Postgres function** called with `.rpc(...)`, and Postgres runs a function as a single transaction (`:192-194`).

- **`create_knowledge_base(p_data, p_owner, p_note)`** (`:198-254`):
    1. Normalizes the domain.
    2. Inserts or reuses the company.
    3. Inserts the knowledge base at version 1.
    4. Writes the first snapshot to `knowledge_versions`, then records the crawl run and consent.
    5. The **database** owns `id`, `version` and timestamps and writes them back into the JSON (`:221-227`).
- **`update_knowledge_base(p_id, p_data, p_expected_version, p_note)`**: locks the row (`for update`), checks the version, bumps it, updates, and snapshots.

Optimistic concurrency means "assume nobody else is editing, but check at save time". The browser sends the version it started from (`expectedVersion`). If the database's version has moved on, someone else saved in between, so the function refuses:

```
-- supabase/migrations/20261009130000_version_conflict_code.sql:23-26
  if p_expected_version is not null and v_kb.version <> p_expected_version then
    raise exception 'Version conflict: expected %, current is %', p_expected_version, v_kb.version
      using errcode = 'PT409';
  end if;
```

That error travels: Postgres → PostgREST → `fromPostgrest()` maps `PT409` to `DbError("CONFLICT")` (`src/lib/db/errors.ts:29-30`) → `errorResponse` maps `CONFLICT` to **HTTP 409** (`src/lib/utils/http.ts:26`) → the UI shows "Someone saved a newer version. Reload to see the latest changes."

> **A real bug story:** the first version raised SQLSTATE `40001` (`…120000_knowledge_schema.sql:277`). PostgREST treats `40001` as "serialization failure, please retry", so it **retried forever** until the request timed out. The second migration (`20261009130000_version_conflict_code.sql:1-4`) switched to PostgREST's custom `PTxxx` codes, which come back immediately as that HTTP status. Lesson: error codes carry meaning to the layers in between.

Check yourself

1. What four things does every `Field<T>` always carry, and which two are optional?
2. Why is `employeeCount` a string and not a number?
3. A scraped phone exists, and an AI suggestion offers a different phone. What happens, and which code decides?
4. What is the difference between *confidence* and *tier*?
5. Why does the project have both `knowledge.ts` and `knowledge.schema.ts`, and what stops them drifting apart?
6. The demo has no login. Who is allowed to read `knowledge_bases`, and how does the server do it?
7. What is optimistic concurrency, and what does the user see when it triggers?
8. Why was the error code changed from `40001` to `PT409`?

Answers

1. Always `value`, `source`, `confidence` and `updatedAt`. Optional: `evidence` and `reviewedAt`.
2. Real answers are often ranges or have a scope ("11 to 50", "168 technicians (Las Vegas)"), and the project never rounds them into fake exact numbers.
3. Phones are tier 1, and the field isn't empty, so `filterByTier` drops the suggestion (`field-tiers.ts:144-147`). Even if it got through, `merge.ts` ranks scraped (4) above ai_live (3).
4. Confidence = how a value *got* there (scraped, AI, user…). Tier = whether AI is *allowed* to fill that field at all (1 read, 2 inferred, 3 never).
5. Types are compile-time only. Zod checks real data at runtime (requests, DB rows, OpenAI replies) and fills defaults for old records. The `Same<…>` check at `knowledge.schema.ts:302-305` fails the build if they differ.
6. With RLS, only signed-in owners can read their own rows, and the public (`anon`) key reads nothing. The server uses the service role key, which bypasses RLS.
7. Each save says which version it started from, and the database refuses if the stored version is newer. The user gets a 409 and "Someone saved a newer version. Reload to see the latest changes."
8. PostgREST auto-retries `40001` (serialization failure), so the request hung until it timed out. `PT409` is returned immediately as HTTP 409.

03 · The scrape flow: from a pasted URL to a knowledge base

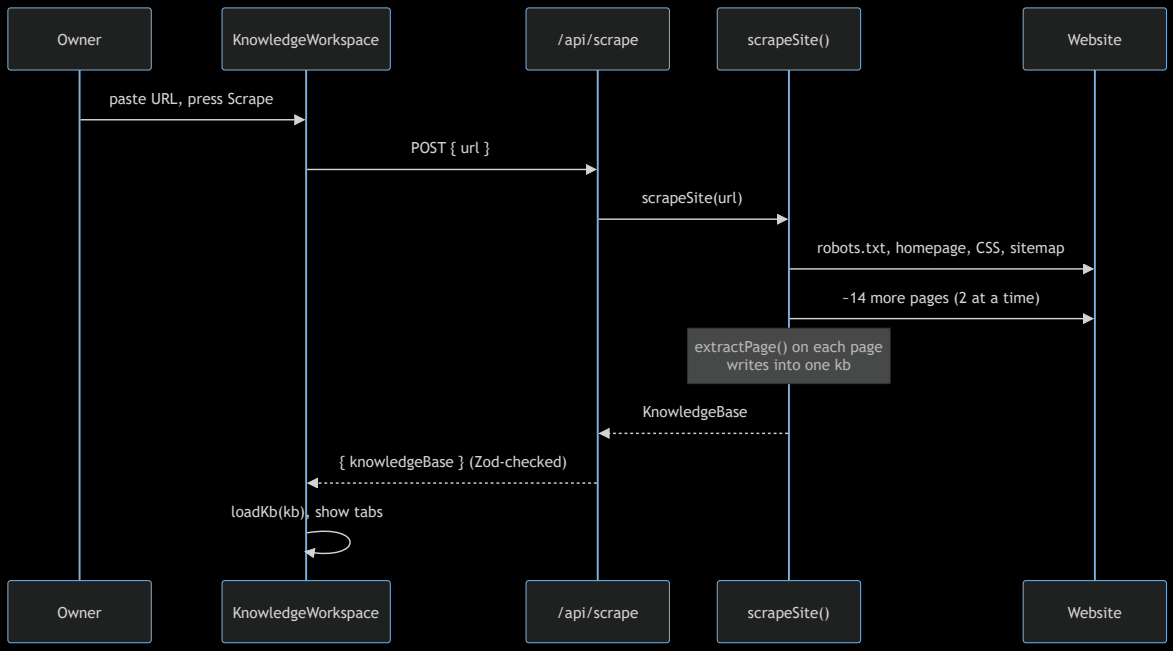
What you'll learn

1. Every step between pressing **Scrape** and seeing tabs, in the order the code runs them.
2. How the crawler decides *which* pages to read, and how robots.txt, consent, budgets and Dig deeper limit it.
3. What SSRF is, how XenFlo blocks it, and the one known gap.

Before this: 01-tech-stack.md (API routes, async, `Promise.all`), 02-data-model.md (`Field`, `emptyKnowledgeBase`)

---

1. The whole trip in one diagram



The browser makes **one** request and waits. All the work happens inside `scrapeSite()` in `src/lib/scraper/index.ts:55-128`. Its own header lists the recipe:

```
// src/lib/scraper/index.ts:50-54
/**
 * Crawl a website and build a knowledge base from it.
 * 1. robots.txt  2. homepage + its CSS  3. discover links (nav + sitemap)
 * 4. priority pages (best page per category)  5. adaptive pages for missing fields  6. score
 */
```

The sections below follow those steps.

---

2. Step 0: turning what you typed into a URL

`normalizeUrl(input)` (`src/lib/scraper/url.ts`) accepts `apexminecrafthosting.com`, `www.site.com/about` or a full `https://…`. It adds `https://` when it's missing and throws `INVALID_URL` when the text can't be a website. That covers the brief's "accept with or without https, reject invalid". Then `emptyKnowledgeBase(startUrl)` creates the blank object everything is written into (chapter 02).

---

3. Step 1: robots.txt and the owner's consent

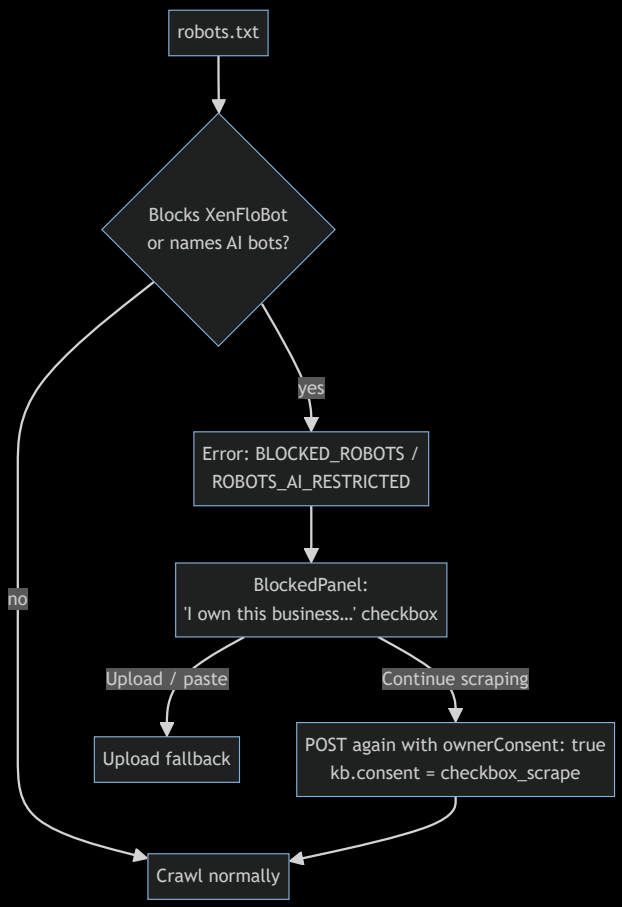
robots.txt is a plain text file at the root of every site (`/robots.txt`) where the owner says which bots may read what. It's a convention, not a lock; polite crawlers follow it. XenFlo identifies itself honestly as `XenFloBot/1.0 (knowledge base builder)` (`src/lib/scraper/fetch.ts:4`) and checks robots.txt *before* anything else.

`loadRobots()` (`src/lib/scraper/robots.ts:65-84`) uses the `robots-parser` library and returns:

- **`isAllowed(url)`**: may XenFloBot fetch this URL?
- **`aiRestricted`**: well-known AI crawlers the site shuts out by name (GPTBot, ClaudeBot, CCBot… the list is at lines 12-27).
- **`crawlDelay`** and **`sitemaps`**.
- If there's no robots.txt, everything is allowed (lines 81-82), which is standard crawler behavior.

Then `scrapeSite` decides:

```
// src/lib/scraper/index.ts:66-86 (trimmed)
  if (blocked && !options.ownerConsent) {
    kb.crawl.robotsAllowed = false;
    throw new ScrapeError("BLOCKED_ROBOTS", "This site's robots.txt doesn't allow automated access.");
  }
  if (aiRestricted && !options.ownerConsent) {
    throw new ScrapeError("ROBOTS_AI_RESTRICTED", `This site's robots.txt asks AI crawlers not to read
it (…).`);
  }
  let isAllowed = robots.isAllowed;
  if ((blocked || aiRestricted) && options.ownerConsent) {
    // Record who allowed it and when; the crawl stays polite (honest User-Agent, crawl delay kept)
    kb.consent = { confirmed: true, timestamp: new Date().toISOString(), method: "checkbox_scrape" };
    kb.crawl.robotsAllowed = !blocked;
    if (blocked) isAllowed = () => true;
  }
```



The panel is `src/components/knowledge/fallback/BlockedPanel.tsx`. Its "Continue" button calls `scrape(url, { ownerConsent: true })` (`KnowledgeWorkspace.tsx:155`).

There's one important distinction (`src/lib/api/messages.ts:39-40`):

- A **robots.txt block** can be lifted by the owner's permission.
- A server that **actively refuses** us (HTTP 401/403/429, or a Cloudflare "Just a moment..." page, which is `BLOCKED_ACCESS`) can't be. For those, the panel only offers uploading or typing.

> **Why it's built this way:** MoFlo's customers *are* the business owners. Refusing outright would block the very people the product is for, and ignoring robots.txt would be rude to everyone else. Asking, and recording the answer (`consent` is stored with the record and in the `upload_consents` table), respects both. The idea is written up in `docs/improvements.md` (the Anime Boba row).

---

4. Step 2: the homepage and its CSS

```
// src/lib/scraper/index.ts:89-103 (trimmed)
  const home = await fetchPage(startUrl);
  if (!isHtml(home.contentType)) throw new ScrapeError("NO_CONTENT", "That address didn't return a web page.");
  const homeUrl = cleanUrl(home.url, home.url) ?? startUrl; // after redirects (http -> https, bare -> www)
  kb.url = homeUrl;
  // …
  const css = await collectCss(cheerio.load(home.body), homeUrl, isAllowed);
  const homeResult = extractPage(home.body, homeUrl, "home", kb, css);
```

- If the homepage fails, the whole scrape fails, with a clear error code.
- The URL *after redirects* becomes the knowledge base's URL.
- Only the homepage gets its stylesheets downloaded (`styles.ts`, max 3 sheets), because fonts and colors are site-wide.
- `extractPage` runs every extractor on the page (chapter 04).
- If the homepage has fewer than `MIN_WORDS = 30` readable words, the log warns that it's probably a JavaScript-only shell.

---

5. Step 3: discovering pages and ranking them

Two sources of links:

1. **Links on the homepage**: `discoverLinks()` (`src/lib/scraper/discover.ts:63-87`) keeps internal links only, skips files and logins (`SKIP_PATH`), and gives each one a **category** and a **score**.
2. **The sitemap**: `discoverFromSitemaps()` reads `sitemap.xml` (or the ones robots.txt names) and follows sitemap index files.

Categorizing is keyword matching on the URL path first, then the link text:

```
// src/lib/scraper/discover.ts:20-37 (excerpt)
const CATEGORY_RULES = [
  // Legal is checked first so "/terms-of-service" isn't mistaken for a services page.
  { category: "legal", score: 5, pattern: /\bprivacy|terms|legal|cookie|disclaimer|refund|tos\b/i },
  { category: "menu", score: 95, pattern: /\bmenus?\b|\bfood\b|\bdrinks?\b|price[- ]?list|\bspecials\b|\bcatering\b/i },
  { category: "about", score: 100, pattern: /\babout|our[- ]story|who[- ]we[-]are|company|history|mission/i },
  { category: "team", score: 90, pattern: /\bteam|staff|leadership|founders?|people|meet[- ]the/i },
  // … services 85, pricing 85, faq 75, testimonials 70, contact 65, locations 60, press 45 …
  { category: "careers", score: 8, pattern: /\bcareers?|jobs|hiring|join[- ]us/i },
  { category: "blog", score: 10, pattern: /\bblog|articles?|posts?\b/i },
];
```

The score is then adjusted: **+10** if the link sits in the nav, header or footer (the site's own "important pages" list), and **−8 per path level** past the first, because deep paths are usually individual posts (`discover.ts:72-75`).

---

6. Steps 4 and 5: priority pages, then adaptive pages

With a pool of ranked links, the crawl runs in two passes.

Pass 1: priority pages. `priorityLinks()` (`discover.ts:157-159`) takes the **best link from each useful category**: one about, one services, one pricing, one FAQ, one contact… This gives *breadth*. You'd rather have about + pricing + FAQ than five blog posts.

Pass 2: adaptive ("crawl for missing"). After pass 1, the crawler asks "which important fields are *still* empty?", and only crawls pages likely to fill them:

```
// src/lib/scraper/crawl.ts (crawlForMissing, trimmed)
  while (kb.crawl.pages.length < maxTotalPages && !outOfTime(session)) {
    const missing = scoreCompleteness(kb).missing;
    const targets = [...session.pool.values()]
      .map((link) => ({ link, points: potentialPoints(kb, link.url, link.category, missing) ||
unknownPagePoints(link, missing) }))
      .filter((t) => t.points > 0)
      .sort((a, b) => b.points - a.points || b.link.score - a.link.score);
    if (!targets.length) { session.log("No remaining pages look likely to fill the missing fields");
return; }
    // Crawl a small batch, then re-check what's missing before choosing more.
    const batch = targets.slice(0, session.concurrency * 2).map((t) => t.link);
    await crawlPages(session, batch, maxTotalPages);
  }
```

`potentialPoints` (`src/lib/scraper/score.ts:73-86`) uses `FIELD_HINTS` (`score.ts:46-58`). For example, `"insights.faqs": { categories: ["faq"], keywords: /help|questions|support/i }` means "if FAQs are still missing, a page in the faq category, or with 'help' in its path, is worth the FAQ weight (4 points)". Pages are tried in order of points, in small batches, re-checking after each batch.

> **Why it's built this way:** 15 pages is very little. Spending them where the *missing* information most likely is beats crawling in link order. That's the brief's "adaptive crawl" requirement, and it's why small sites usually reach a decent score in one pass.

After the passes:

- `crawlLegalPage()` may read one privacy/terms page for the legal name (Phase 11, chapter 04).
- `readMenus()` reads any menu PDFs the crawl found (chapter 06).
- `finish()` (`index.ts`) saves leftover links as `pendingUrls` (for Dig deeper), computes the score, caps the log at 300 lines and stamps the times.

---

7. The worker pool, budgets and politeness

`crawlPages()` (`src/lib/scraper/crawl.ts:44-92`) is the engine. It starts `concurrency` workers (2) that share one queue:

```
// src/lib/scraper/crawl.ts (worker loop, trimmed)
    while (queue.length && kb.crawl.pages.length + inFlight < maxTotalPages) {
      if (outOfTime(session)) return;
      const link = queue.shift()!;
      // … skip visited / filtered links …
      inFlight++;
      try {
        const page = await fetchPage(link.url);
        const result = extractPage(page.body, finalUrl, link.category, kb);
        kb.crawl.pages.push(pageRecord(finalUrl, link.category, page.status, result.title, …));
        addToPool(session, discoverLinks(result.$, finalUrl));
      } catch (err) {
        // … record the failed page with its error, log a warning, keep going …
      } finally {
        inFlight--;
      }
      await sleep(session.delayMs);
    }
```

- `inFlight` counts pages being fetched right now, so two workers can't both take the last slot under the cap.
- **New links** found on each page join the pool, so the crawl can go deeper than the homepage's links.
- **One page failing** never fails the scrape. It's recorded with its error and the crawl continues.

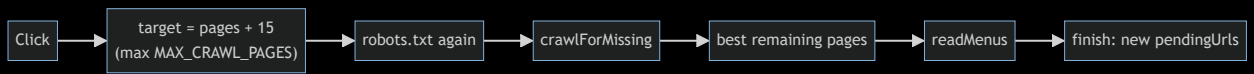
The limits, all in `newSession()` (`src/lib/scraper/index.ts`):

Limit	Value	Why
Pages, first scrape	15 (`DEFAULT_MAX_PAGES`)	Enough for a small business; fast
Time budget	30 s (`timeBudgetMs`); no new page starts after this	Route max is 60 s on Vercel; leaves room for menus and the response
Concurrency	2 workers	Polite to small servers
Delay	300 ms between requests per worker, or the site's `Crawl-delay` **capped at 3 s**	Polite, but a 10 s delay would make the demo unusable (`docs/improvements.md` explains the choice)
Per-request timeout	10 s (`fetchPage` default)	A slow page doesn't eat the whole budget
HTML size	first ~3 MB (`MAX_BODY_CHARS`, `fetch.ts:17`)	Memory safety

---

8. Dig deeper: batches of 15

The brief: "if important fields are still missing, offer **Dig deeper**". The button (Sources tab, or the low-score banner) calls `POST /api/scrape/deeper` with the current knowledge base, and `digDeeper()` continues the crawl from `kb.crawl.pendingUrls`.



- **15 pages per click** (`BATCH_PAGES`), up to `MAX_CRAWL_PAGES` in total. That's an env var: default 200, kept between 15 and 500 by `maxCrawlPages()`.
- **Past 30 pages, only high-value links** (`src/lib/scraper/focus.ts`): menu, locations, services, pricing, about and contact; legal only while the legal name is unknown; and shallow pages with no keyword, like a restaurant group's `/sakana`. Blog, news, tag, archive and pagination pages, and URLs with a query string, are skipped.
- **The same safety rules** apply on every click: robots.txt, the crawl delay, the SSRF checks and the 30 s budget.
- **Untrusted URLs:** `pendingUrls` come back *from the browser*, so `digDeeper` keeps only URLs on the company's own domain (`isSameDomain`). Otherwise someone could plant other URLs in the list.
- **The button:** it shows `Dig deeper (45 of 200 pages)`, and it disables at the max or when no useful links remain ("No more useful pages found").

On the test sites:

- **Dragon Factory:** it has only 22 crawlable pages, so one click finishes it.
- **Goettl:** it went 14 → 28 → 43 → … 103 pages over six clicks, about 20–30 s each.

---

9. SSRF: why a scraper is dangerous, and how XenFlo is protected

SSRF (Server-Side Request Forgery) is when an attacker gets *your server* to fetch a URL *they* chose.

Analogy: XenFlo is a mail clerk who will go fetch any address written on an envelope. Normally that's a public website. But what if someone writes "the vault in our own basement"? The clerk has a key to the basement, the attacker doesn't, and the clerk would happily bring back what's inside. For a server, "the basement" is:

- `localhost` / `127.0.0.1`: the server itself
- `192.168.x.x`, `10.x.x.x`: private office or home networks
- `169.254.169.254`: the **cloud metadata endpoint**, which on many cloud hosts hands out credentials to anyone *on that machine* who asks

XenFlo fetches URLs that users type, *and* URLs those sites link to, so it must refuse all of these. `src/lib/scraper/safety.ts` does it:

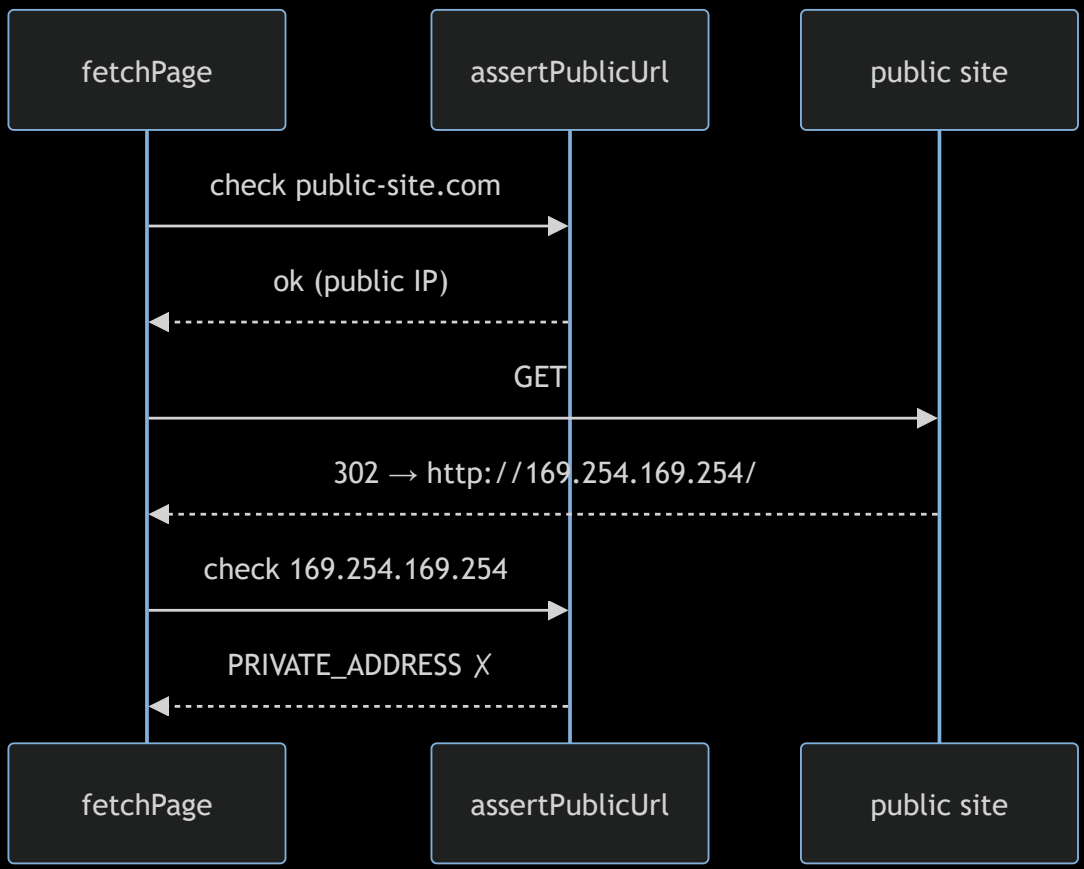
```
// src/lib/scraper/safety.ts:80-106 (trimmed)
export async function assertPublicUrl(url: string): Promise<void> {
  // …parse the URL…
  if (u.protocol !== "http:" && u.protocol !== "https:") throw refuse("XenFlo only reads http and https websites.");
  if (!ALLOWED_PORTS.has(u.port)) throw refuse("XenFlo only reads websites on the standard ports (80 and 443).");
  if (u.username || u.password) throw refuse("Addresses with a username or password aren't supported.");
  const host = u.hostname.replace(/^\[|\]$/g, "");
  if (isIP(host)) { if (isPrivateIp(host)) throw refuse(); return; }
  if (PRIVATE_NAME.test(host) || !host.includes(".")) throw refuse();
  // … (inside try/catch: a failed lookup throws UNREACHABLE) …
    addresses = await lookup(host, { all: true, verbatim: true });
  if (!addresses.length || addresses.some((a) => isPrivateIp(a.address))) throw refuse();
}
```

What's blocked:

- **Other protocols** (`file:`, `ftp:`), **other ports** (admin panels live on those), and **user:password@** URLs.
- **Every private range**, IPv4 and IPv6 (the list is at lines 13-44), including IPv4 hidden inside IPv6 like `::ffff:127.0.0.1` (lines 56-62).
- **Private-only names** like `localhost`, `*.local`, `*.internal`.
- **DNS answers:** the hostname is looked up, and if **any** of its addresses is private, it's refused.

Redirects are the sneaky part. A public site can answer "302, go to `http://169.254.169.254/`". So `fetch.ts` turns off automatic redirects (`redirect: "manual"`) and follows them itself, **checking every hop**:

```
// src/lib/scraper/fetch.ts:29-46 (trimmed)
  for (let hop = 0; ; hop++) {
    await assertPublicUrl(current);
    res = await fetch(current, { redirect: "manual", signal, headers: { "User-Agent": USER_AGENT, … }
});
    const location = res.headers.get("location");
    if (res.status < 300 || res.status >= 400 || !location) break;
    if (hop >= MAX_REDIRECTS) throw new ScrapeError("HTTP_ERROR", "The site redirected too many times.",
res.status);
    await res.body?.cancel(); // we only need the Location header
    current = new URL(location, current).toString();
  }
```



The known gap: DNS rebinding. `assertPublicUrl` looks the name up, and then `fetch` looks it up *again* to connect. An attacker who controls a domain's DNS could answer "public IP" the first time and "private IP" a split second later (with a very short TTL). README.md:473 documents this honestly. Closing it fully means pinning the connection to the IP that was checked (a custom agent with the `undici` package), or running in a network that has no route to private ranges, which serverless hosting like Vercel largely already is.

> **Why it's built this way:** checking up front and on every redirect stops all the common SSRF tricks with about 100 lines and no extra dependency. The remaining gap needs DNS control *and* precise timing, and the hosting environment limits the damage anyway. It's a reasonable place to stop for a demo, and it's written down.

---

10. When things go wrong: error codes

Every failure becomes a `ScrapeError` with a **code** (`src/lib/scraper/errors.ts`). `src/lib/utils/http.ts:10-22` maps each code to an HTTP status, and `src/lib/api/messages.ts` maps it to friendly text:

Code	HTTP	Example cause
INVALID_URL	400	"not a url" typed
PRIVATE_ADDRESS	400	`http://localhost:3000`
BLOCKED_ROBOTS / ROBOTS_AI_RESTRICTED	403	robots.txt (consent can lift it)
BLOCKED_ACCESS	403	401/403/429 or a Cloudflare challenge page
NO_CONTENT	422	not HTML, or an empty JS-only shell
UNREACHABLE / HTTP_ERROR	502	DNS failure, 500 from the site
TIMEOUT	504	homepage slower than 10 s

Check yourself

1. What are the six steps of `scrapeSite`, in order?
2. What's the difference between `BLOCKED_ROBOTS` and `BLOCKED_ACCESS`, and why can consent lift only one?
3. Why does a link in the nav get +10 and a deep path get −8 per level?
4. Explain "adaptive crawl" in one sentence, and name the two functions that make it work.
5. What three things stop two workers from crawling 16 pages when the cap is 15, or crawling forever?
6. What changes after page 30 in Dig deeper, and why?
7. Explain SSRF with the mail clerk analogy, and say why redirects must be checked hop by hop.
8. What is the DNS rebinding gap, and what would close it?

Answers

1. robots.txt → homepage + CSS → discover links (nav + sitemap) → priority pages → adaptive pages for missing fields → finish/score. Since Phases 10–11 there are also `crawlLegalPage` and `readMenus` before `finish`.
2. `BLOCKED_ROBOTS`: the site's robots.txt asks bots not to read it. That's a request, so the owner's permission can lift it. `BLOCKED_ACCESS`: the server actively refused (401/403/429 or a bot challenge). Consent can't change what their server does.
3. Nav/header/footer links are the site's own list of important pages. Deep paths are usually individual posts, which hold fewer company facts (`discover.ts:72-75`).
4. After the priority pages, only crawl pages likely to fill fields that are still empty, re-checking after each small batch. The functions are `crawlForMissing` (`crawl.ts`) and `potentialPoints` (`score.ts:73-86`) with `FIELD_HINTS`.

5. The page cap counts `inFlight` pages; `outOfTime()` stops new pages after 30 s; and `crawlForMissing` stops when no link in the pool targets a missing field (every batch link leaves the pool).
6. Only high-value links are followed: menu, location, services, pricing, about, contact, legal if still needed, and shallow restaurant pages. That way pages 31–200 add real knowledge instead of blog posts, and the saved record stays a sensible size.
7. The server is a clerk who'll fetch any address, including the private "basement" (localhost, internal networks, cloud metadata) that outsiders can't reach. A public site can redirect to a private address, so each hop is re-checked with `assertPublicUrl`.
8. The name is resolved once for the check and again for the real connection, so a hostile DNS server could answer differently the second time. It would be closed by pinning the connection to the checked IP (a custom `undici` agent), or by a network with no route to private ranges.

04 · The extractors: how facts are pulled out of HTML

What you'll learn

1. The order every page goes through, and why structured data runs first.
2. Each extractor's input, output and key heuristic, with the real-site bug that shaped it.
3. How `setField` / `addItem` decide which value wins when several pages disagree.

Before this: 02-data-model.md (`Field`, confidence ranks), 03-scrape-flow.md (where `extractPage` is called)

---

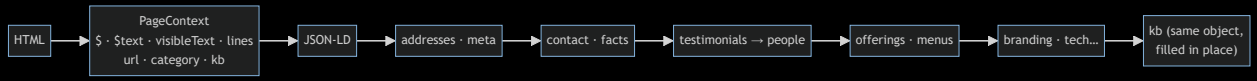
1. One page, fifteen extractors

Every crawled page goes through `extractPage()` in `src/lib/scraper/extract/index.ts:42-77`. It loads the HTML with Cheerio **twice**:

- `$`: the raw page, with scripts, meta tags and JSON-LD intact.
- `$text`: a "readable" copy with spaces between elements (`readableDom` in `text.ts`). Cheerio's `.text()` glues neighbours together: "4 GB" + "RAM" becomes "4 GBRAM".

Then it builds one `PageContext` object (`extract/types.ts`) and hands it to each extractor in order:

```
// src/lib/scraper/extract/index.ts:54-71
  // Structured data first: it's the most trustworthy, and "first value wins".
  extractJsonLd(ctx);
  extractAddresses(ctx); // after JSON-LD, so a structured address wins
  const { title } = extractMeta(ctx);
  extractSocial(ctx);
  extractContact(ctx);
  extractBusinessFacts(ctx);
  extractAbout(ctx);
  // Testimonials before people, so quote authors are tagged customer_partner first.
  extractTestimonials(ctx);
  extractPeople(ctx);
  extractOfferings(ctx);
  extractMenuSources(ctx);
  extractCtas(ctx);
  extractSignals(ctx);
  extractPress(ctx);
  extractBranding(ctx);
  extractTech(ctx);
```



Every extractor has the same shape: `(ctx) => void`. It reads the page and **writes straight into** `ctx.kb`, always through `setField` or `addItem`. Order matters because of the "first value wins" rule (section 13).

`extractPage` also returns per-page **evidence** (meta description, up to 12 headings, up to 10 image clues; lines 79-91). It's saved on the crawl record for AI to read later (chapter 05).

---

2. JSON-LD: the site's own structured data

JSON-LD is a `<script type="application/ld+json">` block that sites add for Google: machine-readable facts like `{"@type": "Restaurant", "name": …, "address": …}`. When it's there, it's the most reliable source, so it runs first (`src/lib/scraper/extract/jsonld.ts:7-10`).

- **Input:** every ld+json block, flattened (arrays and `@graph`).
- **Output:**
  - from Organization-like types (`ORG_TYPE`, line 14): company name, legal name, description, founding date, employees, addresses, phones, emails, areas served
  - from Product/Service/Offer: offerings
  - from FAQPage: FAQs (line 150)
  - from Review: testimonials
  - from Person: people
- **Key heuristic:** the most specific schema type becomes an *inferred* industry hint: `"Dentist"` → "Dentist" (`jsonld.ts:98-100`).

> **Why it's built this way:** most small business sites have *no* JSON-LD, so everything below exists as a fallback for sites that only have plain text.

---

3. Meta and Open Graph tags

`extractMeta` (`src/lib/scraper/extract/meta.ts:17-41`):

- **Homepage only:** the overview comes from `description`, then `og:description`, then `twitter:description`. The name comes from `og:site_name` (scraped), and as a fallback from the `<title>` ("Apex Hosting | Minecraft Servers" → "Apex Hosting"), marked **inferred** because picking the brand out of a title is a guess.
- **Every page:** `theme-color`, the color the site chose for the browser bar, is a strong brand-color hint, *unless* it's a platform default (line 36, more in section 7).

---

4. Contact: emails and phones

`extractContact` (`src/lib/scraper/extract/contact.ts`):

- **Emails:** `mailto:` links first (most reliable), then a regex over the visible text. `FAKE_EMAIL` (line 8) rejects image names like `logo@2x.png` and tool addresses like `@sentry.io`.
- **Phones:** `tel:` links, then a regex that **requires separators**, `(702) 555-1234` or `702.555.1234`, so IDs and timestamps aren't read as phone numbers (comment at line 12). `isUsPhone` checks for a real area code.

Real bug: Goettl's branch phones. Goettl's Las Vegas location page (`/location/las-vegas`) lists phone numbers for *every* branch: Phoenix, Tucson, Austin… Naively, they'd all become "our" phones. The fix:

```
// src/lib/scraper/extract/contact.ts (inside extractContact)
  const city = locationCity(kb.url);
  const isStartPage = url === kb.url;
  const addPhone = (display: string, digits: string, context: string) => {
    if (!isUsPhone(digits)) return;
    const value = PHONE_ONLY.test(display) ? display.trim() : formatPhone(digits);
    if (!city || isStartPage) return addItem(kb.contact.phones, value, url, phoneKey);
    const where = cityMatch(context, city);
    if (where === "other") return;
    if (where === "ours") return addPreferredPhone(ctx, value);
    addItem(kb.contact.phones, value, url, phoneKey);
  };
```

`locationCity("…/location/las-vegas")` returns `"las vegas"`. For every phone, the extractor looks at roughly 150 characters of text around it. If they mention our city, the phone is **ours** and moves to the front. If they contain *another* `City, ST 12345`, it belongs to another branch and is skipped. If neither, it's kept normally.

---

5. Addresses

Two files work together:

- `src/lib/scraper/address.ts`: a pure "find US addresses in text" engine.
- `src/lib/scraper/extract/address.ts`: decides **main address vs other locations**.

The finder is one big regex anchored on a **real state code + ZIP** (`address.ts:14-25`), which keeps false matches rare. It tries single lines first, then joins 2–3 neighbouring lines, because sites often split an address over `<br>` lines (`findAddresses`, lines 59-68).

Real bug: opening hours glued onto the street. Dragon Factory's restaurant pages print hours right above the address:

```
LAST CALL: 11:30 PM
456 Sample Blvd #12, Springfield, NV 89000
```

Joining lines could turn "11:**30** PM" into a street number. The street-name group can't contain commas, and the city must start with a letter, so the hours can't swallow the address. A regression test pins it (`src/lib/scraper/address.test.ts:50-53`):

```
  it("doesn't glue opening hours onto the street", () => {
    const [a] = findAddresses(["LAST CALL: 11:30 PM", "456 Sample Blvd #12, Springfield, NV 89000"]);
    expect(a.street).toBe("456 Sample Blvd #12");
  });
```

Maps-link fallback. When the text has no address, Google or Apple Maps links and embeds often carry one (`?q=123+Example+Ave…`). `addressFromMapsUrl` (`address.ts:75+`) reads it, marked **inferred** because it comes from a URL, not words on the page (`extract/address.ts:25-33`). Anime Boba Cafe showed why this matters: "Henderson" appeared in the text, but there was no street address anywhere except the map (`docs/improvements.md:60`).

Main vs other (`extract/address.ts:7-16`):

- on a location page, the address in that city is main
- a page listing 3+ addresses is a branch directory, so all of them are "other"
- the homepage, about or contact page with 1–2 addresses: the first is main
- any other page (one restaurant of a group): never main

A restaurant group whose site names no head office keeps main **Missing** rather than picking one at random. That's why the Overview offers "Use one of your locations".

---

6. Business facts: legal name, entity, headcount (Phase 11)

`src/lib/scraper/extract/business-facts.ts` reads facts that are tier 3 (AI never guesses them), so only what's *written* counts, and every value keeps its **quote as evidence**.

1. **The footer copyright line:** "© 2013-2026 Apex Hosting LLC". The suffix gives the entity type through one table in `src/lib/scraper/entity-suffixes.ts` (LLC → LLC, Inc/Corp → Corporation, LP/LLP → Partnership, PLLC, PC, Ltd). **No suffix, no type**: "© 2026 by Dragon Factory" leaves it empty.
2. **One legal page:** privacy or terms, read only when the footer gave nothing. It accepts "operated by X LLC" or "X, LLC ("we", "us")".
3. **Headcount:** only when stated, keeping the noun and scope. "168 HVAC & Plumbing Technicians" on the Las Vegas page becomes `168 technicians (Las Vegas)`.

Real bug: Facebook Inc. Apex's privacy page says "*...social media pages: Facebook, operated by Facebook Inc.*". The "operated by" pattern matched, and XenFlo almost decided Apex's legal name was Facebook. The fix: on legal pages, the name must **share a distinctive word with the business**:

```
// src/lib/scraper/extract/business-facts.ts (sharesWord)
function sharesWord(name: string, words: string[]): boolean {
  return name
    .toLowerCase()
    .split(/[^a-z0-9]+/)
    .filter((w) => w.length >= 4 && !GENERIC.has(w))
    .some((w) => words.some((b) => b === w || b.includes(w)));
}
```

`businessWords(kb)` is the company name's words plus the domain label (`goettl`, `apexminecrafthosting`). "facebook" matches neither, so it's rejected. "Goettl Home Services, LLC" shares "goettl", so it's accepted. `GENERIC` (services, group, holdings...) stops a common word from counting as a match.

Platform names (Wix, WordPress, Kadence WP...) are ignored too (`isPlatformName` in `entity-suffixes.ts`), so "© Wix.com Ltd" never becomes the business.

---

7. Branding: logos, fonts, colors

`src/lib/scraper/extract/branding.ts` (295 lines, the biggest extractor).

Logos (`extractLogos`, lines 41-88):

- **Kept only from the homepage.** Inner pages contribute only name clues.
- **Ranked** (`LOGO_RANK`): header logo > logo image > apple-touch-icon > og:image > favicon.
- **Partner logos** on a partners page or under "As seen on" go to `suppliersPartners` instead.

Image clues without AI (`imageClue`, `text.ts:101-114`): use the alt text, or a cleaned file name. `"Sumo Henderson_Logo.png"` → "Sumo Henderson", and `"NekoLoco"` → "Neko Loco". Hash-like names (`a8f3…`, Wix's `~mv2`) and camera names (`IMG_1234`) are thrown away.

Real bug: alt text that describes a photo. One Goettl image had the alt text *"Person holding a wrench in front of the goettl"*, and it became an "alternate name"! (README's Goettl before/after table shows it.) The fix is `isDescriptive()`: alt text only counts as a logo name when it *names* a logo, not when it describes a scene (`branding.ts:60-62`). `addAlternateName` also requires short (≤5 words) names, and on inner pages, a match with the page itself.

Fonts (`extractFonts`, line 112+): Google Fonts links count heavily; then `font-family` in the CSS, with **CSS variables resolved**. Generic fonts (Arial, system-ui) and **icon fonts** (ETmodules, FontAwesome, dashicons…; `ICON_FONTS`) are filtered out.

Colors: the Goettl story. The first version counted how often each color appeared in the CSS. On Goettl (WordPress + Kadence theme), WordPress's admin blues `#007cba` / `#005a87` and Kadence's orange `#f76a0c` won, and Goettl's real red didn't appear at all (`docs/data-quality.md`, "Platform default colors"). The rewrite ranks colors by **where they're used**:

```
// src/lib/scraper/extract/branding.ts:217-225
function selectorWeight(selectorList: string): number {
  const own = selectorList.split(",").map((s) => s.trim()).filter((s) => s &&
!LIBRARY_SELECTOR.test(s));
  if (!own.length) return 0;
  const joined = own.join(",");
  if (BUTTON_SELECTOR.test(joined)) return 6;
  if (HEADER_SELECTOR.test(joined)) return 4;
  if (LINK_SELECTOR.test(joined)) return 2;
  return 1;
}
```

- **Usage weights:** buttons and CTAs ×6, header and nav ×4, links ×2, anything else ×1.
- **Property weights:** background and text color count fully, borders and shadows half (`propertyWeight`, lines 228-232).
- **Plugin rules count zero:** rules that only style plugin markup (`.wp-*`, `.kadence-*`, `.elementor-*`; `LIBRARY_SELECTOR`, line 210).
- **Brand-variable bonus:** colors in brand-named variables the site actually uses (`--primary`, `--brand-500`) get +2.
- **Filters:** known platform defaults are removed (`isPlatformDefault`, `colors.ts:72`), as are near-duplicates (`isSimilar`) and grays (`isNeutral`). An all-black-and-white site keeps its dominant neutral (lines 293-294).

Result on Goettl: navy `#003963` and red `#cd163f`, the real brand.

---

8. Offerings: three strategies

`extractOfferings` (`src/lib/scraper/extract/offerings.ts:22-28`) tries the strategies in order:

```
export function extractOfferings(ctx: PageContext): void {
  // A menu page laid out as lists or tables ("California Roll ... $8.95"): read it line by line.
  // When that works, skip the price-card reader, which would take the section heading ("Rolls") as an
item.
  if (ctx.category === "menu" && extractMenuPage(ctx)) return;
  extractPriceCards(ctx);
  if (ctx.category === "services" || ctx.category === "products") extractServiceSections(ctx);
}
```

1. **Menu pages:** read line by line with the same price parser used for PDFs (chapter 06).
2. **Price cards** (lines 60-97): find the *smallest* element containing a `$` price, then **climb up** to the nearest ancestor that has a heading (`cardFor`, lines 100-111). That's the "card", and its heading is the name. `parsePricing` reads the pricing type: "/mo" is a subscription, "from $" is starting_at, a range, and so on. It skips intro prices like "$11.24 first month".
3. **Service sections** (services/products pages only): an h2/h3 followed by a real paragraph is a service with `pricingType: "unknown"`. That's how a plumber's "Leak Detection" becomes an offering even without a price.

---

9. Testimonials, then people (order matters)

Testimonials (`src/lib/scraper/extract/testimonials.ts`):

- **What counts as a quote:** `<blockquote>` / `<q>`, or a block that starts with a quote mark, which must be inside a testimonial context (a testimonials page, a `.review` container, or under a "What our customers say" heading). Otherwise it needs an author.
- **The author** is found as an inline "— Name", in cite/figcaption, or in the next siblings. `splitAuthor` separates name, title and company.
- **The author becomes a Person tagged** `customer_partner`, never team (lines 66-83). Usernames ("Xmkzink") stay on the quote but aren't added as people.

People (`src/lib/scraper/extract/people.ts:17-40`) runs **after** testimonials and only on team and about pages. A person is a name-looking heading next to a job title (the `ROLE` regex has 60+ titles), or a name and a photo on a dedicated team page. Anything inside a testimonial is skipped, and a name already in People is never re-tagged as team.

> **Why it's built this way:** the brief says "do NOT mix testimonial authors into the team". Running testimonials first means their authors are already filed as customers by the time the team extractor looks.

---

10. Menu sources (Phase 10)

`extractMenuSources` (`src/lib/scraper/extract/menu-sources.ts:1-7`) doesn't read menus. It **notes where they are**:

- PDF links that look like menus or price lists, and big images on menu pages, go into `kb.crawl.menuSources` and are read later (chapter 06).
- Third-party ordering and booking links (Toast, Square, Clover, DoorDash…; `PLATFORMS` table) go into `customers.channels`, as in "Online ordering (Clover)". Those platforms are **never fetched**.

---

11. The smaller extractors (one line each)

Extractor	File	Reads	Writes
Social	`extract/social.ts`	links to LinkedIn, Facebook, Instagram, X, YouTube, TikTok, Twitch, Discord, Pinterest, matched on host	`brand.socialLinks`
About	`extract/about.ts`	"Founded in 2013", "Since 1987", story paragraphs	`company.yearFounded`, `foundingStory`
CTAs	`extract/ctas.ts`	button-like links ("Get a quote"), skipping cookie banners and UI controls	`customers.ctas`
Signals	`extract/signals.ts`	the *smallest* element matching award, certification, rating, "% off", guarantee…	`trustSignals`, `promotions`, `differentiators`
Press	`extract/press.ts`	"As seen in…" sections	`insights.pressMentions`
Tech	`extract/tech.ts`	script/link/iframe sources (Google Analytics, Mailchimp, Stripe, chat widgets)	`customers.suppliersPartners`

Legal links live in `extract/contact.ts:56-62`: any link whose *label* matches `privacy|terms|refund|cookie|accessibility|disclaimer|legal` goes into `insights.legalLinks`.

> **Could be simpler:** that label regex has no word boundaries, so "**illegal** blocks" (an Apex blog post) and other ordinary links slip into the legal list. In the Phase 11 test run, Goettl's list held 17 links, most of them not legal pages. Matching whole words, plus checking the URL path the way `crawlLegalPage` already does (`/(?:^|[/-])privacy/`), would clean it up.

---

12. The upload fallback uses the same extractors

When a site blocks us, the owner can paste text or upload HTML. `src/lib/scraper/content.ts` sends pasted HTML through **exactly the same** `extractPage`. Pasted *plain text* has no structure, so `text-structure.ts` first recovers the obvious parts and describes them as HTML plus **JSON-LD** (`text-structure.ts:1-14`): short lines become headings, lines ending in "?" become FAQ questions, `"Great service!" - Jenna R.` becomes a review, `Full Groom - $65` becomes a priced service, `Maria Ortega - Owner` becomes a team member, and web addresses become links. "Everything comes from the user's own words; nothing is guessed or invented." There's one set of rules for both paths.

---

13. Which value wins: `setField` and `addItem`

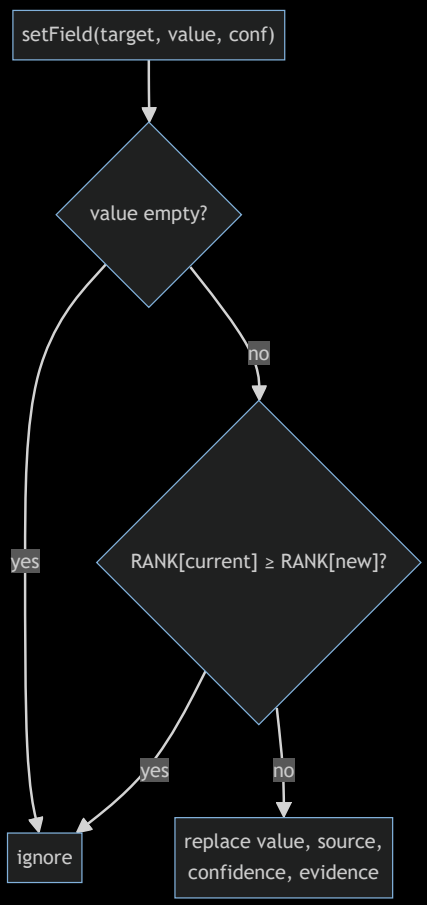
Every extractor writes through two tiny helpers in `src/lib/scraper/merge.ts`:

```ts
// src/lib/scraper/merge.ts:20-35
export function setField<T>(
  target: Field<T>,
  value: T | null | undefined,
  source: string,
  confidence: Confidence = "scraped",
  /** Short quotes the value was read from, shown in the badge tooltip */
  evidence?: string[],
): void {
  if (value === null || value === undefined) return;
  if (typeof value === "string" && !value.trim()) return;
  if (RANK[target.confidence] >= RANK[confidence]) return;
  Object.assign(target, field(value, source, confidence));
  // Don't keep the evidence of the value being replaced
  if (evidence?.length) target.evidence = evidence;
  else delete target.evidence;
}
```

For a **scalar** field:

1. Empty values are ignored. You can't "set" nothing.
2. If the field already holds a value of **equal or higher** rank, keep it. That's "first good value wins".
3. Otherwise, replace it. So a **scraped** value *can* replace an **inferred** one, but never the other way around.

For a **list**, `addItem` (lines 41-53) adds an item unless one with the same **key** exists. The key function is the caller's choice: phones use `phoneKey` (digits only), addresses use `addressKey` (number + unit + ZIP), and logos use their URL.



Why it's built this way: the homepage and JSON-LD run first and are the most trustworthy, so "first wins" naturally prefers them. The rank check still lets a better kind of evidence (scraped) upgrade a guess (inferred), and nothing the scraper does can overwrite the owner (`user_edited` is rank 5).

Could be simpler: `merge.ts` has `RANK` and `addItem`'s key functions; `overlay.ts` (re-scrape) has its *own* `valueKey`; and `field-tiers.ts` has *another* `valueKey`, which is slightly different (it doesn't JSON-encode strings). Three "is this the same value?" rules in three files is easy to get subtly wrong. One shared `sameValue(path, a, b)` helper would make it consistent.

Check yourself

1. Why is the page loaded into Cheerio twice (`$` and `$text`)?
2. Why must `extractJsonLd` run before `extractAddresses` and `extractMeta`?
3. How does the contact extractor decide whether a phone on `/location/las-vegas` belongs to Las Vegas?
4. Why is an address read from a Google Maps link marked *inferred* and not *scraped*?
5. What stopped "Facebook Inc." from becoming Apex's legal name?
6. Goettl's brand colors used to come out as WordPress blue. What two changes fixed that?
7. Why do testimonials run before people?
8. A scraped industry exists, and a later page offers an inferred one. Which wins, and which line decides?

Answers

1. `$` keeps the raw page (scripts, meta, JSON-LD). `$text` is a readable copy with spaces between elements, because Cheerio's `.text()` glues neighbours ("4 GB"+"RAM" → "4 GBRAM").
2. JSON-LD is the most reliable source, and the rule is "first value wins". Running first lets structured values (name, address) take the field before text heuristics try.
3. It looks at about 150 characters of text around each number: our city mentioned means "ours" (moved to the front); another `City, ST ZIP` means another branch (skipped); neither means it's kept.
4. It comes from a URL parameter, not words the business wrote on the page, so it's a derived guess. A scraped address found later can replace it.
5. `sharesWord`: on legal pages the name must share a distinctive word (4+ letters, not generic) with the company name or domain, and "facebook" matches neither.
6. Colors are ranked by *usage*: buttons ×6, header ×4, links ×2, other ×1, with plugin-only rules ignored. And known platform defaults (`#007cba`, Kadence `#f76a0c`…) are filtered out.
7. So testimonial authors are already tagged `customer_partner` when the people extractor runs. It never re-tags an existing name as team.
8. The scraped one stays. `setField` returns when `RANK[current] >= RANK[new]` (`merge.ts`), and scraped (4) beats inferred (2).

05 · AI enrichment: how "Enrich with AI" stays honest

What you'll learn

1. The full path of one Enrich with AI run, from the button to accepted values in the knowledge base.
2. How a Markdown prompt file becomes a real API call, and the prompting techniques used in `understand-business.v1.md`.
3. The code-level guards (confidence bar, citations, tiers), preview mode, cost and caching.

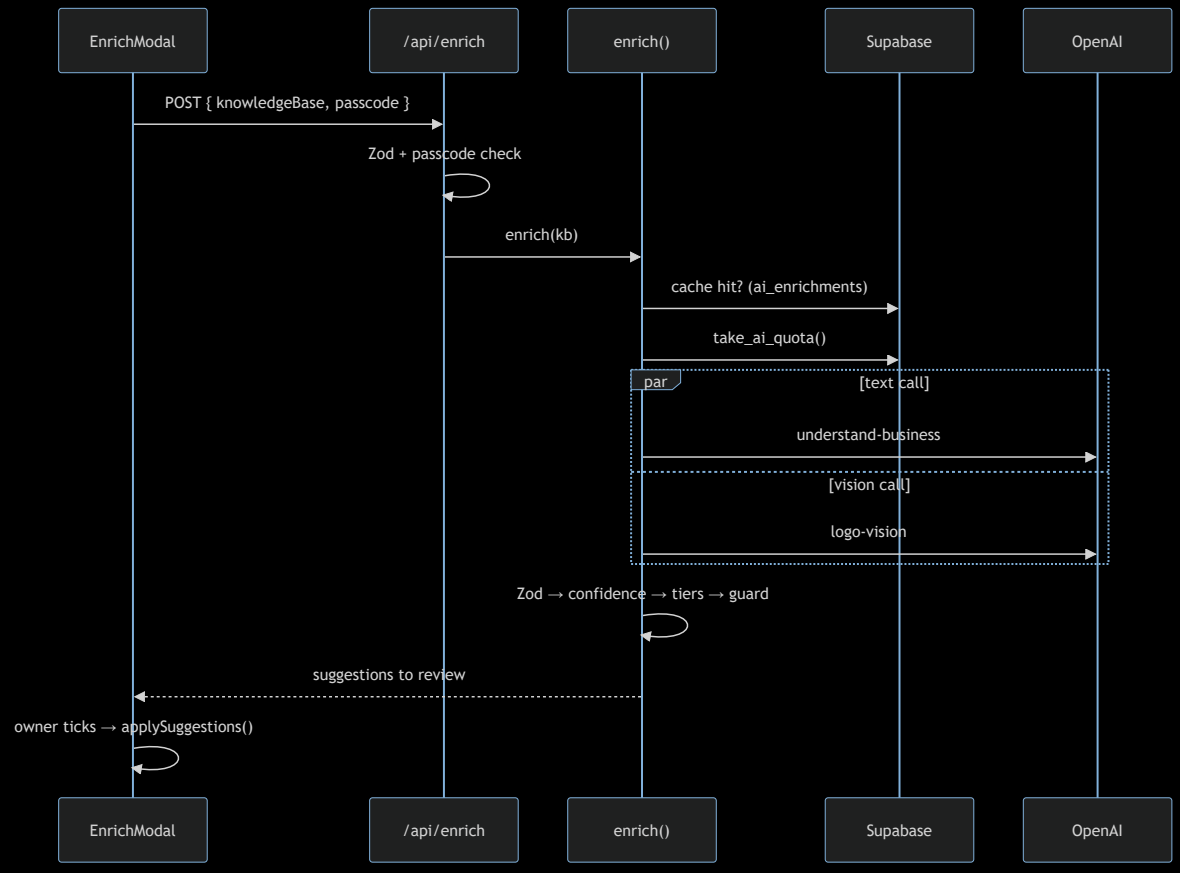
Before this: 02-data-model.md (tiers, confidence), 01-tech-stack.md (Responses API, Zod, `Promise.allSettled`)

---

1. The big idea: AI suggests, code checks, the owner decides

Three rules shape everything in this chapter:

1. **AI never writes directly into the knowledge base.** It returns *suggestions*. The owner ticks which ones to keep (`src/lib/ai/enrich.ts:13`: "Nothing is written here").
2. **Every promise in the prompt is also checked in code.** The prompt says "high confidence only" and "never guess people", and code enforces both even if the model ignores the prompt.
3. **AI is optional.** With no key, no passcode, no quota left, or a failure, the same button returns clearly labeled **preview** suggestions built from your own facts.



---

2. Step by step through the code

2a. The route: passcode first

```
// src/app/api/enrich/route.ts:40-47
  const { passcode, preview } = parsed.data;
  // A wrong passcode is an error the user can fix, not a silent switch to preview.
  if (!preview && liveAvailable() && !passcodeMatches(passcode)) {
    return NextResponse.json({ error: { code: "INVALID_PASSCODE", message: "That passcode isn't right."
} }, { status: 401 });
  }
  try {
    return NextResponse.json(await enrich(kb.data, { preview: preview || !liveAvailable() }));
```

- **`liveAvailable()`** (`src/lib/ai/config.ts`) is true only when the server has **both** `OPENAI_API_KEY` and `AI_PASSCODE`. "Without a passcode we stay in preview mode, so a deploy that forgot it can't run up costs."
- **`passcodeMatches()`** uses `timingSafeEqual`, a **constant-time compare**, so an attacker can't guess the passcode letter by letter from how long the check takes.
- **The browser keeps the passcode in `sessionStorage`**: this tab only, gone when it closes (`src/components/knowledge/ai/passcode.ts`).

> **Why a passcode at all?** The demo is public and has no login. Without a gate, anyone could press the button 1,000 times on Jake's OpenAI bill. The passcode plus a daily cap (below) means a stranger only ever gets preview mode.

2b. `enrich()`: build the evidence, then check the cache and the quota

```
// src/lib/ai/enrich.ts:66-95 (trimmed)
  const system = textSystemPrompt();
  // Evidence gets whatever the input budget leaves after the system prompt.
  const evidence = buildEvidence(kb, LIMITS.inputTokens - estimateTokens(system) - 300);
  // …preview checks…
  const cacheKey = hash({
    // The prompt text itself, so editing a prompt file never serves an old answer
    prompts: [PROMPT_VERSIONS, system, visionSystemPrompt()],
    models,
    payload,
    images: images.map((i) => ({ ...i.meta, url: i.url.split("?")[0] })),
  });
  const cached = await getCachedEnrichment<Omit<EnrichResult, "remainingToday" | "cached" | "hint">>
(cacheKey);
  if (cached) return { ...cached, suggestions: guard(kb, cached.suggestions), cached: true, … };

  if (!(await takeAiQuota(limit))) {
    return preview(kb, evidence, hint, [`Today's live AI limit (${limit} runs) is used up, so this is a
preview. It resets at midnight UTC.`]);
  }
```

The evidence builder (`src/lib/ai/evidence.ts`) turns the knowledge base into a list of short, numbered items the model can cite:

```
{ "id": "p3", "kind": "page:other", "page": "/sakana", "text": "Sakana Las Vegas — headings: AYCE Sushi | Hours" }
```

The items are the overview, each crawled page's title, meta description and headings, CTAs, testimonial *text* (without names), FAQs, locations, offerings and image clues. They're added **most useful first** and cut to a **token budget** (`LIMITS.inputTokens = 16_000`; `estimateTokens` is just `length / 4`, `evidence.ts:35`).

The cache: the key is a **hash** (a short fingerprint, SHA-256) of the prompt *text*, the models, the evidence and the images. Same input means same key, so the stored answer is returned with **no call and no quota**. Editing a prompt file changes the key, so you never get an answer from an old prompt.

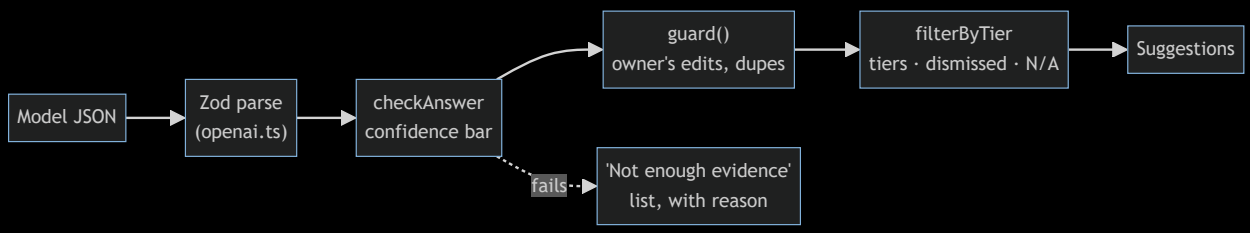
The quota: `takeAiQuota` calls the `take_ai_quota` Postgres function (chapter 02). It increments today's counter only if it's under the limit, in **one statement**, so two clicks at the same instant can't both take the last slot (`…ai_enrichment.sql:31-52`).

2c. Two calls in parallel

```
// src/lib/ai/enrich.ts:97-101
  // One text call and (if there are usable images) one image call, at the same time.
  const [textRun, visionRun] = await Promise.allSettled([
    timed(() => runText(models.text, system, payload)),
    images.length ? timed(() => runVision(models.vision, images)) : Promise.reject(new Error("no raster images")),
  ]);
```

- **The text call** (`prompts/understand-business.v1.md`) fills about 25 tier 2 fields.
- **The vision call** (`prompts/logo-vision.v1.md`) looks at the logo and hero image, plus up to 12 more logos at low detail, for a restaurant group's logo grid (`src/lib/ai/input.ts:1-4`). It returns the **art style** and **brand names read from logos**.
- `allSettled` means one failing doesn't throw away the other (lines 108-129). Only *complete* answers are cached (lines 132-134), "so a timeout isn't remembered for this version forever".

2d. From answers to suggestions: three guards



1. **Zod** (`openai.ts:64-65`): the reply must match `textOutputSchema` exactly, or the call counts as failed.
2. **The confidence bar** (`src/lib/ai/confidence.ts`), next section.
3. **`guard()`** (`enrich.ts:272-289`):
    - never replace a `user_edited` value
    - drop list items that are already there
    - then `filterByTier` drops tier 3, unquoted tier 1, dismissed and N/A (chapter 02, section 5)

Fields that fail the bar aren't hidden. They go to a **"Not enough evidence"** list with the reason, so the owner sees *why* a field stayed empty (`EnrichModal.tsx`, "Not enough evidence (n)").

2e. Review and apply

```
// src/components/knowledge/ai/EnrichModal.tsx:68-79 (trimmed)
    setResult(res.data);
    // Pre-tick list additions and empty fields; replacing something already there is opt-in.
    setChosen(new Set(res.data.suggestions.flatMap((s, i) => (s.list || isEmptyAt(kb, s) ? [i] : []))));
    setStep("review");
  // …
  function apply() {
    const accepted = result.suggestions.filter((_, i) => chosen.has(i));
    applySuggestions(accepted);
    onClose();
    if (accepted[0]) jumpTo(accepted[0].path); // show where the first one landed
  }
```

`applySuggestions` (context) calls `applySuggestionsTo` (`src/lib/ai/apply.ts:17-32`). Accepted values **keep their AI confidence and evidence**, not `user_edited`, so they show a purple "AI" badge with a "Based on: …" tooltip and a **"Wrong? Remove"** button (chapter 07).

---

3. The confidence bar, in code

```
// src/lib/ai/confidence.ts:31-51 (trimmed)
export function checkAnswer(path: string, answer: Answer, values: string[], e: Evidence, inputLower:
string): Verdict {
  const real = [...new Set(answer.evidence.map((c) => resolveCitation(c, e, inputLower)).filter((c): c
is string => !!c))];
  // A direct quote (not an id) that exists in the input and contains the value counts on its own.
  const quoteHasValue = answer.evidence.some(
    (c) => !isId(c) && resolveCitation(c, e, inputLower) !== null && values.some((v) => v.length >= 3 &&
c.toLowerCase().includes(v.toLowerCase())),
  );
  if (GENERATED_FIELDS.has(path)) {
    const ok = answer.confidence !== "low" && real.length >= 1;
    return { ok, evidence: real, reason: … };
  }
  const ok = answer.confidence === "high" && (real.length >= 2 || quoteHasValue);
  // …
}
```

The rules, in words:

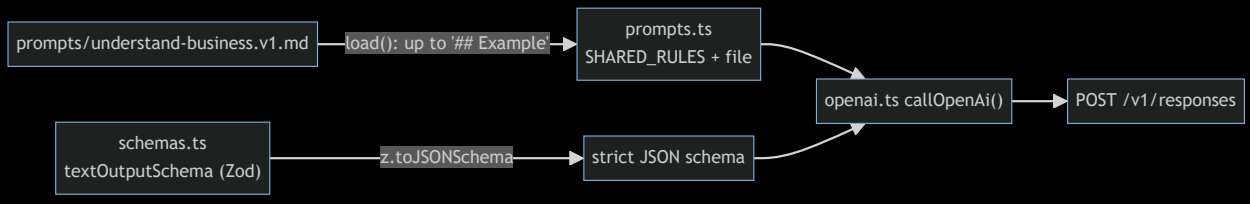
- **A citation is "real"** only if it's an id that exists in the input (`"p3"`) or a quote actually found in the input text. `resolveCitation` (`evidence.ts:146-153`) checks this, so **made-up citations don't count**.
- **Facts** (industry, channels, business model…) need **high** confidence **and** either **2+ real citations**, or **1 quote that contains the value itself**.

- **Generated writing** (pitch, writing style, persona, Content Kit; `GENERATED_FIELDS` in `field-tiers.ts:95-106`) is *written*, not looked up. It needs high or medium confidence and **at least 1** real citation, so it's grounded in something on the site.

> **Why it's built this way:** language models are confident even when wrong, and they sometimes cite things that don't exist. Checking the citations against the actual input turns "trust me" into "show me". Two independent pieces of evidence is the same bar a careful human would use: "Sakana" + "Neko Loco Sushi" + an overview mentioning sushi all point to sushi.

---

4. From a prompt file to an API call



1. **`prompts.ts`** reads the `.md` file at runtime and keeps it **up to its `## Example` heading** (`load()`, lines 20-27). The examples are for humans reading the docs. Then it adds the **shared rules** that every call needs (`SHARED_RULES`, lines 30-35): "Use only the input… Never invent facts… return null and say why… Answer with one JSON object."
2. **`schemas.ts`** defines the answer shape in Zod. The *same* schema is converted to a JSON Schema for OpenAI (`toOpenAiSchema`, lines 81-85) and later used to *validate* the reply. That's one definition used twice.
3. **`openai.ts`** sends `{ model, reasoning: { effort: "none" }, max_output_tokens, store: false, input: [system, user], text: { format: json_schema, strict: true } }` (chapter 01 shows the code).
    - `store: false`: don't keep the conversation on OpenAI's side.
    - `strict: true`: **structured outputs**, where the model can only produce JSON matching the schema.

The schema gives every answer the same envelope:

```
// src/lib/ai/schemas.ts:12-15
/** One scalar answer with its confidence, cited evidence and (when not high) the reason. */
const scalar = z.object({ value: text.nullable(), confidence, evidence: list, reason: text.nullable()
});
/** One list answer, same shape. */
const many = z.object({ values: list, confidence, evidence: list, reason: text.nullable() });
```

---

5. Reading `understand-business.v1.md` like an engineer

Open `prompts/understand-business.v1.md` next to this list. Each section uses a known **prompting technique**:

Section (lines)	What it says	Technique	Why
Role (13-14)	"You are a careful analyst… You write nothing a careful reader couldn't back up"	**Role prompting**	Sets the attitude (careful, evidence-based) before any task

Section (lines)	What it says	Technique	Why
Input format (16-28)	A JSON example of the evidence items and their `kind`s	**Explicit input contract**	The model knows what `p3` or `cta2` means and can cite it
Three tiers (30-33)	Read = not your job; Inferred = your job; Never = don't even mention people, revenue…	**Scope limits / negative instructions**	Says what *not* to do, not just what to do
Confidence bar (35-43)	high = stated or 2 independent items; cite 1–4 ids or exact quotes; `reason` when not high	**Calibrated self-assessment + required citations**	Makes the model show its work so code can check it
Output schema (45-59)	Exact JSON shapes and top-level keys	**Structured output** (also enforced by `strict: true`)	Machine-readable, no prose to parse
Field meanings (61-80)	"`channels`: name the channel type, never the button label ('Book Now' → 'Online booking')"	**Per-field definitions with examples**	Removes ambiguity; small examples steer format
Missing data (82-85)	"An empty field the owner fills is better than a wrong one Flo repeats in every post."	**Explicit permission to abstain**	Models fill blanks unless told that "null" is a good answer
Example (87+)	A fictional input and output, including `null` answers with reasons	**Few-shot example**	Shows the bar in action, including saying "no" (only for humans here; cut before sending)

Two details worth noticing:

- **The persona rule** (line 33): "A persona describes a role ('a first-time server owner'), never a named or gendered person." That's tier 3 in prose.
- **Hashtags** (line 74): "include the brand's own name as a tag, plus its category and city when known". Concrete instructions beat vague ones like "make good hashtags".

> **Could be simpler:** the example section is cut off before sending (`load()` stops at `## Example`), so the model never sees the few-shot example. That saves tokens, but few-shot examples are one of the most reliable ways to steer format. Worth testing whether sending a *short* example improves answers enough to pay for itself.

---

6. Preview mode: no AI, still useful, clearly labeled

When live AI isn't available, `preview()` (`enrich.ts:291+`) combines two sources:

1. **Keyword heuristics** (`src/lib/ai/heuristics.ts`): a small keyword map (`/\bsushi\b/` → Restaurants / Sushi, `/\bplumb(ing|er)\b/` → Home services / Plumbing…; `KEYWORDS`, lines 16+), plus CTA patterns for channels and funnels.
    - **The same 2-evidence rule** applies: a keyword must appear in 2+ evidence items.
    - **Marked** `inferred` (amber badge), with the evidence that triggered it.
2. **Templates** (`src/lib/ai/mock.ts`):
    - **The pitch** is the overview's first sentence, which the business wrote itself. It's skipped if the hero text is glued together.
    - **Content Kit items** come from templates using only existing facts.
    - **Marked** `ai_mock`, with an "AI preview" badge.

- **Writing style, ideal persona and art style are never mocked**, because a template can't fill them honestly (`mock.ts:1-5`).

Both go through the same `guard()`, so preview can't break the tier rules either.

> **Why it's built this way:** the brief says "No required LLM calls… clearly labeled mock outputs by default". Preview mode means a reviewer without a key still sees the whole flow, and nobody mistakes a template for real AI writing.

---

7. Cost, limits and caching

From the comments in `src/lib/ai/config.ts:29-55`:

Thing	Budget	Rough cost on gpt-5.4-mini
Enrich text call	~16k tokens in, 3.5k out, 25 s timeout	worst case ~$0.03 per run
Logo grid (vision)	up to 12 extra logos at low detail	~$0.001
Read menus (chapter 06)	up to 8 pages/images at high detail per run	~$0.01 per page, ≤ ~$0.09 per run
Daily cap	`AI_DAILY_LIMIT` (default 20 runs/day, all users together)	

Caching makes repeats free. The second time the same knowledge base (same evidence) is enriched with the same prompts and models, the answer comes from `ai_enrichments`: no call, no quota, instant. Locally without Supabase, the cache and quota fall back to memory (`src/lib/db/ai.ts:1-5`).

`reasoning: { effort: "none" }` (`openai.ts:42-44`): newer models can "think" before answering, and those hidden tokens count toward `max_output_tokens`. These are summarize-and-format tasks, so the whole budget goes to the answer.

Check yourself

1. Name the two conditions for `liveAvailable()`, and say why the passcode is required even when the key exists.
2. Why does `passcodeMatches` use `timingSafeEqual` instead of `===`?
3. What goes into the cache key, and why is the prompt *text* in it, not just the version name?
4. What makes a citation "real"? What happens to an answer whose citations don't exist?
5. What's the difference between the bar for facts and the bar for generated writing?
6. Why `Promise.allSettled` for the two calls, and why only cache complete answers?
7. Pick two prompting techniques used in `understand-business.v1.md` and explain why each helps.
8. In preview mode, which fields are never suggested, and why?

Answers

1. An `OPENAI_API_KEY` **and** an `AI_PASSCODE` on the server. The passcode keeps a public, login-free demo from running up OpenAI costs; without it everyone gets preview mode.
2. `===` can return faster when early characters differ, which leaks how much of a guess is right. A constant-time compare always takes the same time.

3. The prompt texts and version list, models, the evidence payload, and the images (without signed-URL query strings). Using the text means editing a prompt file, even without bumping its version, never serves an old answer.
4. It's an evidence id that exists in the input (`"p3"`) or a quote actually found in the input text (`resolveCitation`). Answers without enough real citations fail the bar and go to "Not enough evidence" with a reason.
5. Facts need **high** confidence plus 2+ real citations (or one quote containing the value). Generated writing needs high or medium plus at least 1 real citation.
6. So one call failing doesn't discard the other's results. Caching a partial answer would serve a timeout-damaged result forever for that exact input.
7. For example: **role prompting** sets a careful-analyst stance; **required citations** let code verify each answer; **explicit permission to abstain** ("null is better than wrong") reduces filler; **per-field definitions** ("channel type, not button label") remove ambiguity.
8. Writing style, ideal persona and art style. A template can't fill them honestly from facts, so they're left for live AI or the owner.

06 · Menus: PDFs, pictures, prices and brands

What you'll learn

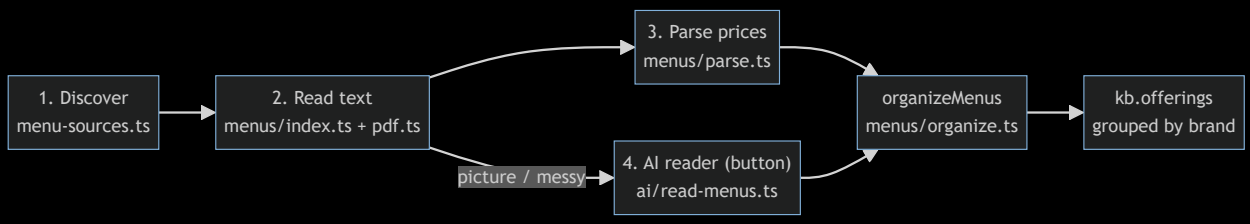
1. How a menu travels from a link on a page to priced offerings under the right restaurant.
2. The "never invent a price" rule, enforced in both the heuristics and the AI path.
3. A real debugging story (Neko Hana) that ties caching, quotas and prompt versions together.

Before this: 03-scrape-flow.md (the crawl), 04-extractors.md (`extractMenuSources`, offerings), 05-ai-enrichment.md (OpenAI calls, cache, quota)

---

1. Why menus get their own chapter

For a restaurant, **what you sell *is* the knowledge**. But restaurants rarely put menus in plain HTML. Dragon Factory, a Las Vegas group of about 20 restaurants, links PDFs, and many of them are photos of printed menus. Phase 10 made offerings "first class" with a four-stage pipeline:



Every menu file is tracked as a `MenuSource` (`src/types/knowledge.ts:271-295`) with a **status**:

Status	Meaning	Shown as (`MENU_STATUS_LABEL`, `src/lib/utils/offerings.ts:34-44`)
`found`	discovered, not read yet	Not read yet
`read`	text read by the heuristics	Read
`no_text`	picture-only PDF, needs AI	Picture menu
`messy`	text found but too jumbled	Needs sorting
`read_ai`	read by AI	Read by AI
`duplicate`	a text copy of a menu already read from its picture	Same as another menu
`too_large` / `blocked` / `failed`	can't be read	shown with the link and a hint

---

2. Stage 1: discovery during the crawl

`extractMenuSources` (`src/lib/scraper/extract/menu-sources.ts`) runs on every page:

- **PDF links** whose text or URL says menu, food, drinks, price, specials, catering… (`MENU_WORD`), or *any* PDF on a menu, pricing or services page, even when the link just says "Download".

- **Big images** (at least 400 px; `MIN_IMAGE_SIDE`) on menu pages, or images named like a menu. Stock photos (Unsplash, Shutterstock…) are skipped.
- **Ordering platforms** (Toast, Square, Clover, DoorDash…) are recorded only as **channels**, never fetched.
- **At most 40 sources** per site (`MAX_SOURCES`), "keeps the saved record small on sites that link hundreds of PDFs".

Menu pages also get a **high crawl priority** (`menu` scores 95 in `discover.ts`), so they make it into the first 15 pages.

---

3. Stage 2: download and read the PDF text (no AI)

After the crawl, `readMenus()` (`src/lib/scraper/menus/index.ts:28-45`) works through the `found` PDFs within strict caps:

```
// src/lib/scraper/menus/index.ts:18-26
export const MENU_LIMITS = {
  maxPdfBytes: 20 * 1024 * 1024,
  maxPdfPages: 4,
  maxPdfsPerScrape: 12,
  budgetMs: 15_000,
  concurrency: 2,
  /** Text kept for messy PDFs, so AI can structure it later without downloading again */
  keepTextChars: 6_000,
};
```

For each PDF (`readPdfSource`, lines 47-84):

1. **robots.txt** applies to files too.
2. `fetchBinary` downloads with the same SSRF checks as pages, plus a **size cap**: refused up front if `Content-Length` is too big, and cut off mid-stream if the server doesn't say (`fetch.ts:135-139`).
3. `isPdf` checks the first bytes are `%PDF-`, so an HTML error page isn't parsed as a menu (`pdf.ts:15-18`).
4. **unpdf** extracts the text of the first 4 pages.
5. `menuTextQuality` decides between `read` (parse it now), `messy` (keep 6,000 characters for AI later) and `no_text` (a picture, which is AI's job).

The file's own name is kept too: `"nabemenu (2).pdf"`, or Wix's `?dn=Captain+6+-+Menu+-+2026.pdf`. It's a clue to which restaurant the file belongs to (`fileNameFrom`, `fetch.ts:116-133`).

---

4. Stage 3: the price parser, and the one hard rule

The header of `src/lib/scraper/menus/parse.ts` says it plainly:

> "The one hard rule: a price is only ever copied from the item's own lines. If no price is printed there, the price stays empty."

`parseMenuLines` (line 99+) reads lines like `California Roll ..... $8.95`, `Edamame 5` or `Gyoza – 6.50` (`TRAILING_PRICE`, line 56). Section words like "Hand Rolls" or "Appetizers" are treated as headings (`SECTION_WORDS`, line 58), which become the item's category. `parsePrice` (lines 43-53) keeps `priceText` **exactly as printed** and computes `priceAmount`:

- **"Market price" / "MP":** kept as text, with no amount, never guessed (line 46).
- **"$10 – $15":** `pricingType: "range"`.
- **Things that look like prices but aren't:** "STEP 2", "Table 4" and "Suite 104" are rejected (`docs/data-quality.md` 1d).

Raised cents. Menus often print "$58^{95}" with small raised cents. In PDFs and AI output that comes out as `$58⁹⁵`, `$58 95` or `$58^95`:

```
// src/lib/scraper/menus/parse.ts:29-37
/**
 * Cents printed raised ("$58⁹⁵") come out of PDFs and models as "$58⁹⁵", "$58 95" or "$58^95".
 * With a dollar sign and exactly two raised digits, that's $58.95. Anything else is left as printed.
 */
export function normalizeCents(raw: string): string {
  return raw
    .replace(/(\$\s?\d{1,4})([⁰¹²³⁴⁵⁶⁷⁸⁹]{2})(?![\d⁰¹²³⁴⁵⁶⁷⁸⁹])/g, (_, d: string, c: string) =>
`${d}.${[...c].map((x) => SUPERSCRIPT[x]).join("")}`)
    .replace(/(\$\s?\d{1,4})(?:\s|\^)(\d{2})(?!\d)/g, "$1.$2");
}
```

Note what it does *not* do: "$5895" stays "$5895", because it can't be told apart from $5,895. When unsure, it leaves the price as printed.

---

5. Stage 4: "Read menus with AI" (a button, not automatic)

Picture menus need a vision model, and that costs money, so it's **a separate button** (`ReadMenusButton.tsx`) with its own route (`/api/menus`) and passcode. That keeps Enrich with AI fast.

Which menus one run reads

```
// src/lib/ai/menus.ts:34-58 (trimmed)
export function pickMenuSources(kb: KnowledgeBase, cap = MENU_LIMITS.units, textCalls =
MENU_LIMITS.textCalls): MenuPlan {
  const pdfs = sources
    .filter((s) => s.kind === "pdf" && PICTURE.includes(s.status))
    .sort((a, b) => Number(!a.group) - Number(!b.group) || (a.bytes ?? Infinity) - (b.bytes ??
Infinity));
  // … images: ones named like a menu first, biggest first …
  for (const source of [...pdfs, ...images]) {
    const units = source.kind === "pdf" ? Math.min(source.pages ?? 2, MENU_LIMITS.pagesPerPdf) : 1;
    if (used + units > cap) { skipped++; continue; }
    vision.push({ source, units });
    used += units;
  }
```

- **The unit cap:** each PDF page or image costs 1 **unit**, and one run reads up to **8 units** plus up to 4 messy-text menus (`MENU_LIMITS` in `src/lib/ai/config.ts:47-55`).
- **Order:** brand pages before hub pages, then smaller files first.

- **What's left over** gets a note: "N more menus found. Run again to read the next batch; menus already read are free."

How one menu is read

`readMenusWithAi()` in `src/lib/ai/read-menus.ts`:

1. **Cache lookup per menu, all in parallel.** The key is `hash({ system prompt, model, url, text })`. Answers from **older prompt versions** (`LEGACY_MENU_PROMPTS = ["menu-reader.v1"]`) are still used for free, unless `fresh` is set.
2. **Quota:** take **one** unit of the daily quota if *anything* needs a live call. A fully cached run is free.
3. **Read every menu in parallel** (`Promise.all`). **Nothing new starts after 40 s** (`deadlineMs`), because the route has 60 s.
4. **PDFs are sent whole** as an `input_file` (base64 data), and OpenAI looks at each page as an image. Images go as `input_image` at high detail.
5. **One last free attempt first:** a PDF the scraper never got to is downloaded and tried as text before paying for vision.

Turning the model's answer into items: confidence rules

```
// src/lib/ai/menus.ts:85-100 (trimmed)
    let verbatim = text !== null && text.includes(norm(name));
    let price = readPrice(raw.price);
    if (price && text !== null) {
      // Every number in the price must be printed in the text, or the price is dropped
      const numbers = price.priceText?.match(/\d+(?:[.,]\d+)?/g) ?? [];
      const inText = numbers.every((n) => text.includes(n));
      if (!inText) price = null;
      verbatim &&= inText;
    }
    // …
    result.push({ item: { name, description, category, price }, confidence: verbatim ? "scraped" :
"ai_live" });
```

Read from	Confidence	Why
a **picture** (image or picture-only PDF)	`ai_live`	nothing to check it against
messy text, sorted by AI, with every name, price, description and section found word for word in the text	`scraped`	AI only *arranged* what's written
messy text, with anything not found	`ai_live`, and **any price whose numbers aren't in the text is dropped**	
any "price" with no digit (other than Market price)	dropped (`readPrice`, lines 110-118)	prices are never invented

---

6. Organizing: which restaurant does each item belong to?

On a multi-brand site, a menu found on `/sakana` belongs to Sakana. But a PDF linked only from a "hub" page ("All you can eat sushi") has no obvious owner. `organizeMenus` (`src/lib/scraper/menus/organize.ts:1-13`) decides, and it runs both after the scrape and **in the browser** after every AI read. It's a **pure function**: same

input, same output, and it doesn't change the input ("Work on copies: the input may be React state", line 26).

The rules, in order:

0. **Clean names:** "Hwaro 2 Ayce Kbbq In Las Vegas" → "Hwaro 2", "Neko Hana Omakase" → "Neko Hana".
1. **A menu with no brand gets one:**
    - **a.** its file name, link text or page title **equals one brand's FULL name** (`"nabemenu (2).pdf"` → "Nabe")
    - **b.** 60%+ of its items (at least 5) are already on one brand's menu (`overlapBrand`, lines 106-122)
    - **c.** otherwise it's **named after itself**: "All You Can Eat Hotpot Menu (PDF)". Never an unnamed "Other items" bucket (`selfName`, lines 125-130).
2. **A text copy of a menu that AI already read from its picture** is marked `duplicate`. Items only in the copy are kept, and the kept items now cite both files (lines 55-78).
3. **Within a brand, one item per name:** the priced and described one wins, citing both menus (`dedupe`, line 137+).

The full-name rule is the important one (`src/lib/scraper/menus/brands.ts:1-7`):

> "Neko", "Neko Supremo", "Neko Loco" and "Neko Hana" are four restaurants; "Sumo Sushi" and "Sushi Sumo Decatur" are two.

`normName()` makes "Sumo Hotpot & Sushi" equal "SUMO HOTPOT AND SUSHI", and nothing looser. A "contains" or "starts with" match would merge different restaurants' menus. That's the bug this rule prevents.

---

7. Review before Flo uses a price

AI-read items start **unreviewed**: `needsReview(f)` is true when the confidence is `ai_live` or `ai_mock` (`src/lib/utils/offerings.ts:66`).

- **On screen:** the Overview flags their prices "needs review", and the menu view shows **Mark as reviewed (n)** per brand (`MenuView.tsx:117-118`).
- **Mark as reviewed** (`reviewGroup`, `offerings.ts:72-82`) turns each one into `user_edited` with a `reviewedAt` timestamp. The badge then reads **"Reviewed"** (green), and the source and evidence are kept.
- **Kept out of every prompt:** until reviewed, AI-read prices are left out of everything sent to AI:

```
// src/lib/ai/evidence.ts:155-162
/**
 * A price AI may see and quote: anything except prices AI read from a menu picture that the owner
hasn't
 * reviewed yet ("Mark as reviewed"). Small print and raised cents are easy to misread, so those stay
out
 * of every prompt until the owner checks them.
 */
export function quotablePrice(f: Field<Offering>): string | null {
  return needsReview(f) ? null : (f.value?.priceText ?? null);
}
```

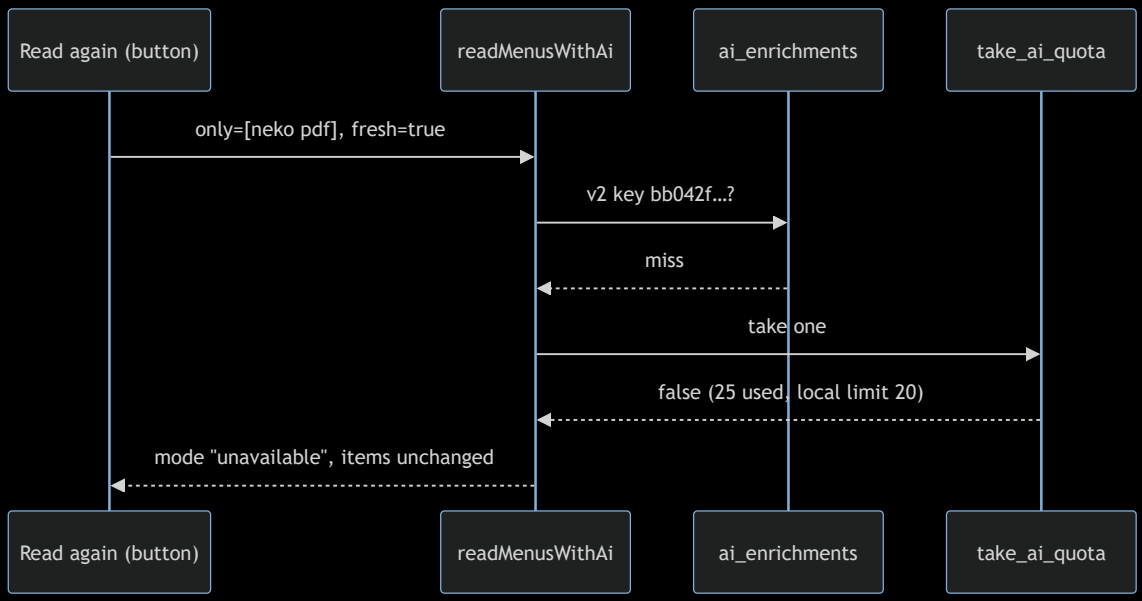
> **Why it's built this way:** a wrong price in a social post is a real problem for a restaurant: customers show up expecting $48.95. Being honest about what's unchecked, and keeping it out of generated content until checked, is cheaper than fixing a public mistake.

---

8. Worked example: Neko Hana's missing omakase prices

This really happened on October 9, 2026, and every number below comes from the database (`ai_enrichments`, `ai_usage`) and the code.

The symptom. Neko Hana Omakase's menu is a 2-page picture PDF. After "Read menus with AI", its 16 omakase courses appeared, but the two things customers actually buy, **"Omakase AYCE $58.95"** and **"Omakase $48.95"**, were missing. Jake clicked **"Read again with AI"**, and nothing changed.



Step 1: what was cached? One answer existed for that PDF, with key `5f68204ca154…`, written at **20:49 UTC**. Recomputing the cache key locally for each prompt version gave:

- `menu-reader.v1` → `5f68204ca154` ✓ (the match)
- `menu-reader.v2` → `bb042f0b93f9`, with no row

So the stored answer was from **v1**. That fits the timing: v2 was committed at 21:11 UTC. The v1 answer had put "OMAKASE ALL YOU CAN EAT 16 COURSE EXPERIENCE" only as a **section heading** over the 16 dishes, with no item and no price.

Step 2: why didn't "Read again" fix it? The button sends `only` + `fresh: true`, so the v1 answer was correctly skipped and a live v2 call was needed. But today's counter in `ai_usage` was **25**, and locally `AI_DAILY_LIMIT` wasn't set, so the limit was the default **20**. `takeAiQuota` returned false, and the job was skipped silently:

```
// src/lib/ai/read-menus.ts (job loop, before the fix)
      if (out) cachedCount++;
      else if (!quota) return;
```

The only message was the generic "Today's live AI limit is used up", which was easy to miss, and nothing was replaced, so it looked like "AI ignored me".

Fix 1: the prompt (v2). `prompts/menu-reader.v2.md` adds rule 3:

> "**A large headline with its own price is an item, not just a section title.** All-you-can-eat packages, course menus and lunch/dinner/all-day tiers ("OMAKASE AYCE, 16 course menu, Including Unlimited Nigiri, $58.95", "LUNCH $26.95") are things customers buy."

It also tells the model to write raised cents as "$58.95" and to read every page. v1 is kept in `prompts/` so its cached answers stay free.

Fix 2: say it by name (commit `a2c9b94`). When a chosen menu can't be read again, the note now names it and says nothing changed: "*Neko Hana Omakase wasn't read again: today's live AI limit is used up. Its current items are unchanged.*" It's shown in an amber warning box, and there's a test for it in `read-menus.test.ts`.

Fix 3: configuration. Jake added `AI_DAILY_LIMIT=100` to `.env.local` (Vercel already had a higher limit).

The result. The re-read at **22:43 UTC** stored v2 answer `bb042f0b93f9` with 36 items, including:

- `OMAKASE AYCE | $58.95`
- `OMAKASE | $48.95`
- `OMAKASE ALL YOU CAN EAT 16 COURSE EXPERIENCE | $58.95`

> **Could be simpler / watch out:** that third line is the *headline* read as an item too, so it duplicates "OMAKASE AYCE" under a longer name. `dedupe` only merges items with the same normalized name, so both survive. A rule like "same brand + same price + one name contains the other's key words → keep the shorter one" (or simply asking the owner to remove it once) would clean it up.

Lessons:

1. **Caches need versions in their keys.** Here the key included the prompt text, so v1 and v2 never got mixed up.
2. **"Nothing happened" is the worst error message.** When a user asks for an action and it's skipped, say what was skipped and why.
3. **Check configuration before blaming the model.** The model was never even called.

Check yourself

1. List the menu source statuses and say which ones the AI reader picks up.
2. What is "the one hard rule" in `parse.ts`, and how is the same rule enforced for AI-read menus?
3. Why is "$5895" *not* turned into "$58.95"?
4. How many units does a 3-page picture PDF cost, and what's the cap per run?
5. When does an AI-read menu item get `scraped` confidence instead of `ai_live`?
6. Why must brands match by their FULL name? Give the Neko example.
7. What does Mark as reviewed change, and what does it unlock?
8. In the Neko Hana story, why was "Read again" a cache miss *and* a no-op?

Answers

1. found, read, no_text, messy, read_ai, duplicate, too_large, blocked, failed. The AI reader takes `no_text` and `found` PDFs, `found` images, and `messy` text (`pickMenuSources`; `waitingForAi` in `offerings.ts:47`).

2. A price is only copied from the item's own printed lines, and otherwise stays empty. For AI: a price with no digit is dropped (`readPrice`), and for messy text any price whose numbers aren't in the source text is dropped (`menus.ts:87-92`).
3. Without a visible separator it can't be told apart from $5,895, and the code only converts when there's a dollar sign plus exactly two raised or separated digits.
4. 3 units (one per page, up to 4 per PDF). The cap is 8 units per run, plus up to 4 messy-text menus.
5. When it came from messy *text* and every name, price, description and section appears word for word in that text. AI only arranged it.
6. Different restaurants share words. "Neko", "Neko Supremo", "Neko Loco" and "Neko Hana" are four places, so a partial match would merge their menus.
7. Each unreviewed AI item becomes `user_edited` with `reviewedAt` ("Reviewed" badge). Its price becomes quotable in AI prompts (`quotablePrice`).
8. Cache miss: `fresh: true` skipped the v1 answer, and no v2 answer existed yet. No-op: a live call needed quota, the counter (25) was over the local default limit (20), so the job returned without reading, and the old items stayed.

07 · The UI: components, context and one click traced end to end

What you'll learn

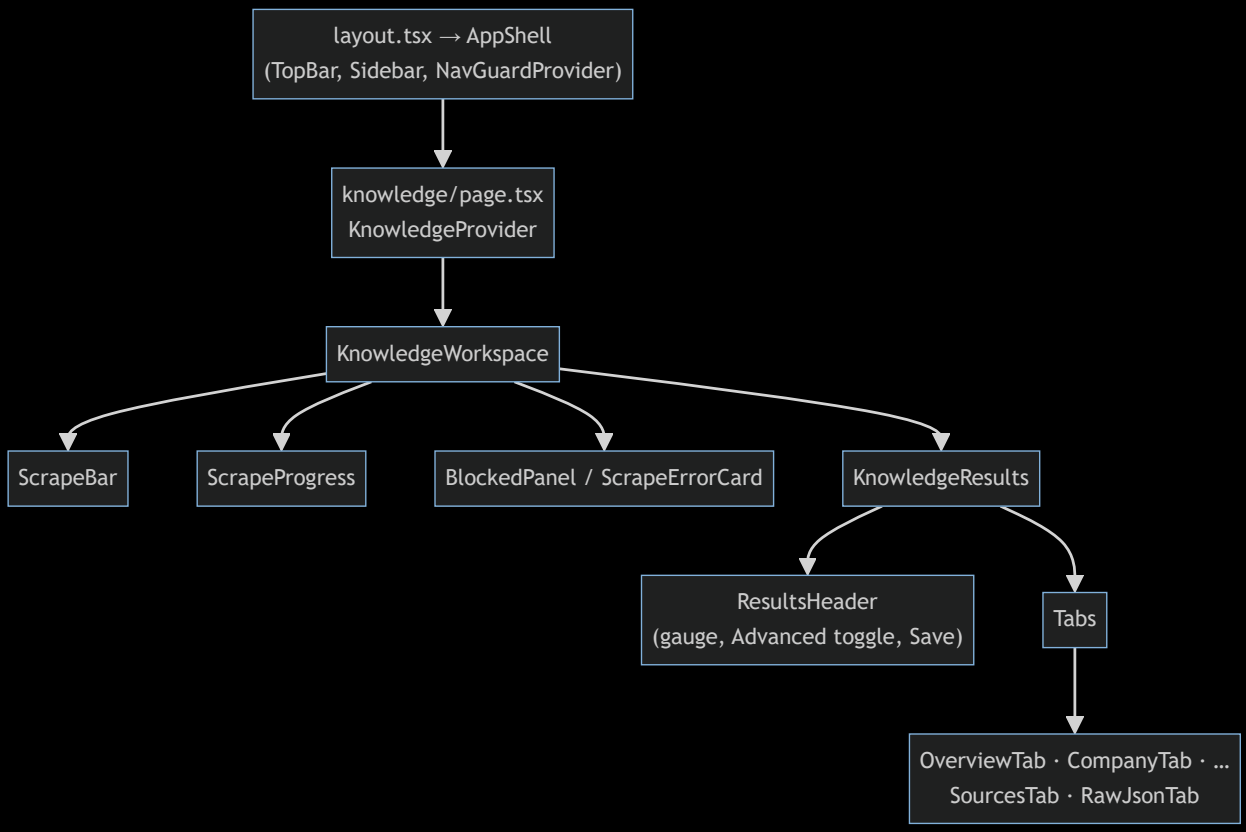
1. The component tree of both pages, and where the knowledge base lives while you edit.
2. Every action in `KnowledgeContext`, who calls it, and the "edit → rescore → dirty" loop.
3. How tabs, the health score, Next to do, quick picks, badges, animations and the layout fit together.

Before this: 01-tech-stack.md (client components, hooks), 02-data-model.md (`Field` , confidence)

---

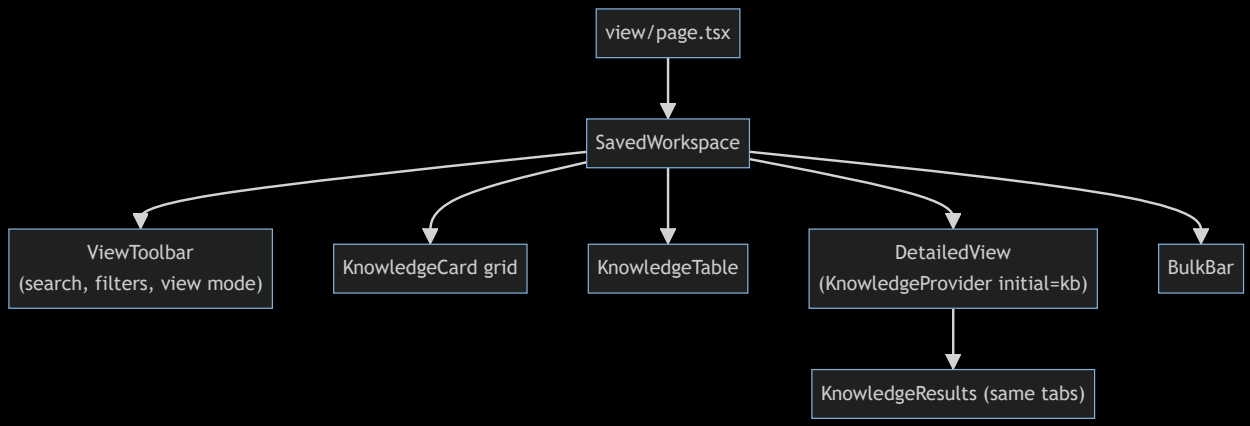
1. Component trees

`/knowledge` (the builder)



- **`KnowledgeWorkspace`** (`src/components/knowledge/KnowledgeWorkspace.tsx`) is the page's brain: `status` is `idle` , `loading` , `opening` , `error` or `done` (line 32). It shows the scrape bar, then progress, then results or an error. With `?id=<uuid>` it opens a saved record instead of scraping.
- **`KnowledgeResults`** (`KnowledgeResults.tsx:43-94`) shows the header, crawl summary, low-score banner, the tab strip and the active tab.

`/knowledge/view` (the saved list)



`DetailedView` reuses the **same** `KnowledgeResults` and tabs inside its own `KnowledgeProvider initial={kb}`, so editing a saved record works exactly like editing a fresh scrape. Chapter 08 covers this page's filters and bulk actions.

---

2. `KnowledgeContext`: where the knowledge base lives

Context is React's way to share state with a whole subtree without passing props down every level. `KnowledgeProvider` (`src/context/KnowledgeContext.tsx:102-236`) holds:

State	Meaning
`kb`	the knowledge base being edited (or `null` before a scrape)
`saved`	`{ id, version }` once saved; `null` for a new scrape
`dirty`	unsaved changes? Drives the Save button and the "leave page?" warning
`advanced`	Advanced view on/off
`tab`	the active tab
`busy`	a save, dig or AI read is running, so editing is paused

Components read with `useKnowledge()`, or `useField(path)` / `useList(path)` for one value by dot path (lines 238-254).

The edit loop

Every change goes through one function:

```
// src/context/KnowledgeContext.tsx:129-133
  // All edits go through here: apply the change, rescore, mark dirty.
  const edit = useCallback((change: (kb: KnowledgeBase) => KnowledgeBase) => {
    setKb((prev) => (prev ? finalize(change(prev)) : prev));
    setDirty(true);
  }, []);
```

`finalize` (lines 84-88) recomputes `completeness` (so the gauge moves live) and keeps `companyName` in sync. Changes use **`setAt`** (`src/lib/utils/path.ts:13-20`), which **never mutates**: it copies each object along the path. React only notices a change when it gets a *new* object, so mutating in place would leave the screen stale.

Every action and who calls it

Action	What it does	Called from
`loadKb(kb, {dirty})`	replace the whole kb	KnowledgeWorkspace (scrape, open), useKnowledgeActions (dig)
`markSaved(kb)`	store the saved id/version, clear dirty	useKnowledgeActions.save
`setField(path, v)`	set one value as **user_edited** (`source: "user"`); clearing it makes it Missing	EditableField, QuickPicks
`addItem / updateItem / removeItem`	list edits as user_edited; clearing an item removes it	EditableList, RecordList, RecordForm
`applySuggestions(s[])`	accepted AI values, **keeping** their AI confidence and evidence	EnrichModal
`applyMenuResult(r)`	merge a "Read menus with AI" result, then `organizeMenus`	ReadMenusButton
`reviewMenuGroup(brand)`	"Mark as reviewed"	MenuView
`dismissValue(path, i?)`	"Wrong? Remove": clear and **remember**	AiValueTag (via EditableField, EditableList, RecordList, MenuView, OfferingsTab)
`setNotApplicable(path, on)`	mark/unmark N/A (counts as complete)	OverviewTab, QuickPicks, SourcesTab
`promoteLocation(i)`	make another location the main address	OverviewTab ("Use one of your locations")
`jumpTo(path)`	switch tab (turning Advanced on if needed), scroll, flash the field	Next to do, EnrichModal

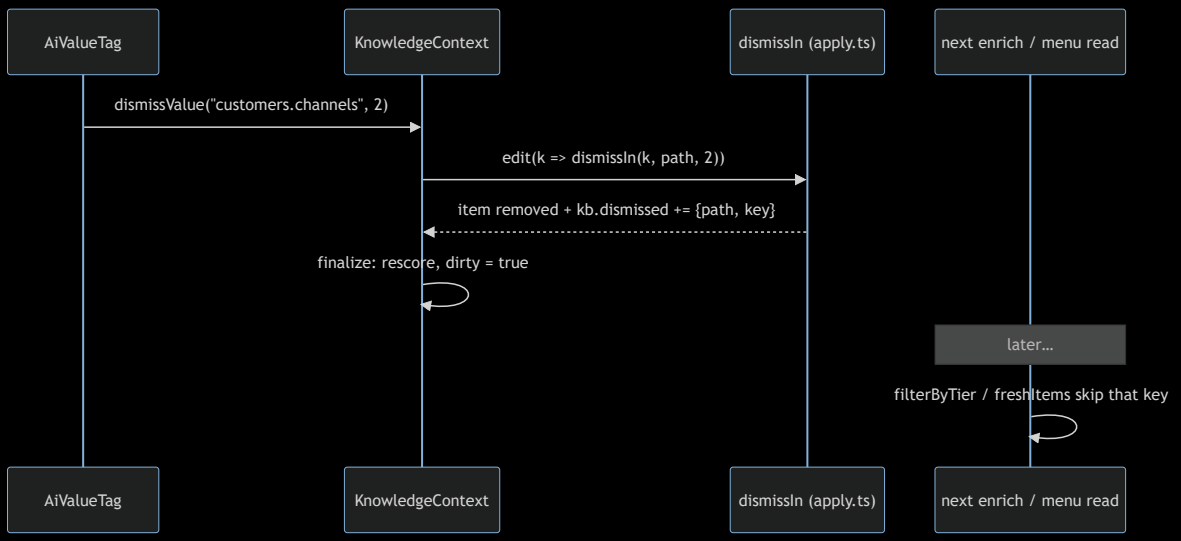
Save and Dig deeper live in a hook, **`useKnowledgeActions`** (`src/components/knowledge/useKnowledgeActions.ts`), because they talk to the API and show toasts. They set `busy` while running so nothing typed mid-crawl gets overwritten.

> **Why it's built this way:** the brief allows "React hooks and context only". One context with one `edit()` funnel means every change, whatever the source, rescores the gauge and marks unsaved changes the same way. It also means there are no bugs where one button forgot to update the score.

---

3. One click, traced: "Wrong? Remove"

An AI value shows a purple badge and a small **Wrong? Remove** link. What happens when the owner clicks it?



1. **The button** (`src/components/ui/AiValueTag.tsx:30-41`) appears only when `isAi(confidence)`, meaning `ai_live`, `ai_mock` or `inferred` (`src/lib/utils/fields.ts:21`). `stopPropagation()` keeps the click from also opening the editor underneath.
2. `dismissValue` wraps it in `edit(...)`, so it rescores and marks the page dirty.
3. `dismissIn` (`src/lib/ai/apply.ts:34+`) clears the value (or removes the list item) and appends `{ path, key, at }` to `kb.dismissed`. The key is the normalized value. For offerings it's **brand + name** (`offeringKey`), so Sakana's Dragon Roll is a different item from Umami's.
4. **Next time:**
   - Enrich with AI's `filterByTier` drops any suggestion whose `path|key` is in `dismissed` (`field-tiers.ts:139,148-149`).
   - "Read menus with AI" uses `freshItems` (`src/lib/ai/menus.ts:129-132`) to skip removed items.

What's *not* covered (see chapter 08):

- **A re-scrape** starts a fresh knowledge base, and `keepUserEdits` (`overlay.ts`) doesn't copy `dismissed`, so the list is lost.
- **The scraper** itself never reads `dismissed`.
- **Scraped values** don't get a "Wrong? Remove" button at all. Deleting a scraped list item goes through `removeItem`, which records nothing, so the next scrape simply finds it again.

---

4. Tabs and Advanced view

`KnowledgeResults.tsx:27-41` defines two tab groups:

- **Basic:** Overview, Company, Customers, Brand, People, Offerings.
- **Power** (shown when Advanced is on): Insights, Content Kit, Sources, Raw JSON.

Two smart touches:

- **Offerings first for food and retail.** If menus were found, or the industry looks like food, drink or retail, Offerings moves right after Overview (`offeringsFirst`, `src/lib/utils/offerings.ts:13-17`).
- **Turning Advanced off** while on a power tab falls back to Overview (lines 62-65).

Tab changes animate with `AnimatePresence mode="wait"`: the old tab fades out *before* the new one fades in (lines 78-91).

In **Advanced view**, every field shows its confidence badge. In basic view, only AI and inferred values do, because those are the ones the owner should double-check (`EditableField.tsx:56-62`).

---

5. The health score and "Next to do"

The score (`src/lib/scraper/score.ts`) is a weighted checklist: 23 checks whose weights add to 100. Name is 8, overview 7, offerings 7, then 5s and 4s down to 2s (`SCORE_CHECKS`, lines 15-38).

```
// src/lib/scraper/score.ts:60-70
export function scoreCompleteness(kb: KnowledgeBase): Completeness {
  let score = 0;
  const missing: string[] = [];
  // "Not applicable" fields (marked by the owner, e.g. no head office) count as complete
  const na = new Set(kb.notApplicable ?? []);
  for (const check of SCORE_CHECKS) {
    if (check.filled(kb) || na.has(check.path)) score += check.weight;
    else missing.push(check.path);
  }
  return { score: Math.min(100, score), missing };
}
```

The same `missing` list serves three purposes: the gauge, the **Next to do** cards, and the adaptive crawl (chapter 03). One definition, three uses.

The gauge (`src/components/ui/Gauge.tsx`) is an SVG circle whose stroke animates with Framer Motion. It's amber below 50, blue from 50, and green from 80 (line 13), "styled like MoFlo's Brand Power".

Next to do (`src/components/knowledge/tabs/OverviewTab.tsx`):

- **The cards:** the 4 missing checks with the biggest weights, each with a "+N" badge.
- **Buttons by tier:** tier 2 fields get **Fill with AI**; tier 3 gets **Add it yourself**, never AI.
- **Not applicable** is offered for every card. The main-address card can say "Not applicable (no head office)", or offer "Use one of your locations".

---

6. Editing in place, and quick picks

- **`EditableField`** (`src/components/ui/EditableField.tsx`): shows a value, or a dashed **"+ year"** pill when empty. Click to edit inline; Enter or blur saves, and Esc cancels. It only writes when the text actually changed, "so we don't mark untouched fields as edited". The `alwaysBadge` prop shows the badge outside Advanced view; business facts use it so a scraped "LLC" shows "Scraped" with its quote.
- **`EditableList`**, **`RecordList`**, **`RecordForm`** do the same for lists and multi-field items (people, offerings, FAQs).
- **`QuickPicks`** (`src/components/knowledge/QuickPicks.tsx`, Phase 11): one-tap pills for entity type, employees and revenue. A pick is stored as user_edited. "Not sure" and "Prefer not to say" mark the field Not applicable instead. The options and that rule are a tiny pure module, `src/lib/utils/quick-picks.ts`, with its own test.

---

7. Badges and tooltips

`ConfidenceBadge` (`src/components/ui/Badge.tsx:43-53`) maps confidence to a label and color. Its **tooltip** is the HTML `title` attribute:

```
// src/components/ui/Badge.tsx:46-47
  // Tooltip: what the value is based on (AI evidence), or where it was read
  const title = evidence?.length ? `Based on:\n${evidence.map((e) => `• ${e}`).join("\n")}` : source ?
`Source: ${source}` : undefined;
```

So hovering "AI" on a channel shows `Based on: • /menu: "order online" → …`, and hovering "Scraped" shows `Source: https://…/about`. An AI value the owner reviewed reads **"Reviewed"** in green instead of "User edited" (line 45).

> **Could be simpler / later:** `title` tooltips don't appear on touch screens (iPad, phones). A small popover on tap would make evidence visible on mobile.

---

8. Animations (Framer Motion)

Effect (from the brief)	Where
Staggered dropdown items (~40 ms apart)	`src/components/ui/Menu.tsx` (`motion.ul` with `variants`, lines ~118-151)
Modals fade and float in	`src/components/ui/Modal.tsx`
Tab content fade	`KnowledgeResults.tsx:78-91`
Gauge arc animates to the new score	`Gauge.tsx` (`motion.circle`)
First-visit spotlight tour	`src/components/tour/Tour.tsx`
Quick pick pills stagger in	`QuickPicks.tsx` (`delay: i * 0.04`)

The tour is clever: the page "dims" because the highlight ring has a **huge box-shadow** around it, so the highlighted element stays bright without being cloned or moved (`Tour.tsx:3-10`). The ring moves between elements with a **spring** animation. Steps live in `tourSteps.ts`; each one finds its element by a `data-tour="…"` attribute (scrape bar, tabs, health gauge, Advanced toggle, save). `tourStorage.ts` remembers that you've seen it.

All motion respects **"reduce motion"**: `<MotionConfig reducedMotion="user">` in `src/app/providers.tsx`.

---

9. Layout and responsiveness

- **`AppShell`** (`src/components/shell/AppShell.tsx`): the top bar, a thin icon sidebar on desktop (a bottom bar on phones), and a centered content column (`max-w-6xl`). The padding classes `pt-20 sm:pl-24 lg:pl-28` make room for the bars at each screen size.
- **Tailwind breakpoints** (`sm:` ≥640px, `md:` ≥768px, `lg:` ≥1024px) switch grids from one column to two or three, as in `grid-cols-1 lg:grid-cols-3` in OverviewTab.
- **The tab strip and wide rows** scroll horizontally on phones instead of squashing.
- **`NavGuardContext`** warns before leaving the page with unsaved changes. `KnowledgeProvider` turns it on whenever `dirty` is true (lines 111-116).

Check yourself

1. What six pieces of state live in `KnowledgeProvider`?
2. Why must `setAt` copy objects instead of changing them in place?
3. What three things happen on *every* edit through `edit()`?
4. Which confidences show a "Wrong? Remove" button, and what does clicking it record?
5. Why do accepted AI suggestions keep `ai_live` instead of becoming `user_edited`?
6. Where does the list of "Next to do" cards come from, and which other feature uses the same list?
7. How does the tour dim the page without hiding the highlighted element?
8. Name one UX gap on touch devices mentioned in this chapter.

Answers

1. `kb`, `saved`, `dirty`, `advanced`, `tab`, `busy`.
2. React decides whether to re-render by comparing object identity. A mutated object is the "same" object, so the screen wouldn't update.
3. The change is applied, `finalize` rescores (and syncs `companyName`), and the page is marked dirty.
4. `ai_live`, `ai_mock` and `inferred` (`isAi`). It removes the value and adds `{ path, key, at }` to `kb.dismissed`, with offerings keyed by brand + name.
5. So the owner can still see they came from AI: the badge, the "Based on" evidence, and the "Wrong? Remove" button stay. Editing the value later makes it `user_edited`.
6. From `completeness.missing` (`scoreCompleteness`), sorted by weight. The adaptive crawl (`crawlForMissing`) uses the same list.
7. The highlight ring has an enormous box-shadow that darkens everything around it, while the ring itself is transparent.
8. Badge tooltips use the HTML `title` attribute, which doesn't show on touch screens.

08 · Saving, versions, re-scrape and the saved list

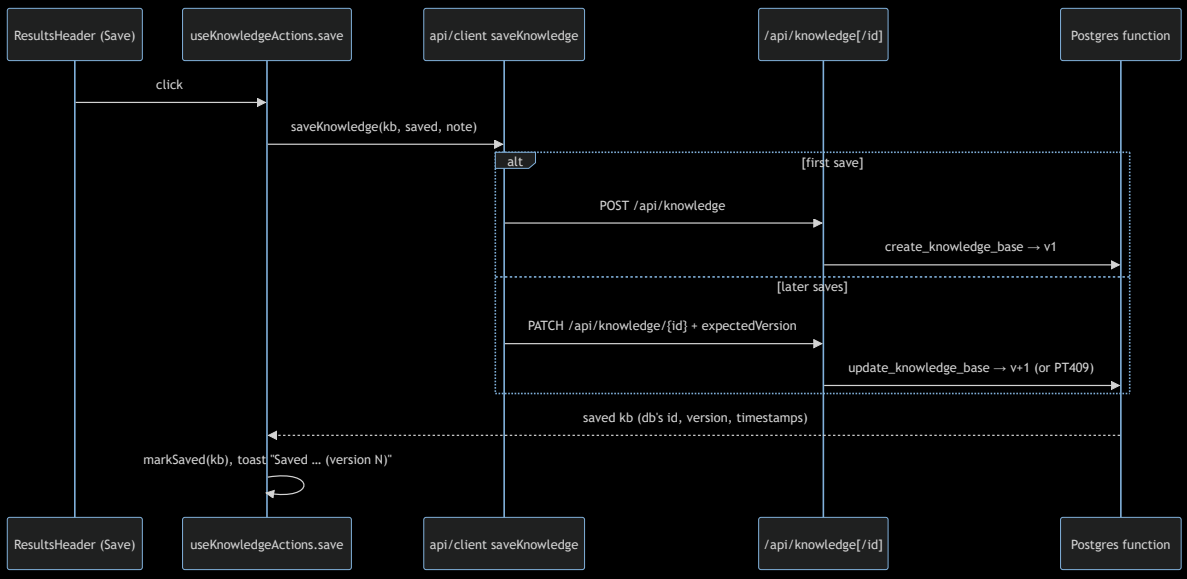
What you'll learn

1. Exactly what happens when you press Save: the request, the database function, the new version, and the 409.
2. How versions, restore, duplicate and re-scrape work, and what survives a re-scrape (and what doesn't).
3. How `/knowledge/view` keeps filters in the URL and does bulk actions.

Before this: 02-data-model.md (tables, RPCs, optimistic concurrency), 07-ui.md (context, `markSaved`)

---

1. Save, from button to database



The client decides POST or PATCH based on whether this knowledge base was saved before:

```
// src/lib/api/client.ts (saveKnowledge)
/**
 * First save creates a record (POST); later saves add a version (PATCH).
 * `note` is stored with the version ("Re-scraped", "Restored from version 2"...).
 */
export async function saveKnowledge(
  kb: KnowledgeBase,
  saved: { id: string; version: number } | null,
  note?: string,
): Promise<ApiResult<KnowledgeBase>> {
  const res = saved
    ? await call<KbResponse>(`/api/knowledge/${saved.id}`, {
        method: "PATCH",
        body: JSON.stringify({ knowledgeBase: kb, expectedVersion: saved.version, note }),
      })
    : await post<KbResponse>("/api/knowledge", { knowledgeBase: kb, note });
  return res.error ? res : { data: res.data.knowledgeBase };
}
```

Then:

1. **The route validates** the whole knowledge base with Zod (`src/app/api/knowledge/[id]/route.ts:14-19`) and the id as a UUID (line 12).
2. `updateKnowledgeBase` (`src/lib/db/knowledge.ts:117-132`) recomputes the score (`withScore`, so the `completeness` column always matches the content) and calls the Postgres function.
3. **The function** locks the row, checks `expectedVersion`, bumps the version, updates `knowledge_bases`, inserts a full snapshot into `knowledge_versions`, and records consent (chapter 02, section 9).
4. **The database's answer comes back:** its id, version and timestamps. `markSaved` stores `{ id, version }` and clears `dirty` (`KnowledgeContext.tsx:123-127`).

Notes on each version:

- **First save:** "Initial save" (set in SQL).
- **Later saves from the builder:** "Edited" (`useKnowledgeActions.ts:30-31`).
- **Other actions:** "Re-scraped", "Restored from version N", "Duplicated from X".

What the user sees on a conflict: if someone saved in between, the function raises `PT409`, which becomes HTTP **409**, which becomes the toast "*Someone saved a newer version. Reload to see the latest changes.*" (`src/lib/db/errors.ts:29-30`).

> **Why the database owns id, version and timestamps:** if the browser chose them, two tabs could both claim "version 3". The function assigns them inside a locked transaction, so they're always unique and in order (`…120000_knowledge_schema.sql:221-227`).

---

2. The JSON output

The brief says Save "also produces the JSON". The same object is:

- shown in the **Raw JSON** tab (copy and download; `RawJsonTab.tsx`),

- exported from `/knowledge/view` (one record, or several in one file; `useRecordActions.exportJson`, lines 77-91),
- and saved as real examples in `data/examples/` (Apex, Dragon Factory, Goettl).

---

3. Versions and restore: history is never rewritten

Every save is a new row in `knowledge_versions` with the full snapshot. The **VersionDrawer** (`src/components/view/VersionDrawer.tsx`) lists them (`GET /api/knowledge/{id}/versions`: version, score, note, date; no snapshots, to keep the list light) and can open any one (`GET …/versions/{n}`).

Restore doesn't go back in time. It copies forward:

```
// src/components/view/useRecordActions.ts:113-128 (trimmed)
  /** Save an old version's snapshot as the newest version. */
  async function restore(id: string, version: number, currentVersion: number): Promise<KnowledgeBase |
null> {
    const snap = await getVersion(id, version);
    // …
    const saved = await saveKnowledge(snap.data, { id, version: currentVersion }, `Restored from version
${version}`);
    // …
    toast({ tone: "success", title: `Restored version ${version}`, text: `Saved as version
${saved.data.version}. Nothing was deleted.` });
```

Restoring v2 when you're on v5 creates v6, a copy of v2. Versions 3–5 are still there, so a restore can itself be undone.

---

4. The saved list: `/knowledge/view`

The view lives in the URL. Mode (card, table, detailed), search, industry, health band, date range, sort and the open record are all **query parameters** (`src/lib/view/filters.ts:1-5`). "Any view can be bookmarked or shared", and the browser's back button works naturally. `parseView` checks each value against the allowed list and falls back to defaults, so a hand-edited URL can't break the page.

Filters run in the database, not the browser. `listKnowledgeBases` (`src/lib/db/knowledge.ts:84-111`) builds the query:

- **Search:** `ILIKE '%term%'` on name, URL and industry, fast thanks to trigram indexes. Characters with special meaning in the query syntax are stripped first (line 95).
- **Health band:** `completeness` between min and max (needs work 0–49, good 50–79, great 80+).
- **Date range:** `updated_at` within the last 7/30/90 days.
- **Paging and sorting:** done in SQL.
- **The list query** reads only summary columns, plus the logo list via PostgREST's JSON path `logos:data->brand->logos` (lines 17-20), never the whole document.

Actions (`src/components/view/useRecordActions.ts`):

Action	What it does
Edit	Detailed mode: the full tabs inside `KnowledgeProvider initial={kb}`

Action	What it does
Duplicate	loads, renames to "X (copy)" (as a user edit), and saves as a *new* record
Re-scrape	fresh crawl + `keepUserEdits`, saved as the next version (section 5)
Export JSON	one file, or one combined file for several (`Promise.all` loads them in parallel)
Delete	with a confirm modal. Versions, crawl runs and consents **cascade** (`on delete cascade`); screenshots in Storage are removed separately, "best effort" (`knowledge.ts:138-145`)
Bulk	select several → `BulkBar` slides in → export together or delete together. Deletes run one at a time so partial failures can be reported as "Deleted 3 of 4"

---

5. Re-scrape: what survives?

Re-scraping means: crawl the site again from scratch, then **put the owner's work back**:

```
// src/lib/utils/overlay.ts (overlay)
function overlay(fresh: unknown, prev: unknown): unknown {
  if (isField(prev)) return prev.confidence === "user_edited" ? prev : fresh;

  if (Array.isArray(prev) && Array.isArray(fresh)) {
    const have = new Set(fresh.map((f) => (isField(f) ? valueKey(f.value) : valueKey(f))));
    const edits = prev.filter((f) => isField(f) && f.confidence === "user_edited" &&
!have.has(valueKey(f.value)));
    return [...fresh, ...edits];
  }
  // …objects: recurse into each key…
  return fresh;
}
```

It walks both knowledge bases side by side through the content sections only (`CONTENT_KEYS`: company, contact, customers, brand, people, offerings, insights, contentKit):

- a **user-edited field** wins over the fresh value
- **user-edited list items** are appended, unless the fresh list already has the same value
- everything else comes from the fresh scrape

Survives a re-scrape?	Answer	Why
Values the owner typed or picked (`user_edited`)	✅ yes	`overlay` keeps them
Menu items marked reviewed (now `user_edited`)	✅ yes	same
Accepted AI values (`ai_live`)	❌ no, replaced by the fresh scrape	only `user_edited` is protected; run Enrich again (free if cached)
`consent`	✅ yes	`merged.consent = fresh.consent ?? previous.consent`
`dismissed` ("Wrong? Remove" memory)	❌ **no**	not copied: the fresh scrape starts with `[]`

Survives a re-scrape?	Answer	Why
`notApplicable`	❌ **no**	not copied either
Uploads list	❌ no	not in `CONTENT_KEYS`

Could be simpler / a real bug to fix: `keepUserEdits` should also carry `dismissed` and `notApplicable` from the previous version. Today, an owner who marked "No head office" or removed a wrong AI channel has to do it again after every re-scrape. And the scraper itself never checks `dismissed`, so a removed item the site still shows comes right back. The fix is small: copy both arrays in `keepUserEdits` (`overlay.ts`), skip dismissed values in `setField` / `addItem`, and add a test (there's no `overlay.test.ts` yet).

Another gap: re-scrape calls `scrapeUrl(prev.data.url)` **without** `ownerConsent` (`useRecordActions.ts:59`), even when the record has `consent.method === "checkbox_scrape"`. A site that needed the owner's permission (robots.txt block) will fail the re-scrape with `BLOCKED_ROBOTS`. Dig deeper gets this right, since it reads `kb.consent` (`src/lib/scraper/index.ts`, `digDeeper`). Passing `{ ownerConsent: prev.data.consent?.method === "checkbox_scrape" }` would fix it.

Note on "Wrong? Remove" and scraped values: the button only appears on AI and inferred values (`isAi`, `src/lib/utils/fields.ts:21`). A *scraped* value can be edited or deleted like any field, but that deletion goes through `removeItem` / `setField`, which records nothing in `dismissed`. So a scraped value the owner deletes comes back on the next re-scrape, even after the fix above.

Check yourself

1. When does Save send POST and when PATCH? What extra field does PATCH carry, and why?
2. Who decides the new version number, the browser or the database, and why?
3. What does "Restore version 2" do when the record is at version 5?
4. Why does `/knowledge/view` keep its filters in the URL?
5. Why does the list query read `logos:data->brand->logos` instead of `data`?
6. After a re-scrape, which of these survive: a typed founding year, an accepted AI industry, a dismissed AI channel, a "Not applicable" main address?
7. What goes wrong when re-scraping a site that needed the owner's consent, and how would you fix it?

Answers

1. POST for the first save (no `saved` id yet), PATCH afterwards. PATCH carries `expectedVersion`, so the database can refuse (409) if someone saved a newer version in between.
2. The database, inside `update_knowledge_base` with the row locked, so two saves can never both become the same version.
3. It loads version 2's snapshot and saves it as **version 6** with the note "Restored from version 2". Versions 3–5 stay in history.
4. So any view can be bookmarked, shared or reached with the back button, and the page can rebuild the exact view from the URL alone.
5. The full JSON is large. The JSON path pulls just the small logo list for the card or row thumbnail.

6. Only the typed founding year (`user_edited`). The AI industry is replaced by the fresh scrape, and `dismissed` and `notApplicable` aren't carried over by `keepUserEdits`.
7. `rescrape` calls `scrapeUrl` without `ownerConsent`, so robots.txt blocks it again (`BLOCKED_ROBOTS`). Pass `ownerConsent: true` when the previous record's `consent.method` is `checkbox_scrape`.

09 · Testing and deploying

What you'll learn

1. How the tests are organized, how to run them, and three real tests explained line by line.
2. Every environment variable: what it does, local vs Vercel, secret or public.
3. How a push becomes a live deploy, the 60-second limit, where logs are, and the commit author problem we hit.

Before this: 01-tech-stack.md (Vitest, env vars, Vercel)

---

1. How tests are organized

- **Runner: Vitest**. `npm test` runs `vitest run` (`package.json` scripts).
- **Config:** `vitest.config.mts`, which finds `src/**/*.test.ts`, runs in a Node environment, and copies the `@/` alias so tests import like the app does.
- **Location:** each test sits **next to the file it tests**: `business-facts.ts` and `business-facts.test.ts`.
- **What's tested: pure logic** — parsers, rules and filters. UI components aren't tested; they're checked by hand in the browser.
- **Size:** 15 test files and **144 tests** (last full run, after the Dig deeper batches commit).

Area	Test files
Scraper parsing	`address.test.ts` (16), `extract/business-facts.test.ts` (14), `extract/menu-sources.test.ts` (8), `focus.test.ts` (6)
Menus	`menus/parse.test.ts` (15), `menus/organize.test.ts` (11), `menus/group.test.ts` (3)
AI rules	`ai/confidence.test.ts` (7), `ai/menus.test.ts` (7), `ai/read-menus.test.ts` (7), `ai/field-tiers.test.ts` (5), `ai/apply.test.ts` (4), `ai/heuristics.test.ts` (3)
Utils	`utils/offerings.test.ts` (3), `utils/quick-picks.test.ts` (3)

(The numbers count `it(...)` / `it.each(...)` blocks. An `it.each` with 12 rows runs 12 tests, which is why the total is 144.)

The four checks to run after any change (README "Checks"):

```
npx tsc --noEmit   # type check (no files written)
npm test           # unit tests
npm run lint       # ESLint
npm run build      # the real production build
```

Why test pure functions and not the UI? Most of the risk in XenFlo is in *judgment*: is this a price? whose phone is this? is this citation real? Those are pure functions: same input, same output, no network. They're cheap to test with made-up examples. Everything the tests use is **fictional** ("Sunny Paws Grooming LLC", `harborgroup.example`), so no real business data ends up in the repo.

---

2. Test 1: a table-driven parser test

From `src/lib/scraper/extract/business-facts.test.ts`:

```
import { describe, expect, it } from "vitest";
import { parseCopyright, parseEmployeeCount, parseLegalEntity } from "./business-facts";

describe("parseCopyright", () => {
  it.each([
    ["© 2024 Sunny Paws Grooming LLC. All rights reserved.", "Sunny Paws Grooming LLC", "LLC"],
    ["Copyright 2025 Red Rock Plumbing, Inc. All rights reserved.", "Red Rock Plumbing, Inc.",
"Corporation"],
    ["© 2026 by Neon Noodle Bar LLC.", "Neon Noodle Bar LLC", "LLC"],
    // … 9 more rows …
  ])("%s", (text, legalName, entityType) => {
    expect(parseCopyright(text)).toMatchObject({ legalName, entityType });
  });

  it("ignores the site builder or theme", () => {
    expect(parseCopyright("© 2024 Wix.com Ltd. Proudly created with Wix")).toBeNull();
  });
});
```

Line by line:

- `describe("parseCopyright", …)` groups related tests under one name in the output.
- `it.each([...rows])("%s", fn)` is a **table-driven test**: the same check runs once per row. `%s` puts the first value (the footer text) in the test's name, so a failure tells you exactly which footer broke.
- `(text, legalName, entityType) =>` destructures each row's three values.
- `expect(x).toMatchObject({...})` passes if `x` *contains* these keys with these values. Extra keys (like `quote`) are fine.
- `toBeNull()`: for a platform name, the function must return nothing at all.

Adding a new case is one new row, and that's the point of the style.

---

3. Test 2: mocking the network, the database and OpenAI

`src/lib/ai/read-menus.test.ts` tests `readMenusWithAi`, a function that normally downloads PDFs, calls OpenAI and touches Supabase. Doing that for real in tests would be slow, cost money and depend on the internet. So the test replaces those modules with **mocks** (fake versions the test controls):

```
// src/lib/ai/read-menus.test.ts:9-18 (trimmed)
const cache = new Map<string, unknown>();
const state = { quota: true, calls: 0 };

vi.mock("@/lib/db/ai", () => ({
  getCachedEnrichment: async (k: string) => cache.get(k) ?? null,
  putCachedEnrichment: async (k: string, v: unknown) => void cache.set(k, v),
  takeAiQuota: async () => state.quota,
  aiQuotaRemaining: async () => 5,
}));
vi.mock("./openai", () => ({
  callOpenAi: async () => {
    state.calls++;
    return {
      // … data: an answer with two items, "Dragon Roll" at $14.50 and "Miso Soup" with no price …
      usage: { input: 2500, output: 300 },
    };
  },
}));
```

- **`vi.mock(path, factory)`**: whenever any code imports `@/lib/db/ai`, it gets this object instead. The cache becomes a plain `Map`, the quota is whatever `state.quota` says, and OpenAI returns a fixed answer and counts how many times it was called.
- **`const { readMenusWithAi } = await import("./read-menus")`** (after the mocks): the module under test is imported *after* `vi.mock`, so it picks up the fakes.
- **`beforeEach`** resets `cache`, `state.quota = true` and `state.calls = 0` before every test, so tests can't affect each other.

The test added for the Neko Hana fix (chapter 06):

```
  it("names a menu that couldn't be read again, and replaces nothing", async () => {
    const kb = kbWith(["sakana"]);
    const done = { ...kb, crawl: { ...kb.crawl, menuSources: kb.crawl.menuSources!.map((s) => ({ ...s,
status: "read_ai" as const, readAs: "picture" as const })) } };
    state.quota = false;
    const r = await readMenusWithAi(done, { only: [`${SITE}/files/sakana.pdf`], fresh: true });
    expect(state.calls).toBe(0);
    expect(r.mode).toBe("unavailable");
    expect(r.replaces).toEqual([]);
    expect(r.notes).toContain("sakana wasn't read again: today's live AI limit is used up. Its current
items are unchanged.");
  });
```

1. **Arrange:** a knowledge base with one menu already read by AI. The quota is turned **off**.
2. **Act:** "Read again" on that menu (`only` + `fresh`).
3. **Assert:**
    - no AI call was made (`calls` is 0)
    - mode is `unavailable`
    - nothing was replaced
    - the note names the menu

This **Arrange, Act, Assert** shape is worth copying.

---

4. Test 3: environment variables, restored afterwards

`src/lib/scraper/focus.test.ts` checks `maxCrawlPages()`, which reads `process.env.MAX_CRAWL_PAGES`:

```
describe("maxCrawlPages", () => {
  const original = process.env.MAX_CRAWL_PAGES;
  afterEach(() => {
    if (original === undefined) delete process.env.MAX_CRAWL_PAGES;
    else process.env.MAX_CRAWL_PAGES = original;
  });

  it("reads the env var, kept within 15 to 500", () => {
    process.env.MAX_CRAWL_PAGES = "50";
    expect(maxCrawlPages()).toBe(50);
    process.env.MAX_CRAWL_PAGES = "3";
    expect(maxCrawlPages()).toBe(15);
    process.env.MAX_CRAWL_PAGES = "9999";
    expect(maxCrawlPages()).toBe(500);
  });
});
```

- `process.env` **is shared** by every test in the run. If one test sets it and doesn't put it back, a *different* test can fail mysteriously later.
- `afterEach` runs after every test in the block and restores the original value. It deletes the variable if it wasn't set, because setting it to `"undefined"` would be a different value.
- **The checks cover the edges:** a normal value, below the minimum (clamped to 15) and above the maximum (clamped to 500). **Edge cases** are where bugs live.

---

5. Environment variables

From `.env.example` and README's deploy table:

Variable	What it does	Secret?	Local (`.env.local`)	Vercel
`NEXT_PUBLIC_SUPABASE_URL`	the Supabase project URL	public	set	set
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	public key (**unused by the code today**; see chapter 01)	public	optional	set
`SUPABASE_SERVICE_ROLE_KEY`	server key, bypasses RLS	**secret**	set	set (Sensitive)
`OPENAI_API_KEY`	live AI	**secret**	optional	Production only
`AI_PASSCODE`	the passcode users type for live AI	**secret**	optional	Production only

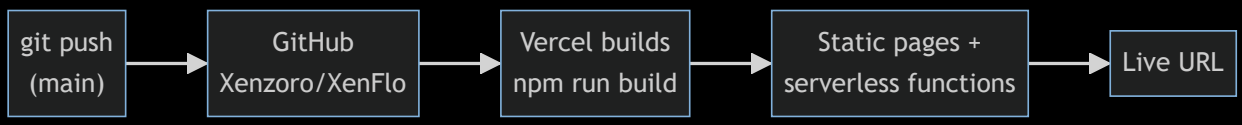
Variable	What it does	Secret?	Local (`.env.local`)	Vercel
`AI_MODEL_TEXT`, `AI_MODEL_VISION`	model names (default `gpt-5.4-mini`)	no	optional	optional
`AI_DAILY_LIMIT`	live AI runs per UTC day, all users (default **20**)	no	Jake set **100** after the Neko Hana debugging	raised on Vercel
`MAX_CRAWL_PAGES`	Dig deeper total cap (default 200, clamped 15–500)	no	optional	optional
`SCRAPER_USER_AGENT`	override the bot name (default `XenFloBot/1.0 …`)	no	optional	optional

Rules worth remembering:

- **Never commit `.env.local`.** Keep `.env.example` updated with blank values (CLAUDE.md).
- **On Vercel, changing a variable needs a redeploy** to take effect (README).
- **Preview deploys** (from branches) leave the OpenAI key and passcode empty, so they run in preview mode and can't spend money.
- **Local and Vercel can differ**, and that confused debugging once (chapter 06): the local limit was 20 while Vercel's was higher.

---

6. From `git push` to a live site



1. **Vercel watches the GitHub repo** (`origin` is `github.com/Xenzoro/XenFlo`). A push to `main` makes a **production** deploy; other branches make **preview** deploys.
2. **It runs `npm run build`.** If the build fails (a TypeScript error, for example), the old version stays live. That's why the working rule is "run the build after each major step".
3. **Pages become static files**, and each `src/app/api/*/route.ts` becomes a **serverless function**.
4. `maxDuration = 60` in the scrape, deeper, enrich and menus routes asks for the Hobby plan's maximum of 60 seconds. Every internal budget is set to finish inside that:
   - crawl: 30 s
   - menu PDFs: 15 s
   - each AI call: 25 s
   - AI menu reads: none start after 40 s
5. **Logs:** `console.error` on the server (for example in `errorResponse`, `src/lib/utils/http.ts:44-52`) shows up in Vercel → Project → **Logs**. Secrets are never logged (CLAUDE.md rule).

---

7. The commit author problem we hit

What happened. Git stamps every commit with an author name and email (`git config user.name` / `user.email`). In WSL, nobody had set them at first, so git used the machine's default identity: `xen <xen@PC-XEN.localdomain>`. The history still shows **47 commits** with that author and 25 with the GitHub noreply address (`git log --format="%an <%ae>"`).

Why it mattered. Vercel checks who authored the commit it's asked to deploy. `xen@PC-XEN.localdomain` isn't linked to any GitHub account, so **Vercel blocked the deploys**.

The fix:

1. Set the email to the GitHub **noreply** address (GitHub gives every account one, so your real email stays private):

```
git config user.email "157854543+Xenzoro@users.noreply.github.com"
```

2. Push a new commit so Vercel deploys again. An **empty** commit is enough:

```
git commit --allow-empty -m "Trigger deploy"
git push
```

3. **Pushing** needed `gh auth login` (the GitHub CLI), because GitHub no longer accepts account passwords for git over HTTPS. You sign in once, and `gh` stores a token that git uses.

Lesson: on a new machine, set `user.name` and `user.email` *before* the first commit (`git config --global …`), and use the noreply address if you want privacy.

Check yourself

1. Where do test files live, and what does `vitest.config.mts` add so imports work?
2. What does `it.each` do, and why put `%s` in the name?
3. Why does `read-menus.test.ts` import the module *after* calling `vi.mock`?
4. Why must a test that changes `process.env` restore it in `afterEach`?
5. Which env vars are secrets, and which should be left empty on preview deploys?
6. A route needs 70 seconds on Vercel Hobby. What happens, and how does XenFlo avoid it?
7. Why did Vercel refuse to deploy, and what three steps fixed it?

Answers

1. Next to the code they test (`*.test.ts`). The config sets the `@/` → `src/` alias and the Node environment.
2. It runs the same test once per row of a table. `%s` puts the row's first value in the test name, so failures say which input broke.
3. `vi.mock` must be registered before the module (and its imports) loads, so the module gets the fakes instead of the real database, OpenAI and fetch.
4. `process.env` is shared across tests, and a leftover value can make unrelated tests fail in confusing ways.
5. Secrets: `SUPABASE_SERVICE_ROLE_KEY`, `OPENAI_API_KEY`, `AI_PASSCODE`. Leave `OPENAI_API_KEY` and `AI_PASSCODE` empty on preview deploys.
6. The function is cut off at 60 s and the request fails. XenFlo's internal budgets (30 s crawl, 15 s menus, 25 s per AI call, 40 s menu deadline) stop new work early enough to answer in time.
7. Commits were authored as `xen@PC-XEN.localdomain`, which isn't linked to a GitHub account. The fix: set `git config user.email` to the GitHub noreply address, push an empty "Trigger deploy" commit, and use `gh auth login` because GitHub doesn't accept passwords for git.

10 · Glossary

> **What you'll learn**
>
> 1. Every technical term used in this guide, in plain words.
> 2. Where each term matters in XenFlo (chapter number in brackets).
> 3. Use this as a lookup page while reading the other chapters.

`addItem`: The helper that adds a value to a list field unless an equal one is already there. "Equal" is decided by a key function (digits for phones, number + unit + ZIP for addresses). [04]

Adaptive crawl: After the priority pages, only crawling pages likely to fill fields that are still empty, re-checking after each small batch. [03]

Advanced view: The toggle that reveals the Insights, Content Kit, Sources and Raw JSON tabs, plus a confidence badge on every field. [07]

AI preview / `ai_mock`: Confidence for template-made suggestions when live AI isn't available. Shown with a purple "AI preview" badge. [02, 05]

`ai_live`: Confidence for values returned by a real AI model. [02]

API route: A `route.ts` file under `src/app/api` that answers HTTP requests on the server, with no UI. [01]

App Router: Next.js's folder-based routing: the folder path under `src/app` becomes the URL. [01]

Arrange, Act, Assert: A test structure: set up the situation, run the code, check the result. [09]

async / await: Syntax for working with promises. `await` pauses the current function until a result is ready without freezing the program. [01]

`AnimatePresence`: A Framer Motion component that lets elements animate *out* before React removes them. [01, 07]

Badge: The small colored label next to a value showing its confidence (Scraped, Inferred, AI, User edited, Missing). [02, 07]

Base64: A way to write binary data (like a PDF) as plain text, used to send menu PDFs to OpenAI inside JSON. [06]

Bot challenge: A "Just a moment..." page from services like Cloudflare that blocks automated visitors. XenFlo treats it as `BLOCKED_ACCESS`. [03]

Cache / cache key: A stored answer reused for identical input. The key is a hash of everything that affects the answer (prompt text, model, input). [05, 06]

Cascade (delete): A database rule: deleting a parent row deletes its children too (versions, crawl runs, consents). [02, 08]

Cheerio: A server-side library that loads HTML and lets you query it with CSS selectors, without running the page's JavaScript. [01, 04]

Client component: A React component whose code ships to the browser (marked `"use client"`), so it can use state and clicks. [01]

Completeness / Knowledge Health: The 0–100 score from weighted checks, shown as the gauge. [07]

Concurrency: How many tasks run at the same time. The crawler uses 2 workers. [03]

Confidence: How a value got into the knowledge base: scraped, inferred, ai_mock, ai_live, user_edited or missing. [02]

Confidence bar: XenFlo's code-level rule for accepting AI answers: facts need high confidence plus 2 real citations; generated writing needs at least 1. [05]

Consent: The owner's checkbox confirmation ("I own this business or have permission…"), stored with a timestamp and method. [03, 02]

Constant-time compare: Comparing secrets in a way that always takes the same time, so timing can't leak how close a guess was (`timingSafeEqual`). [05]

Context (React): A way to share state with a whole component subtree without passing props through every level. [07]

CORS: A browser security rule that stops a web page from reading responses from other sites. It's why scraping must happen on the server. [01]

Crawl budget: The time limit for a crawl (30 s), after which no new pages start. [03]

`Crawl-delay`: A robots.txt line asking bots to wait between requests. XenFlo honors it, capped at 3 s. [03]

CSS variable: A named value in CSS (`--color-primary`). XenFlo resolves them to find real brand colors and fonts. [04]

CTA (call to action): A button or link asking the visitor to act ("Book now", "Order online"). [04]

Design tokens: Named design values (colors, radius, shadows) defined once in `globals.css` and used everywhere. [01]

Dig deeper: Continue a crawl in batches of 15 pages, up to `MAX_CRAWL_PAGES`. [03]

Dirty: Having unsaved changes. Drives the Save button and the "leave page?" warning. [07]

Dismissed (`kb.dismissed`): The list of values the owner removed with "Wrong? Remove", so AI won't suggest them again. [07, 08]

DNS / DNS rebinding: DNS turns names into IP addresses. Rebinding is an attack where a name resolves to a public IP for the safety check and a private IP for the real connection. [03]

Edge runtime: A lighter alternative to Node.js for Next.js routes. XenFlo uses `runtime = "nodejs"` because Cheerio and long downloads need full Node. [01]

Edge case: An input at the boundary of what's allowed (empty, maximum, minimum), where bugs often hide. [09]

Env var (environment variable): A setting passed to the program from outside the code (`.env.local` locally, the dashboard on Vercel). [01, 09]

Evidence: The short quotes or page items a value is based on, stored on the field and shown in the badge tooltip. [02, 05]

Extractor: A function that reads one kind of fact from a page and writes it into the knowledge base (`(ctx) => void`). [04]

`Field<T>`: XenFlo's wrapper for every value: `{ value, source, confidence, updatedAt, evidence?, reviewedAt? }`. [02]

`FieldList<T>`: A list of `Field<T>`, so each item carries its own source and confidence. [02]

Few-shot example: Showing the model a sample input and output inside the prompt to steer its format. [05]

`filterByTier`: The code guard that drops AI suggestions for tier 3 fields, unquoted tier 1 fields, dismissed values and N/A fields. [02, 05]

Framer Motion: A React animation library (`motion.div`, `AnimatePresence`, springs). [01, 07]

Generic (TypeScript): A type parameter like `<T>` that lets one definition work for many types (`Field<number>`, `Field<Address>`). [02]

Hash: A short fixed-length fingerprint of data (SHA-256). The same input always gives the same hash. [05]

Heuristic: A rule of thumb in code (a regex, a keyword list) that's usually right but not guaranteed. [04, 05]

Hook: A React function starting with `use` (`useState`, `useEffect`, custom ones like `useKnowledgeActions`) that adds state or behavior to a component. [01]

Inferred: Confidence for values derived by rules from scraped data, like a name from a logo file name. [02]

`input_file` / `input_image`: OpenAI Responses API content types for sending a whole file (a PDF) or an image to the model. [06]

JSON: The text format for data on the wire: `{"key": "value"}`. [01]

JSON-LD: Structured data in a `<script type="application/ld+json">` block that sites add for search engines. It's the most reliable source when present. [04]

JSON Schema: A standard way to describe a JSON shape. XenFlo converts Zod schemas to it for OpenAI's strict mode. [05]

JSONB: Postgres's binary JSON column type. It stores the whole knowledge base in one column. [02]

Knowledge base: The one object holding everything XenFlo knows about a business (`KnowledgeBase` type). [02]

`maxDuration`: The maximum seconds Vercel lets a function run (60 on the Hobby plan). [01, 09]

Menu source: A tracked menu file (PDF or image) with a status like `found`, `read`, `no_text` or `read_ai`. [06]

Migration: A SQL file that changes the database structure, applied in order by date. [02]

Mock (testing): A fake version of a module (database, OpenAI) that a test controls (`vi.mock`). [09]

Noreply email: The private email GitHub gives every account, so commits don't reveal a real address. [09]

`notApplicable`: Field paths the owner marked N/A. They count as complete in the score. [07, 08]

Optimistic concurrency: Assume no one else is editing, but check the version at save time and refuse (409) if it changed. [02, 08]

Overlay (re-scrape): Putting the owner's edits back on top of a fresh scrape (`keepUserEdits`). [08]

PageContext: The object every extractor receives: the page in two Cheerio forms, the text, lines, URL, category and the knowledge base. [04]

Passcode: A shared secret users type to use live AI on the public demo. [05]

Pool (link pool): All discovered but not-yet-crawled links, keyed by `pageKey`. [03]

Postgres: The SQL database Supabase hosts. [01, 02]

PostgREST: The tool Supabase uses to turn Postgres into an HTTP API. It's also where the `PTxxx` error-code convention comes from. [02]

Priority pages: The best link from each useful category (about, services, pricing…), crawled first for breadth. [03]

Promise: An object for "a value that will exist later". [01]

`Promise.all` **/** `Promise.allSettled`: Run several promises at once. `all` fails if any fails; `allSettled` reports each result. [01, 05]

Prompt: The instructions sent to a language model. XenFlo's are versioned `.md` files in `prompts/`. [05]

Pure function: A function whose output depends only on its input, with no network or side effects. It's easy to test. [06, 09]

Quota (daily AI quota): The cap on live AI runs per UTC day (`AI_DAILY_LIMIT`), counted atomically in `ai_usage`. [05]

Rank (confidence rank): The order deciding which value wins: user_edited > scraped > ai_live > inferred > ai_mock > missing. [02, 04]

Redirect: An HTTP response (3xx) saying "go to this other URL instead". XenFlo checks every hop for SSRF. [03]

Regex (regular expression): A pattern for matching text, like the phone or address patterns. [04]

Responses API: OpenAI's HTTP endpoint for model calls (`/v1/responses`). [01, 05]

RLS (Row Level Security): A Postgres feature where the database checks, per row, whether the current user may read or change it. [02]

robots.txt: A file where site owners say which bots may read what. Polite crawlers follow it. [03]

RPC (remote procedure call): Calling a Postgres function over the API (`supabase.rpc("update_knowledge_base", …)`). [02]

Scraped: Confidence for values read directly from the website. [02]

Scraper: The server code that downloads pages and extracts facts. [00, 03]

Schema: A description of the allowed shape of data. [01, 02]

Serverless function: A small program the host starts on demand for each request (each API route on Vercel). [01]

Service role key: Supabase's secret server key that bypasses RLS. It must never reach the browser. [02]

`setAt` **/** `getAt`: Helpers to read and write nested values by dot path ("company.yearFounded"), never mutating. [07]

`setField`: The helper that writes a scalar field only if the new value outranks the current one. [04]

Sitemap: An XML file listing a site's pages for search engines. XenFlo reads it during discovery. [03]

SSRF (Server-Side Request Forgery): Tricking a server into fetching a URL the attacker chose, often a private one. [03]

Strict mode (TypeScript): All type checks on, especially null checks. [01]

Structured outputs (`strict: true`): OpenAI mode where the model can only answer with JSON matching the given schema. [05]

Supabase: Hosted Postgres plus an API, Storage and auth. [01]

Tailwind CSS: Styling with small utility classes in the markup (`rounded-2xl p-6`). [01]

Tier (field tier): Who may fill a field: 1 read (stated facts), 2 inferred, 3 never guessed. [02]

Token: The unit language models count text in (about 4 characters). Budgets and costs are in tokens. [05]

Transaction: A group of database writes that all succeed or all fail together. [02]

Trigram index: A Postgres index that makes `ILIKE '%term%'` search fast. [02, 08]

TypeScript: JavaScript with types, checked before the code runs. [01]

unpdf: A pure-JavaScript PDF text extractor (a serverless build of pdf.js). [01, 06]

`user_edited`: Confidence for values the owner typed or changed. It's the highest rank and is never overwritten. [02]

User agent: The name a program sends with each request. XenFlo's is `XenFloBot/1.0 (knowledge base builder)`. [03]

UTC: Coordinated Universal Time. The AI quota resets at midnight UTC. [05]

UUID: A random unique id (`04f2d85d-…`) used for records. [02]

Vercel: The hosting platform that builds and serves the app. [01, 09]

Version (knowledge base): Each save creates a numbered snapshot in `knowledge_versions`. [02, 08]

Vision model: An AI model that can read images and PDFs (logos, menu pictures). [05, 06]

Vitest: The test runner (`npm test`). [01, 09]

Worker pool: A fixed number of workers pulling tasks from one shared queue. [01, 03]

"Wrong? Remove": The button on AI and inferred values that removes them and remembers the removal. [07]

Zod: A library that describes data shapes and checks real data against them at runtime. [01, 02]

Check yourself

1. In one sentence each: SSRF, RLS, JSONB.
2. What's the difference between *confidence* and *tier*?
3. What's the difference between a client component and a serverless function?
4. Why does a hash make a good cache key?
5. What does "pure function" mean, and why are those the easiest to test?

Answers

1. SSRF: tricking a server into fetching a URL the attacker picked, often a private one. RLS: the database checks per row whether the current user may access it. JSONB: Postgres's binary JSON column type, which holds the whole knowledge base.
2. Confidence is how a value got there; tier is whether AI may fill that field at all.
3. A client component is UI code that runs in the browser. A serverless function is server code (an API route) that the host starts on demand.
4. Identical input always gives the identical hash, and any change, even one character of the prompt, gives a different one.
5. Its output depends only on its input, with no network, database or clock. Tests can call it with made-up input and check the output directly.

11 · Simplify later: every "could be simpler" note, ranked

What you'll learn

1. Every complexity, duplication, dead code and real bug the guide flagged, in one list.
2. Which ones to do first: high impact and low effort at the top.
3. The files involved in each, so you can plan the work.

How to read the ranking

- **Impact:** how much it helps owners or prevents wrong data. **High** means a real bug or wrong data; **Med** means a confusing or annoying UX; **Low** means cleanliness only.
- **Effort: S** is under an hour with a test, **M** is a few hours, **L** is a day or more.
- The order is roughly *impact ÷ effort*.

The list

#	What	Kind	Impact	Effort	Files	Chapter
1	**Re-scrape forgets `dismissed` and `notApplicable`.** Owners redo "Wrong? Remove" and "Not applicable" after every re-scrape.	bug	High	S	`src/lib/utils/overlay.ts` (`keepUserEdits`), add `overlay.test.ts`	08
2	**Re-scrape doesn't pass the owner's consent.** A robots-blocked site that was scraped with permission fails `BLOCKED_ROBOTS` on re-scrape (Dig deeper gets it right).	bug	High	S	`src/components/view/useRecordActions.ts:59`	08
3	**The scraper ignores `kb.dismissed`.** A removed value the site still shows comes back on re-scrape and Dig deeper.	bug	High	M	`src/lib/scraper/merge.ts` (`setField` / `addItem` would need the dismissed list), `index.ts`	07, 08
4	**Deleting a scraped value records nothing.** There's no "Wrong? Remove" for scraped values, and `removeItem` doesn't add to `dismissed`, so it returns on the next scrape.	design gap	Med	S	`src/context/KnowledgeContext.tsx` (`removeItem`), `src/components/ui/AiValueTag.tsx`	07, 08
5	**The legal links regex is too loose.** No word boundaries: "illegal blocks" and many ordinary links land in `insights.legalLinks` (Goettl had 17).	data quality	Med	S	`src/lib/scraper/extract/contact.ts:56-62`	04

#	What	Kind	Impact	Effort	Files	Chapter
6	**Menu headline duplicated.** v2 returned both "OMAKASE AYCE $58.95" and "OMAKASE ALL YOU CAN EAT 16 COURSE EXPERIENCE $58.95". `dedupe` only merges identical names.	data quality	Med	M	`src/lib/scraper/menus/organize.ts` (`dedupe`), or the menu-reader prompt	06
7	**Badge tooltips don't work on touch screens.** `title` attributes never show on iPad or phones, so evidence is invisible on mobile.	UX	Med	M	`src/components/ui/Badge.tsx:43-53`	07
8	**Three different "same value?" rules.** `merge.ts` key functions, `overlay.ts` `valueKey` (JSON-encodes) and `field-tiers.ts` `valueKey` (doesn't JSON-encode strings).	duplication	Med	M	`merge.ts`, `overlay.ts`, `field-tiers.ts`	04
9	**Articles slip past the focus filter.** `/expert-center/…solution` was categorized as *services* because the slug contains "solution".	data quality	Low	S	`src/lib/scraper/discover.ts` (`CONTENT_HUB` could add `expert-center	resources
10	**Leftover cap in the scrape route.** `maxPages` is still `max(30)` from before `MAX_CRAWL_PAGES`. Harmless (the UI never sends it), but inconsistent.	dead code	Low	S	`src/app/api/scrape/route.ts:13`	01
11	**Unused env var.** `NEXT_PUBLIC_SUPABASE_ANON_KEY` is listed and set on Vercel but never read by code.	dead config	Low	S	`.env.example`, README deploy table	01, 09
12	**Reference-only prompt files look live.** Four of eight prompts are never loaded (merged into understand-business).	clarity	Low	S	`prompts/` → `prompts/reference/`, `prompts/README.md`	00
13	**The progress list isn't live.** `ScrapeProgress` shows a timed walkthrough; `onProgress` in `ScrapeOptions` is unused. Real progress needs a streamed response.	feature	Med	L	`src/lib/scraper/index.ts`, `src/app/api/scrape/route.ts`, `ScrapeProgress.tsx`	01
14	**The few-shot example never reaches the model.** `load()` cuts prompts at `## Example`. Test whether a short example improves answers enough to pay for the tokens.	experiment	?	M (needs live runs)	`src/lib/ai/prompts.ts:20-27`	05
15	**Types written twice.** `knowledge.ts` and `knowledge.schema.ts` describe the same shape (kept honest by the `Same<>` guard). Deriving types from Zod would mean one source.	design choice	Low	L	`src/types/*` and every import	02
16	**A small copied helper.** `titleCaseIfLower` in `menus/brands.ts` is a copy of one in `extract/text.ts`, on purpose, because `text.ts` loads Cheerio and `brands.ts` runs in the browser too. Moving the helper to a tiny shared file would remove the copy.	duplication	Low	S	`src/lib/scraper/menus/brands.ts:13-14`, `extract/text.ts`	06

Already fixed (in the Dig deeper batches commit `84dc743`): the page cap was hard-coded as 200 in `SourcesTab.tsx` while the server used 30. It now reads `kb.crawl.maxPages` from the server.

A suggested order

1. **One small PR for the re-scrape bugs (#1, #2):** copy `dismissed` and `notApplicable` in `keepUserEdits`, pass `ownerConsent` on re-scrape, and add `overlay.test.ts`. That's about an hour.
2. **Data quality (#5, #9):** the word-boundary and path check for legal links, and two more `CONTENT_HUB` words. Add a test row each.
3. **Dismissals end to end (#3, #4):** let the scraper skip dismissed values, and record deletions of scraped values. That needs a design choice: should deleting a scraped phone mean "never again", or "not now"?
4. **Cleanup (#10–#12, #16):** five-minute changes, nice before a demo.
5. **Bigger items (#7, #8, #13, #15):** only if there's time after the challenge.

Check yourself

1. Which two items would you fix first, and why?
2. Why is #3 harder than #1 even though both are about `dismissed`?
3. What design question does #4 raise?
4. Why is #16 a copy in the first place?

Answers

1. #1 and #2: both are real bugs that make owners redo work or block a re-scrape, and each is a few lines plus a test.
2. #1 only copies two arrays during the overlay. #3 needs the scraper's write helpers to know about dismissals: a new parameter through `setField` / `addItem`, and matching value keys (see #8).
3. Whether deleting a scraped value means "this is wrong, never bring it back" (record a dismissal) or just "remove it for now" (current behavior). The two need different UI wording.
4. `extract/text.ts` imports Cheerio, which is server-heavy, and `menus/brands.ts` also runs in the browser (via `organizeMenus`), so importing from `text.ts` would pull Cheerio into the browser bundle.